Pruebas de Integración en Arquitecturas Basadas en Mensajes y Eventos

Sistema de procesamiento de pedidos con RabbitMQ y Apache Kafka: montaje, ejecución de los casos CP-01 a CP-12 y evidencia técnica

Integrantes	Santiago Betancur Diego Alzate Carlos Mendez Heyner Mena
Fecha de ejecución de las pruebas	30 de octubre de 2026
Fecha de entrega	1 de noviembre de 2026
Código fuente	https://github.com/Xantyagoh/pruebas-integracion-mensajes-eventos · también se entrega `pruebas-integracion-mensajes-eventos.zip` (código fuente y `docker-compose.yml` de cada arquitectura)

Entorno de ejecución	Versión
Sistema operativo	Windows 11 Pro 10.0.26200
Docker Engine / Docker Compose	29.4.0 / v5.1.1 (Docker Desktop)
RabbitMQ (imagen `rabbitmq:4-management`)	4.4.0 sobre Erlang 29.1.2
Kafka (imagen `bitnamilegacy/kafka:3.7`, modo KRaft)	3.7.1
Kafka UI (imagen `provectuslabs/kafka-ui`)	v0.7.2
Servicios (API y consumidores)	Node.js 22 (node:22-alpine), amqplib 0.10.9, kafkajs 2.2.4

Contenido

1. Resumen de resultados
2. Implementación mínima
3. Montaje del entorno y evidencias de arranque
4. Ejecución de pruebas: RabbitMQ
5. Ejecución de pruebas: Kafka
6. Conclusiones
7. Anexo: reproducción y archivos de evidencia

1. Resumen de resultados

Se implementaron dos sistemas independientes de procesamiento de pedidos, uno basado en mensajes (RabbitMQ) y otro basado en eventos (Kafka). Cada componente corre en su propio contenedor y se levanta con Docker Compose. Los casos se ejecutaron con los scripts `rabbitmq/pruebas.sh` y `kafka/pruebas.sh`, que guardan la salida real de cada paso en `evidencias/`. Las capturas de la consola de RabbitMQ y de Kafka UI se tomaron con el navegador durante esa misma ejecución.

ID	Caso / propiedad	RabbitMQ	Kafka
CP-01	Creación de pedido vía API	**Cumple**	**Cumple**
CP-02	Envío de mensaje a RabbitMQ	**Cumple**	—
CP-03	Consumo de mensaje (RabbitMQ)	**Cumple**	—
CP-04	Publicación de evento en Kafka	—	**Cumple**
CP-05	Consumo de evento (Kafka)	—	**Cumple**
CP-06	Trazabilidad del flujo	**Cumple**	**Cumple**
CP-07	Responsive	**Cumple**	**Cumple**
CP-08	Message-driven	**Cumple**	**Cumple**
CP-09	Resilient	**Cumple**	**Cumple**
CP-10	Elastic	**Cumple**	**Cumple**
CP-11	Desacoplamiento temporal	**Cumple**	**Cumple**
CP-12	Recuperación	**Cumple**	**Cumple***

* En Kafka CP-12 no se perdió ningún evento, pero se observó un evento procesado dos veces después del reinicio del broker (entrega *al menos una vez*). Se documenta como hallazgo en el caso.

> **Nota sobre el puerto de la API.** En el equipo de pruebas el puerto 3000 ya estaba ocupado por otro proyecto. El puerto publicado de orders-api es configurable (`"${API_PORT:-3000}:3000"`): por defecto es 3000,

> como pide la guía, y en esta ejecución se fijó en 3010 mediante el archivo `.env`. Dentro de Docker la API sigue escuchando en el 3000. Para usar el 3000 basta con borrar el `.env`.

2. Implementación mínima

2.1 Arquitectura basada en mensajes (RabbitMQ)

Contenedor	Rol	Puertos	Detalles de implementación
orders-api	Productor	3010→3000 (HTTP)	`POST /orders`: valida el JSON, genera `orderId` (ORD-XXXXXXXX) y `correlationId` (cabecera `X-Correlation-Id` o generado), publica `ProcessOrder` en la cola `pedidos` (durable, mensaje persistente) con *publisher confirms* y responde 202 de inmediato. Reintenta la conexión al broker y se reconecta si se cae. `GET /health`.
rabbitmq	Broker	5672 (AMQP), 15672 (consola)	`rabbitmq:4-management`, usuario admin/admin, volumen de datos, `hostname` fijo, healthcheck, plugin `rabbitmq_tracing` para la bitácora de mensajes (CP-06) y estadísticas cada 1 s.
notification-worker	Consumidor	ninguno	Consume `pedidos` con `prefetch(1)` y ACK manual, simula el envío de la notificación (3 s) y confirma. Sin `container_name`, para poder escalarlo con `--scale`.

2.2 Arquitectura basada en eventos (Kafka)

Contenedor	Rol	Puertos	Detalles de implementación
orders-api	Productor de eventos	3010→3000 (HTTP)	`POST /orders`: valida, construye `OrderCreated` (eventId, eventType, eventVersion, occurredAt, source, correlationId, data con total) y lo publica en `orders.events` con key = orderId; responde 202. Registra la partición y el offset asignados.
kafka	Broker / log	9092 (interno), 29092 (host)	KRaft de un nodo; autocreación de tópicos desactivada; volumen `kafka-data`; healthcheck.
kafka-init	Inicialización	—	Contenedor de un solo uso que crea `orders.events` con 3 particiones (requisito para repartir carga) y termina.
kafka-ui	Observación	8080	Tópicos, particiones, mensajes y consumer groups.
inventory-consumer	Consumidor	ninguno	Grupo `inventory-service-group`: simula la reserva de inventario.
billing-consumer	Consumidor	ninguno	Grupo `billing-service-group`: simula la factura (quantity × unitPrice).

Contenedor	Rol	Puertos	Detalles de implementación
notification-consumer	Consumidor	ninguno	Grupo `notification-service-group`: simula la notificación al cliente.
audit-consumer	Consumidor posterior	ninguno	Perfil `late` (no arranca con el resto). Grupo `audit-service-group`; se usa para demostrar la lectura del log desde el inicio.

En ambas arquitecturas el productor no conoce a los consumidores, ningún consumidor expone HTTP y los servicios se conectan al broker por el nombre de servicio de Docker (`rabbitmq:5672`, `kafka:9092`), nunca por `localhost`. Como `depends_on` no garantiza que el broker esté listo, se combinan healthchecks con reintentos de conexión en el código.

3. Montaje del entorno y evidencias de arranque

3.1 RabbitMQ

Comando: `docker compose up -d --build` en `rabbitmq/`. Contenedores en ejecución, puertos expuestos, logs de arranque del broker (RabbitMQ 4.4.0, plugins y conexiones autenticadas de orders-api y del worker) y de los servicios, y cola `pedidos` creada:

Montaje del entorno RabbitMQ — evidencias/rabbitmq/entorno.txt

```
==================== Montaje del entorno RabbitMQ ====================
# Ejecutado: 2026-10-30 12:40:41 (hora local) / 17:40:41 UTC

$ docker compose down -v --remove-orphans 2>&1 | tail -n 3
 Network rabbitmq-pedidos_default Removing
 Volume rabbitmq-pedidos_rabbitmq-data Removed
 Network rabbitmq-pedidos_default Removed

$ docker compose up -d --build 2>&1 | grep -E 'Built|Started|Healthy|Created'
 Image rabbitmq-pedidos-notification-worker Built
 Image rabbitmq-pedidos-orders-api Built
 Network rabbitmq-pedidos_default Created
 Volume rabbitmq-pedidos_rabbitmq-data Created
 Container rabbitmq Created
 Container orders-api Created
 Container rabbitmq-pedidos-notification-worker-1 Created
 Container rabbitmq Started
 Container rabbitmq Healthy
 Container rabbitmq Healthy
 Container rabbitmq-pedidos-notification-worker-1 Started
 Container orders-api Started

$ docker compose ps --format 'table {{.Name}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}'
NAME                                    IMAGE                                   STATUS                   PORTS
orders-api                              rabbitmq-pedidos-orders-api             Up 1 second              0.0.0.0:3010->3000/tc
p, [::]:3010->3000/tcp
rabbitmq                                rabbitmq:4-management                   Up 8 seconds (healthy)   0.0.0.0:5672->5672/tc
p, [::]:5672->5672/tcp, 0.0.0.0:15672->15672/tcp, [::]:15672->15672/tcp
rabbitmq-pedidos-notification-worker-1   rabbitmq-pedidos-notification-worker   Up 1 second

# Puertos publicados en el host

$ docker port rabbitmq; docker port orders-api
5672/tcp -> 0.0.0.0:5672
5672/tcp -> [::]:5672
15672/tcp -> 0.0.0.0:15672
15672/tcp -> [::]:15672
3000/tcp -> 0.0.0.0:3010
3000/tcp -> [::]:3010

# Logs de arranque del broker

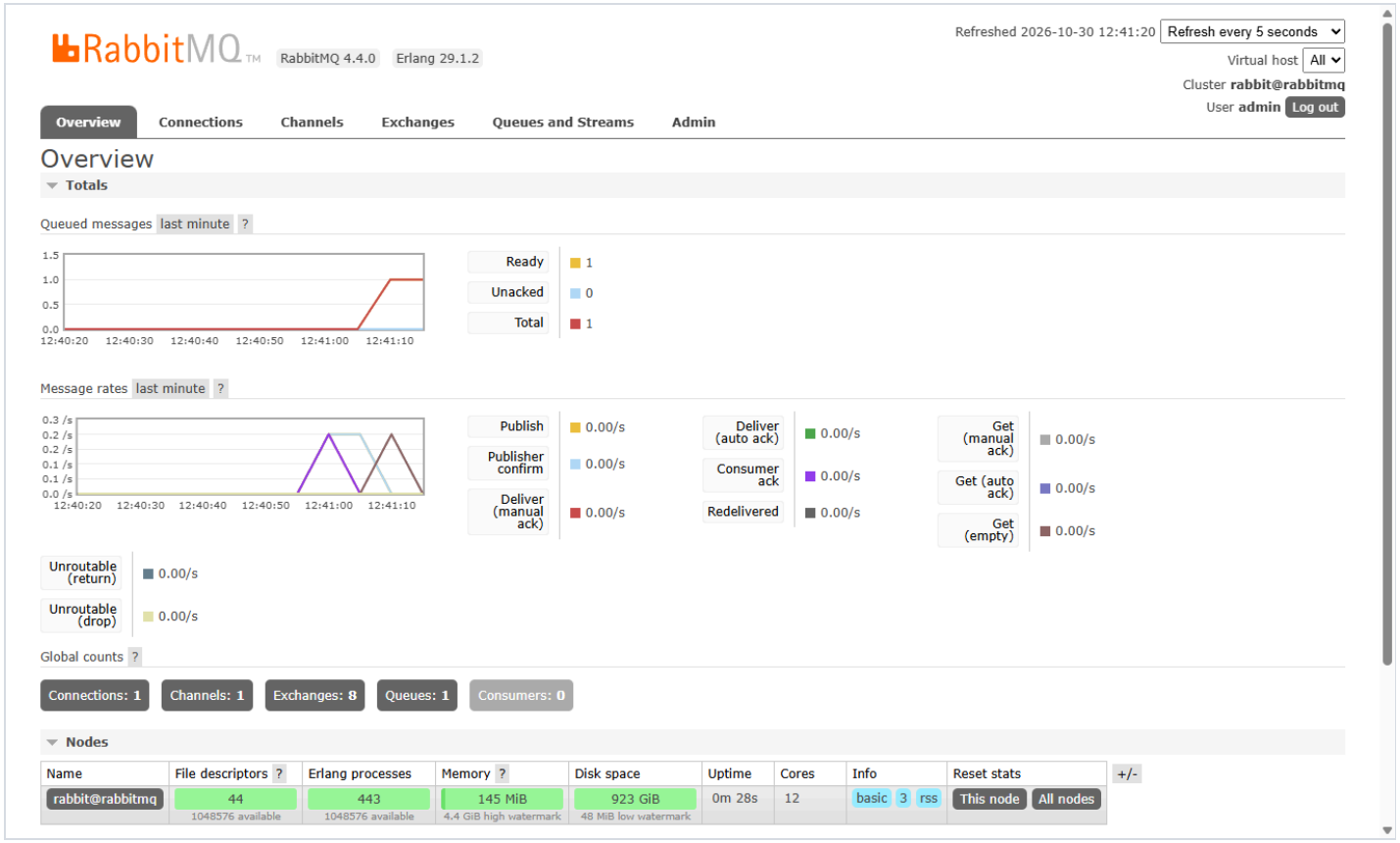
$ docker logs rabbitmq 2>&1 | sed 's/\x1b\[[0-9;]*m//g' | grep -E 'Starting RabbitMQ|Server startup complete|plugins started|ra
bbitmq_management|rabbitmq_tracing|Management plugin: HTTP|user .admin. authenticated' | head -n 20
2026-10-30 17:40:50.814225+00:00 [info] <0.209.0>  Starting RabbitMQ 4.4.0 on Erlang 29.1.2 [jit]
2026-10-30 17:40:51.790249+00:00 [info] <0.209.0> Running boot step rabbit_mgmt_reset_handler defined by app rabbitmq_managemen
t
2026-10-30 17:40:51.790282+00:00 [info] <0.209.0> Running boot step rabbit_mgmt_db_handler defined by app rabbitmq_management_a
gent
2026-10-30 17:40:51.989880+00:00 [info] <0.209.0> Running boot step rabbit_management_load_definitions defined by app rabbitmq_
management
2026-10-30 17:40:52.085299+00:00 [info] <0.1008.0> Management plugin: HTTP (non-TLS) listener started on port 15672
2026-10-30 17:40:52.213595+00:00 [info] <0.924.0> Server startup complete; 5 plugins started.
```

```
2026-10-30 17:40:52.213595+00:00 [info] <0.924.0>  * rabbitmq_tracing
2026-10-30 17:40:52.213595+00:00 [info] <0.924.0>  * rabbitmq_management
2026-10-30 17:40:52.213595+00:00 [info] <0.924.0>  * rabbitmq_management_agent
2026-10-30 17:40:53.653900+00:00 [info] <0.1072.0> connection 172.20.0.3:49068 -> 172.20.0.2:5672 - notification-worker@c5e9162
2592e: user 'admin' authenticated and granted access to vhost '/'
2026-10-30 17:40:53.718106+00:00 [info] <0.1089.0> connection 172.20.0.4:37510 -> 172.20.0.2:5672 - orders-api: user 'admin' au
thenticated and granted access to vhost '/'

# Logs de arranque de los servicios

$ docker compose logs --no-log-prefix orders-api notification-worker
2026-10-30T17:40:53.591Z [notification-worker@c5e91622592e] Iniciando worker
2026-10-30T17:40:53.671Z [notification-worker@c5e91622592e] Conectado a RabbitMQ. Escuchando la cola "pedidos" (prefetch=1, ack
manual, procesamiento=3000ms)
2026-10-30T17:40:53.656Z [orders-api] API REST escuchando en el puerto 3000 (POST /orders)
2026-10-30T17:40:53.724Z [orders-api] Conectado a RabbitMQ (amqp://***@rabbitmq:5672). Cola "pedidos" declarada (durable=true)

# Cola creada y visible en el broker
[cola pedidos] durable=true Ready=0 Unacked=0 Total=0 consumers=1
```



Consola de administración de RabbitMQ (http://localhost:15672), pestaña Overview (`rabbitmq/img/01-overview.png`)

3.2 Kafka

Comando: `docker compose up -d --build` en `kafka/`. Contenedores, puertos, arranque del broker (Kafka 3.7.1 en modo KRaft), creación del tópico con 3 particiones, arranque de Kafka UI (HTTP 200 en el puerto 8080) y de los servicios. Los errores `The group coordinator is not available` del arranque son transitorios: Kafka todavía estaba creando su tópico interno `__consumer_offsets`, kafkajs reintentó y los consumidores se unieron a sus grupos 2–3 s después.

Montaje del entorno Kafka — `evidencias/kafka/entorno.txt`

```
==================== Montaje del entorno Kafka ====================
# Ejecutado: 2026-10-30 12:46:39 (hora local) / 17:46:39 UTC

$ docker compose --profile late down -v --remove-orphans 2>&1 | tail -n 3

$ docker compose up -d --build 2>&1 | grep -E 'Built|Started|Healthy|Exited|Created'
 Image kafka-pedidos-billing-consumer Built
 Image kafka-pedidos-inventory-consumer Built
 Image kafka-pedidos-notification-consumer Built
 Image kafka-pedidos-orders-api Built
 Network kafka-pedidos_default Created
 Volume kafka-pedidos_kafka-data Created
 Container kafka Created
 Container kafka-init Created
 Container kafka-ui Created
 Container kafka-pedidos-billing-consumer-1 Created
 Container kafka-pedidos-notification-consumer-1 Created
 Container orders-api Created
 Container kafka-pedidos-inventory-consumer-1 Created
 Container kafka Started
 Container kafka Healthy
 Container kafka Healthy
 Container kafka-init Started
 Container kafka-ui Started
 Container kafka-init Exited
 Container kafka-init Exited
 Container kafka-init Exited
 Container kafka-init Exited
 Container kafka-pedidos-inventory-consumer-1 Started
 Container kafka-pedidos-billing-consumer-1 Started
 Container kafka-pedidos-notification-consumer-1 Started
 Container orders-api Started

$ docker compose ps -a --format 'table {{.Name}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}'
NAME                                  IMAGE                                 STATUS                     PORTS
kafka                                 bitnamilegacy/kafka:3.7               Up 16 seconds (healthy)    0.0.0.0:29092->29092/t
cp, [::]:29092->29092/tcp
kafka-init                            bitnamilegacy/kafka:3.7               Exited (0) 6 seconds ago
kafka-pedidos-billing-consumer-1      kafka-pedidos-billing-consumer        Up 6 seconds
kafka-pedidos-inventory-consumer-1    kafka-pedidos-inventory-consumer      Up 6 seconds
kafka-pedidos-notification-consumer-1 kafka-pedidos-notification-consumer   Up 6 seconds
kafka-ui                              provectuslabs/kafka-ui:v0.7.2         Up 9 seconds               0.0.0.0:8080->8080/tc
p, [::]:8080->8080/tcp
orders-api                            kafka-pedidos-orders-api              Up 6 seconds               0.0.0.0:3010->3000/tc
p, [::]:3010->3000/tcp

# Puertos publicados en el host

$ docker port kafka; docker port kafka-ui; docker port orders-api
29092/tcp -> 0.0.0.0:29092
29092/tcp -> [::]:29092
8080/tcp -> 0.0.0.0:8080
8080/tcp -> [::]:8080
3000/tcp -> 0.0.0.0:3010
3000/tcp -> [::]:3010
```

```
# Logs de arranque del broker Kafka (modo KRaft)

$ docker logs kafka 2>&1 | sed 's/\x1b\[[0-9;]*m//g' | grep -E 'Kafka version|Kafka Server started|Transition from STARTING to
STARTED|Awaiting socket connections on' | head -n 8
[2026-10-30 17:47:26,629] INFO Awaiting socket connections on 0.0.0.0:9093. (kafka.network.DataPlaneAcceptor)
[2026-10-30 17:47:27,119] INFO Awaiting socket connections on 0.0.0.0:9092. (kafka.network.DataPlaneAcceptor)
[2026-10-30 17:47:27,120] INFO Awaiting socket connections on 0.0.0.0:29092. (kafka.network.DataPlaneAcceptor)
[2026-10-30 17:47:27,123] INFO [BrokerServer id=1] Transition from STARTING to STARTED (kafka.server.BrokerServer)
[2026-10-30 17:47:27,124] INFO Kafka version: 3.7.1 (org.apache.kafka.common.utils.AppInfoParser)
[2026-10-30 17:47:27,125] INFO [KafkaRaftServer nodeId=1] Kafka Server started (kafka.server.KafkaRaftServer)

# Creación del tópico (contenedor kafka-init)

$ docker compose logs --no-log-prefix kafka-init
WARNING: Due to limitations in metric names, topics with a period ('.') or underscore ('_') could collide. To avoid issues it i
s best to use either, but not both.
Created topic orders.events.
Topic: orders.events    TopicId: Npzu0elsS_y_VE3viKXIUA PartitionCount: 3       ReplicationFactor: 1    Configs:
        Topic: orders.events    Partition: 0    Leader: 1       Replicas: 1     Isr: 1
        Topic: orders.events    Partition: 1    Leader: 1       Replicas: 1     Isr: 1
        Topic: orders.events    Partition: 2    Leader: 1       Replicas: 1     Isr: 1

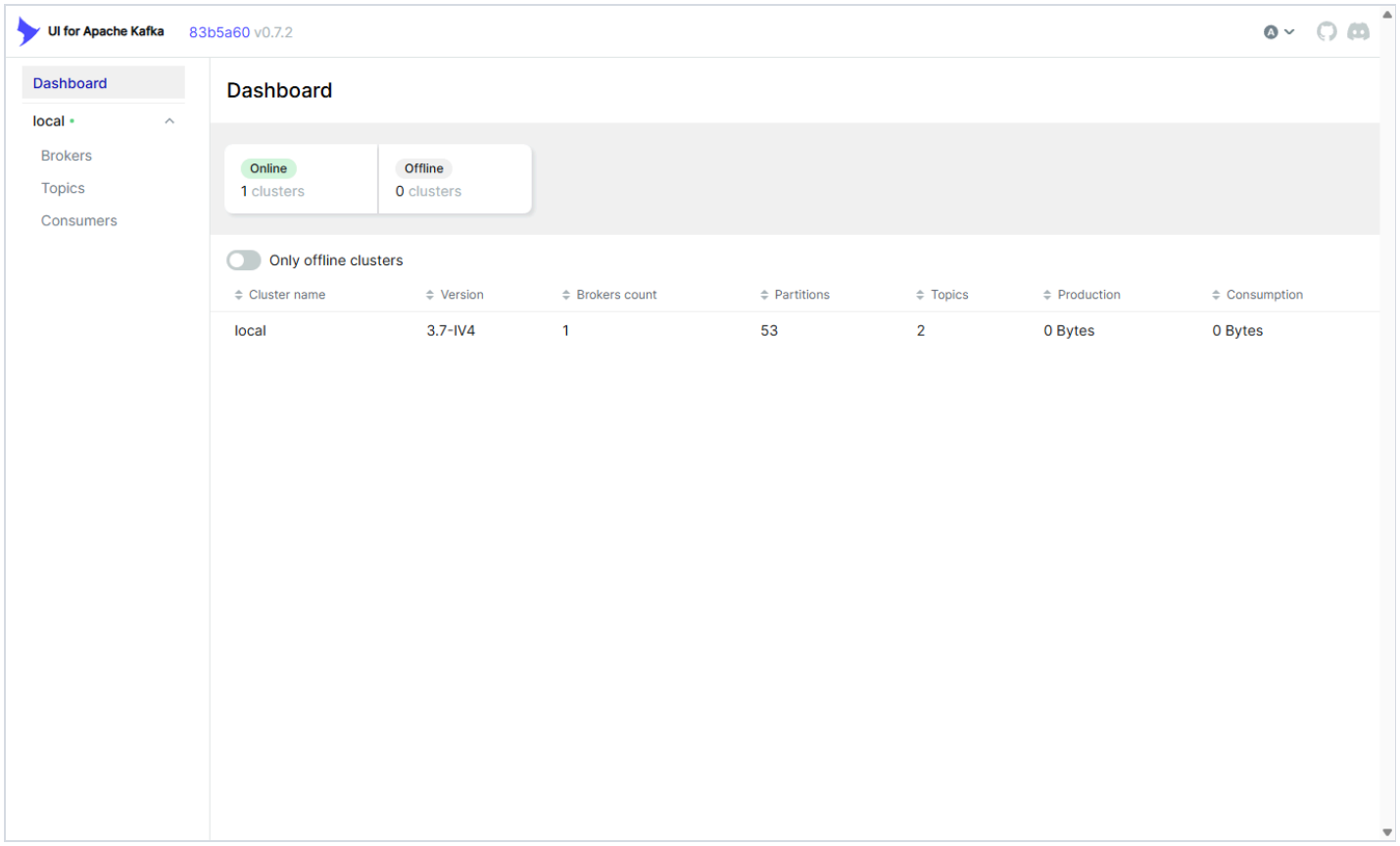
# Kafka UI

$ docker compose logs --no-log-prefix kafka-ui 2>&1 | grep -E 'Started KafkaUiApplication|Netty started' | head -n 3
2026-10-30 17:47:35,826 INFO  [main] o.s.b.w.e.n.NettyWebServer: Netty started on port 8080
2026-10-30 17:47:35,847 INFO  [main] c.p.k.u.KafkaUiApplication: Started KafkaUiApplication in 6.153 seconds (process running f
or 6.979)

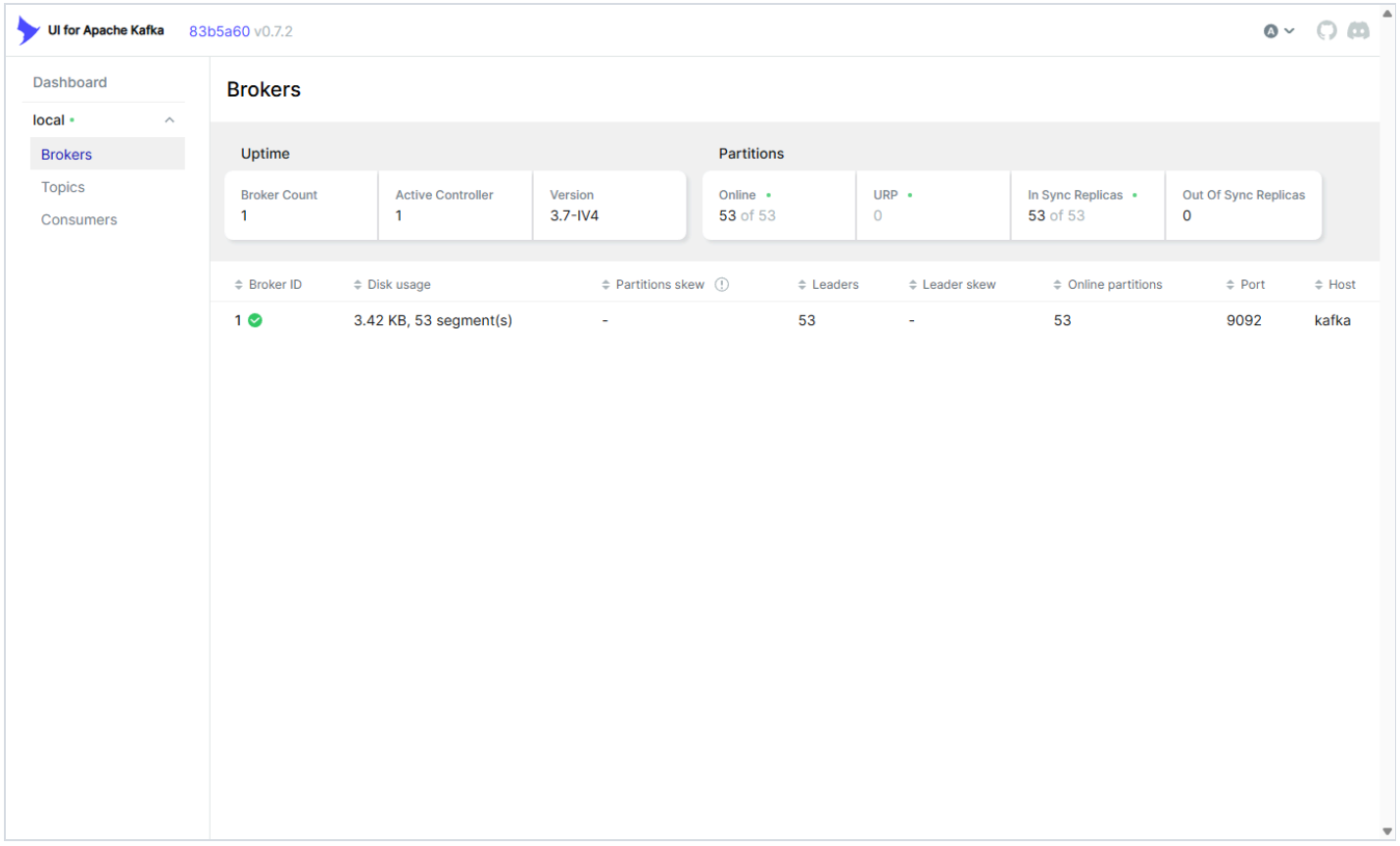
$ curl -s -o /dev/null -w 'GET http://localhost:8080 -> HTTP %{http_code}\n' http://localhost:8080
GET http://localhost:8080 -> HTTP 200

# Logs de arranque de los servicios

$ docker compose logs --no-log-prefix orders-api inventory-consumer billing-consumer notification-consumer
2026-10-30T17:47:32.652Z [billing-consumer@286de68024c8] Iniciando consumidor
2026-10-30T17:47:32.863Z [orders-api] API REST escuchando en el puerto 3000 (POST /orders)
2026-10-30T17:47:32.732Z [billing-consumer@286de68024c8] Suscrito a orders.events con groupId=billing-service-group
2026-10-30T17:47:32.612Z [inventory-consumer@d5d9dfbc2cc6] Iniciando consumidor
2026-10-30T17:47:32.900Z [orders-api] Productor conectado a Kafka (kafka:9092). Tópico de publicación: orders.events
2026-10-30T17:47:32.708Z [notification-consumer@38db5bcd9c8a] Iniciando consumidor
{"level":"ERROR","timestamp":"2026-10-30T17:47:32.748Z","logger":"kafkajs","message":"[Connection] Response GroupCoordinator(ke
y: 10, version: 2)","broker":"kafka:9092","clientId":"billing-consumer","error":"The group coordinator is not available","corre
lationId":2,"size":55}
2026-10-30T17:47:35.022Z [billing-consumer@286de68024c8] Unido al consumer group "billing-service-group". Particiones asignadas
de orders.events: [0,1,2]
2026-10-30T17:47:32.793Z [notification-consumer@38db5bcd9c8a] Suscrito a orders.events con groupId=notification-service-group
2026-10-30T17:47:34.775Z [notification-consumer@38db5bcd9c8a] Unido al consumer group "notification-service-group". Particiones
asignadas de orders.events: [0,1,2]
2026-10-30T17:47:32.684Z [inventory-consumer@d5d9dfbc2cc6] Suscrito a orders.events con groupId=inventory-service-group
{"level":"ERROR","timestamp":"2026-10-30T17:47:32.707Z","logger":"kafkajs","message":"[Connection] Response GroupCoordinator(ke
y: 10, version: 2)","broker":"kafka:9092","clientId":"inventory-consumer","error":"The group coordinator is not available","cor
relationId":2,"size":55}
2026-10-30T17:47:35.710Z [inventory-consumer@d5d9dfbc2cc6] Unido al consumer group "inventory-service-group". Particiones asign
adas de orders.events: [0,1,2]
```



Kafka UI (http://localhost:8080): clúster local en línea (`kafka/img/01-kafka-ui-dashboard.png`)



Kafka UI: broker activo (`kafka/img/02-broker-activo.png`)

4. Ejecución de pruebas: RabbitMQ

Evidencias requeridas por la guía: **cola creada** (sección 3.1 y CP-02), **mensaje enviado** (CP-01, CP-02), **mensaje visible en la cola** (CP-02, CP-07, CP-09), **consumidor procesando** (CP-03, CP-11). Pruebas obligatorias: **consumidor apagado** (CP-09, CP-11), **múltiples consumidores** (CP-10) e **independencia productor-consumidor** (CP-07, CP-08).

Los estados de la cola se consultaron con `rabbitmqctl list_queues`, que lee el estado real del broker en ese momento; la consola web puede mostrar los contadores con algunos segundos de retraso.

CP-01 · Creación de pedido vía API RabbitMQ

ID del caso	CP-01
Arquitectura	RabbitMQ
Objetivo	Validar que el endpoint `POST /orders` recibe el pedido, lo valida y responde correctamente.
Precondiciones	• Stack `rabbitmq/docker-compose.yml` en ejecución (rabbitmq, orders-api, notification-worker). • API conectada a RabbitMQ (log "Conectado a RabbitMQ").
Datos de entrada	`{"customerName":"Carlos","product":"Libro de arquitectura","quantity":1}`; casos negativos: `{"customerName":"","quantity":0}` y el texto `esto no es json`.
Pasos ejecutados	1. Enviar el pedido válido con curl a `http://localhost:3010/orders`. 2. Enviar dos pedidos inválidos. 3. Revisar el log de orders-api.
Resultado esperado	Pedido válido: HTTP 202 con `status: accepted`, mensaje y `orderId`. Pedidos inválidos: HTTP 400 con la lista de errores.
Resultado obtenido	HTTP 202 Accepted en 0,0105 s con `orderId=ORD-5575850C`. Los pedidos inválidos devolvieron HTTP 400 con los errores de validación. El log registra la publicación y el rechazo.
Estado	**Cumple**
Análisis técnico	Se usa 202 (Accepted) porque el pedido queda aceptado para procesamiento asíncrono, no procesado. La API valida la entrada antes de publicar: un pedido inválido nunca llega a la cola. Además de lo pedido, la respuesta incluye un `correlationId`.

Evidencia

Ejecución CP-01 — `evidencias/rabbitmq/CP-01.txt`

```
==================== CP-01 Creación de pedido vía API ====================
# Ejecutado: 2026-10-30 12:40:57 (hora local) / 17:40:57 UTC

# Pedido válido

$ curl -s -i -X POST http://localhost:3010/orders -H 'Content-Type: application/json' -d '{"customerName":"Carlos","product":"L
ibro de arquitectura","quantity":1}' -w '\n(tiempo total: %{time_total}s)\n'
HTTP/1.1 202 Accepted
Content-Type: application/json; charset=utf-8
Date: Fri, 30 Oct 2026 17:40:57 GMT
Connection: keep-alive
Keep-Alive: timeout=5
Transfer-Encoding: chunked

{"status":"accepted","message":"Pedido recibido y enviado a RabbitMQ","orderId":"ORD-5575850C","correlationId":"corr-cd6e8ab2-5
a3b-44c4-9b3a-b98f6b9654f4"}
(tiempo total: 0.010506s)

# Pedido inválido (validación de JSON básico)
```

```
$ curl -s -i -X POST http://localhost:3010/orders -H 'Content-Type: application/json' -d '{"customerName":"","quantity":0}'
HTTP/1.1 400 Bad Request
Content-Type: application/json; charset=utf-8
Date: Fri, 30 Oct 2026 17:40:58 GMT
Connection: keep-alive
Keep-Alive: timeout=5
Transfer-Encoding: chunked

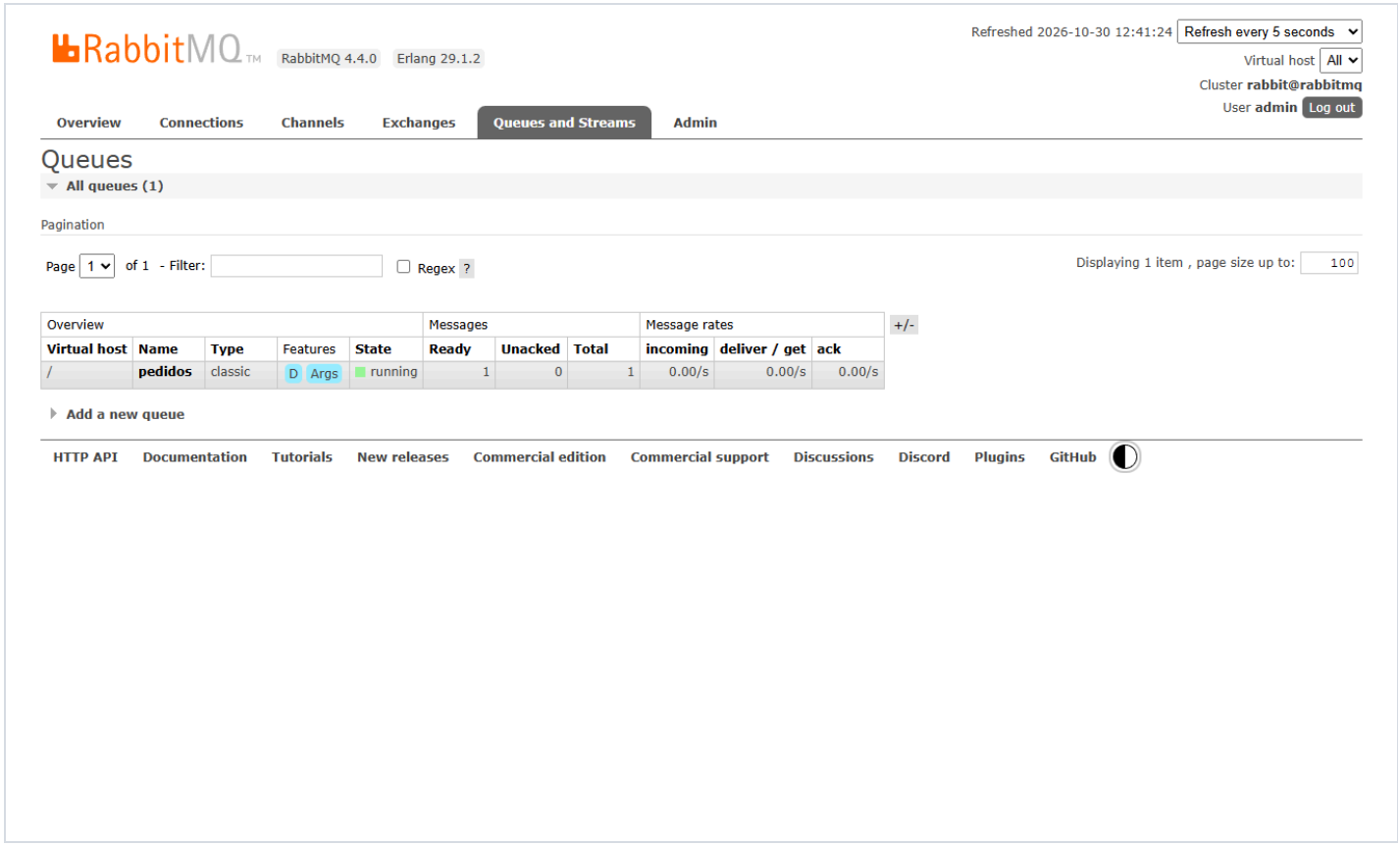
{"status":"rejected","errors":["customerName es obligatorio","product es obligatorio","quantity debe ser un entero positivo"]}
$ curl -s -i -X POST http://localhost:3010/orders -H 'Content-Type: application/json' -d 'esto no es json'
HTTP/1.1 400 Bad Request
Content-Type: application/json; charset=utf-8
Date: Fri, 30 Oct 2026 17:40:58 GMT
Connection: keep-alive
Keep-Alive: timeout=5
Transfer-Encoding: chunked

{"status":"rejected","errors":["JSON inválido"]}
$ docker compose logs --no-log-prefix orders-api | tail -n 4
2026-10-30T17:40:53.656Z [orders-api] API REST escuchando en el puerto 3000 (POST /orders)
2026-10-30T17:40:53.724Z [orders-api] Conectado a RabbitMQ (amqp://***@rabbitmq:5672). Cola "pedidos" declarada (durable=true)
2026-10-30T17:40:57.952Z [orders-api] Pedido recibido y publicado en cola "pedidos" orderId=ORD-5575850C correlationId=corr-cd6
e8ab2-5a3b-44c4-9b3a-b98f6b9654f4 customer="Carlos"
2026-10-30T17:40:58.017Z [orders-api] Pedido rechazado: customerName es obligatorio; product es obligatorio; quantity debe ser
un entero positivo
```

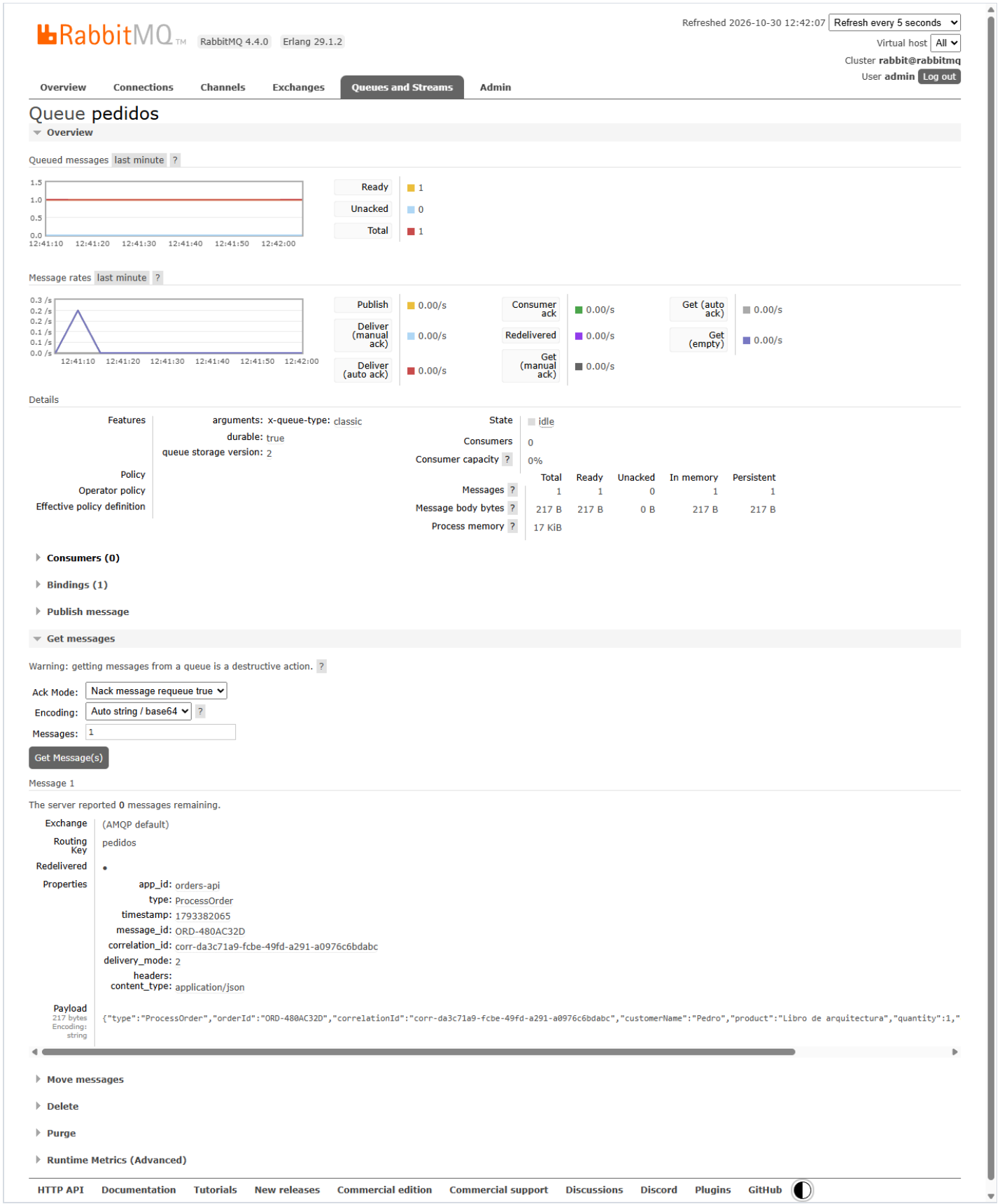
CP-02 · Envío de mensaje a RabbitMQ RabbitMQ

ID del caso	CP-02
Arquitectura	RabbitMQ
Objetivo	Verificar que cada pedido genera un mensaje persistente en la cola `pedidos`.
Precondiciones	• Stack `rabbitmq/docker-compose.yml` en ejecución (rabbitmq, orders-api, notification-worker). • Worker detenido para que el mensaje permanezca visible en la cola.
Datos de entrada	`{"customerName":"Pedro","product":"Libro de arquitectura","quantity":1}`
Pasos ejecutados	1. `docker compose stop notification-worker`. 2. Consultar la cola (Ready=0). 3. Enviar el pedido. 4. Consultar la cola de nuevo (rabbitmqctl) y leer el mensaje sin consumirlo (API de administración, `ack_requeue_true`). 5. Revisar la cola en la consola web y usar "Get messages".
Resultado esperado	Ready pasa de 0 a 1. El mensaje contiene `type=ProcessOrder`, el `orderId` y el `correlationId`, con `delivery_mode=2` (persistente).
Resultado obtenido	Ready 0 → 1 (Total=1, consumers=0). El mensaje leído tiene `message_id=ORD-480AC32D`, `correlation_id`, `app_id=orders-api`, `delivery_mode=2` y el payload JSON completo. La consola muestra la cola `pedidos` (durable, D) con Ready=1.
Estado	**Cumple**
Análisis técnico	La cola es durable y el mensaje persistente, de modo que sobrevive a reinicios del broker (ver CP-12). La API publica con *publisher confirms*: solo responde 202 cuando RabbitMQ confirmó que recibió el mensaje, lo que evita aceptar pedidos que no llegaron al broker.

Evidencia



Consola RabbitMQ: cola pedidos con Ready=1 y worker detenido `(rabbitmq/img/03-cp02-cola-pedidos-ready1.png)`



Consola RabbitMQ: detalle de la cola y contenido del mensaje (Get messages) (`rabbitmq/img/04-cp02-contenido-mensaje.png`)

Ejecución CP-02 evidencias/rabbitmq/CP-02.txt

```
==================== CP-02 Envío de mensaje a RabbitMQ ====================
# Ejecutado: 2026-10-30 12:41:02 (hora local) / 17:41:02 UTC

# Se detiene el worker para que el mensaje quede visible en la cola

$ docker compose stop notification-worker 2>&1
 Container rabbitmq-pedidos-notification-worker-1 Stopping
 Container rabbitmq-pedidos-notification-worker-1 Stopped

# Estado antes de enviar
[cola pedidos] durable=true Ready=0 Unacked=0 Total=0 consumers=0

# Envío del pedido
{"status":"accepted","message":"Pedido recibido y enviado a RabbitMQ","orderId":"ORD-480AC32D","correlationId":"corr-da3c71a9-f
cbe-49fd-a291-a0976c6bdabc"}  -> HTTP 202 en 0.005154s

# Estado después de enviar: Ready aumenta en 1
[cola pedidos] durable=true Ready=1 Unacked=0 Total=1 consumers=0

# Contenido del mensaje en la cola (lectura sin consumir: ackmode=ack_requeue_true)
{
  "routing_key": "pedidos",
  "redelivered": false,
  "properties": {
    "app_id": "orders-api",
    "type": "ProcessOrder",
    "timestamp": 1793382065,
    "message_id": "ORD-480AC32D",
    "correlation_id": "corr-da3c71a9-fcbe-49fd-a291-a0976c6bdabc",
    "delivery_mode": 2,
    "headers": {},
    "content_type": "application/json"
  },
  "payload": {
    "type": "ProcessOrder",
    "orderId": "ORD-480AC32D",
    "correlationId": "corr-da3c71a9-fcbe-49fd-a291-a0976c6bdabc",
    "customerName": "Pedro",
    "product": "Libro de arquitectura",
    "quantity": 1,
    "createdAt": "2026-10-30T17:41:05.709Z"
  }
}
```

CP-03 · Consumo de mensaje (RabbitMQ) RabbitMQ

ID del caso	CP-03
Arquitectura	RabbitMQ
Objetivo	Validar que el consumidor recibe, procesa y confirma (ACK manual) el mensaje.
Precondiciones	• Stack `rabbitmq/docker-compose.yml` en ejecución (rabbitmq, orders-api, notification-worker). • Mensaje de CP-02 pendiente en la cola (Ready=1).
Datos de entrada	Mensaje `ORD-480AC32D` publicado en CP-02.
Pasos ejecutados	1. `docker compose start notification-worker`. 2. Consultar la cola durante el procesamiento (3 s simulados). 3. Consultar la cola después del ACK. 4. Revisar el log del worker.
Resultado esperado	Durante el procesamiento el mensaje pasa a Unacked=1; tras el ACK la cola queda en 0. El log muestra recepción, contenido, simulación y confirmación.
Resultado obtenido	Durante el procesamiento: Ready=0, Unacked=1. Tras el ACK: Total=0. El worker registró "Mensaje recibido", el contenido JSON, "Simulando envío de notificación a Pedro..." y "ACK confirmado" 3 s después.
Estado	**Cumple**
Análisis técnico	El estado Unacked demuestra el ACK manual: RabbitMQ no borra el mensaje hasta que el worker lo confirma. El campo `redelivered=true` se explica porque la lectura de inspección de CP-02 (consola y API) devolvió el mensaje a la cola; RabbitMQ lo marca como reentregado aunque ningún consumidor lo haya procesado.

Evidencia

Ejecución CP-03 — `evidencias/rabbitmq/CP-03.txt`

```
==================== CP-03 Consumo de mensaje (RabbitMQ) ====================
# Ejecutado: 2026-10-30 12:42:19 (hora local) / 17:42:19 UTC

# Se inicia el worker con el mensaje de CP-02 esperando en la cola

$ docker compose start notification-worker 2>&1
 Container rabbitmq Waiting
 Container rabbitmq Healthy
 Container rabbitmq-pedidos-notification-worker-1 Starting
 Container rabbitmq-pedidos-notification-worker-1 Started

# Durante el procesamiento (2-5 s): el mensaje pasa a Unacked
[cola pedidos] durable=true Ready=0 Unacked=1 Total=1 consumers=1

# Después del ACK manual: la cola queda vacía
[cola pedidos] durable=true Ready=0 Unacked=0 Total=0 consumers=1

$ docker compose logs --no-log-prefix --since 2026-10-30T17:42:19Z notification-worker
2026-10-30T17:42:20.487Z [notification-worker@c5e91622592e] Iniciando worker
```

```
2026-10-30T17:42:20.559Z [notification-worker@c5e91622592e] Conectado a RabbitMQ. Escuchando la cola "pedidos" (prefetch=1, ack
manual, procesamiento=3000ms)
2026-10-30T17:42:20.562Z [notification-worker@c5e91622592e] Mensaje recibido orderId=ORD-480AC32D correlationId=corr-da3c71a9-f
cbe-49fd-a291-a0976c6bdabc redelivered=true deliveryTag=1
2026-10-30T17:42:20.562Z [notification-worker@c5e91622592e] Contenido: {"type":"ProcessOrder","orderId":"ORD-480AC32D","correla
tionId":"corr-da3c71a9-fcbe-49fd-a291-a0976c6bdabc","customerName":"Pedro","product":"Libro de arquitectura","quantity":1,"crea
tedAt":"2026-10-30T17:41:05.709Z"}
2026-10-30T17:42:20.562Z [notification-worker@c5e91622592e] Simulando envío de notificación a Pedro...
2026-10-30T17:42:23.566Z [notification-worker@c5e91622592e] Notificación enviada. ACK confirmado orderId=ORD-480AC32D correlati
onId=corr-da3c71a9-fcbe-49fd-a291-a0976c6bdabc
```

CP-06 · Trazabilidad del flujo RabbitMQ

ID del caso	CP-06
Arquitectura	RabbitMQ
Objetivo	Seguir un mismo pedido (orderId / correlationId) en los logs de la API, del broker y del consumidor.
Precondiciones	• Stack `rabbitmq/docker-compose.yml` en ejecución (rabbitmq, orders-api, notification-worker). • Plugin `rabbitmq_tracing` habilitado (archivo `enabled_plugins`).
Datos de entrada	Cabecera `X-Correlation-Id: corr-CP06-124228`; body `{"customerName":"Laura","product":"Curso de QA","quantity":2}`.
Pasos ejecutados	1. Crear una traza del broker con la API de administración (`PUT /api/traces/%2F/cp06`). 2. Enviar el pedido con el correlationId. 3. Buscar el correlationId en el log de orders-api, en la bitácora de tracing del contenedor rabbitmq y en el log del worker.
Resultado esperado	El mismo orderId y correlationId aparecen en los tres puntos del flujo, con marcas de tiempo coherentes.
Resultado obtenido	`ORD-E4B0124F` / `corr-CP06-124228`: API 17:42:29.500 "publicado en cola"; broker 17:42:29.499 "Message published" (conexión de orders-api, cola enrutada `pedidos`) y 17:42:29.500 "Message received" (conexión del worker); worker 17:42:29.500 "Mensaje recibido" y 17:42:32.505 "ACK confirmado".
Estado	**Cumple**
Análisis técnico	La bitácora del broker es evidencia independiente del código propio: muestra quién publicó (IP 172.20.0.4 = orders-api) y a quién se entregó (172.20.0.3 = worker), con las propiedades AMQP `message_id` y `correlation_id`. El correlationId puede venir del cliente (cabecera) o lo genera la API.

Evidencia

Ejecución CP-06 — `evidencias/rabbitmq/CP-06.txt`

```
==================== CP-06 Trazabilidad del flujo (correlationId) ====================
# Ejecutado: 2026-10-30 12:42:28 (hora local) / 17:42:28 UTC

# Se activa la traza del broker (plugin rabbitmq_tracing) para registrar publicaciones y entregas
PUT /api/traces/%2F/cp06 -> HTTP 201

# Pedido con cabecera X-Correlation-Id: corr-CP06-124228
{"status":"accepted","message":"Pedido recibido y enviado a RabbitMQ","orderId":"ORD-E4B0124F","correlationId":"corr-CP06-12422
8"}  -> HTTP 202 en 0.006590s

# 1) Log de la API (productor)

$ docker compose logs --no-log-prefix orders-api | grep corr-CP06-124228
2026-10-30T17:42:29.500Z [orders-api] Pedido recibido y publicado en cola "pedidos" orderId=ORD-E4B0124F correlationId=corr-CP0
6-124228 customer="Laura"
```

```
# 2) Log del broker (bitácora de tracing dentro del contenedor rabbitmq)

$ docker exec rabbitmq cat /var/tmp/rabbitmq-tracing/cp06.log   (entradas que contienen corr-CP06-124228)
2026-10-30T17:42:29.499+00:00: Message published

Node:         rabbit@rabbitmq
Connection:   172.20.0.4:37510 -> 172.20.0.2:5672
Virtual host: /
User:         admin
Channel:      1
Exchange:
Routing keys: [<<"pedidos">>]
Routed queues: [<<"pedidos">>]
Properties:   [{<<"app_id">>,longstr,<<"orders-api">>},
               {<<"type">>,longstr,<<"ProcessOrder">>},
               {<<"timestamp">>,signedint,1793382149},
               {<<"message_id">>,longstr,<<"ORD-E4B0124F">>},
               {<<"correlation_id">>,longstr,<<"corr-CP06-124228">>},
               {<<"delivery_mode">>,signedint,2},
               {<<"headers">>,table,[]},
               {<<"content_type">>,longstr,<<"application/json">>}]
Payload:
{"type":"ProcessOrder","orderId":"ORD-E4B0124F","correlationId":"corr-CP06-124228","customerName":"Laura","product":"Curso de Q
A","quantity":2,"createdAt":"2026-10-30T17:42:29.498Z"}
---------------------------------------
2026-10-30T17:42:29.500+00:00: Message received

Node:         rabbit@rabbitmq
Connection:   172.20.0.3:40144 -> 172.20.0.2:5672
Virtual host: /
User:         admin
Channel:      1
Exchange:
Routing keys: [<<"pedidos">>]
Queue:        pedidos
Properties:   [{<<"app_id">>,longstr,<<"orders-api">>},
               {<<"type">>,longstr,<<"ProcessOrder">>},
               {<<"timestamp">>,signedint,1793382149},
               {<<"message_id">>,longstr,<<"ORD-E4B0124F">>},
               {<<"correlation_id">>,longstr,<<"corr-CP06-124228">>},
               {<<"delivery_mode">>,signedint,2},
               {<<"headers">>,table,[]},
               {<<"content_type">>,longstr,<<"application/json">>}]
Payload:
{"type":"ProcessOrder","orderId":"ORD-E4B0124F","correlationId":"corr-CP06-124228","customerName":"Laura","product":"Curso de Q
A","quantity":2,"createdAt":"2026-10-30T17:42:29.498Z"}

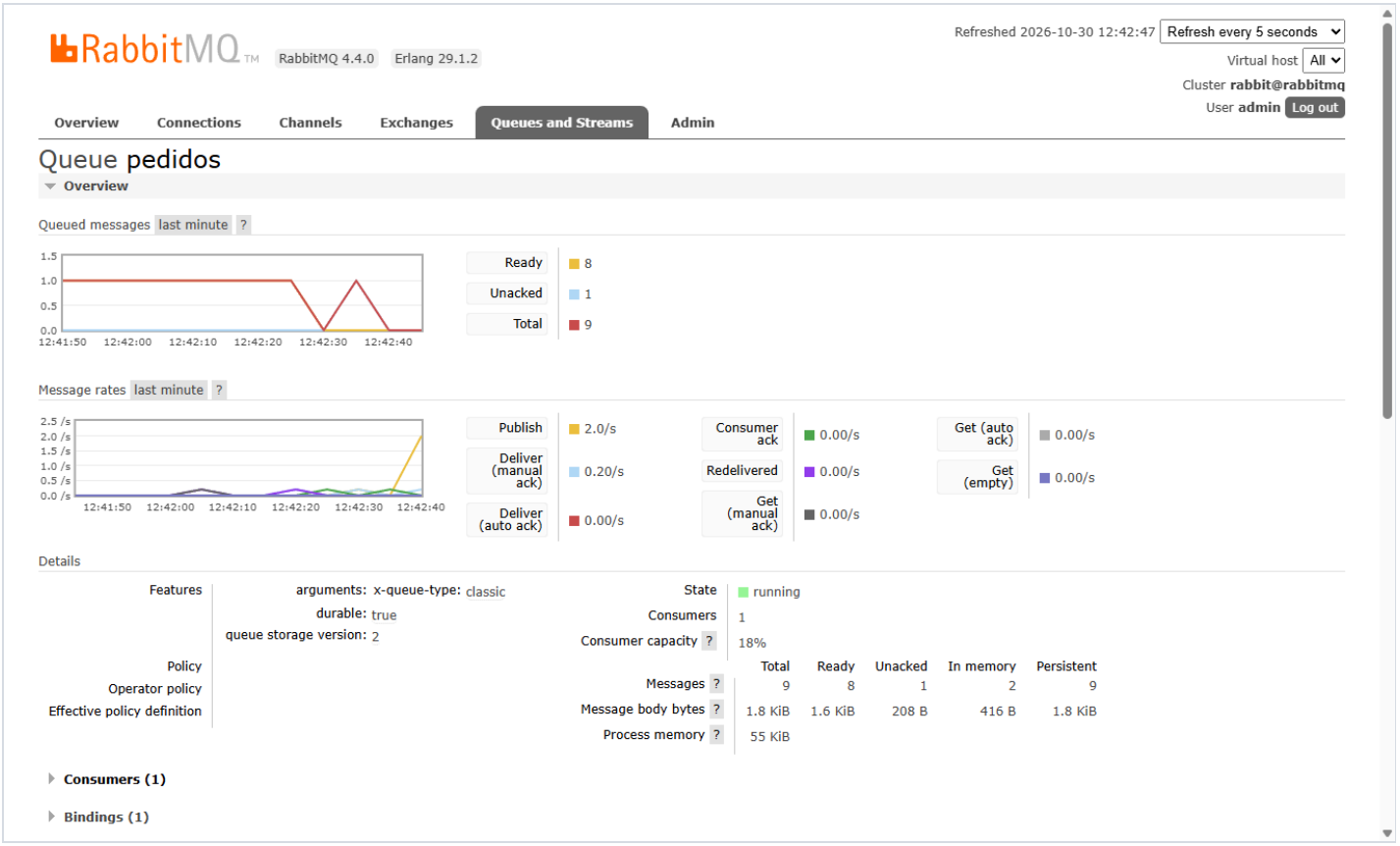
# 3) Log del consumidor

$ docker compose logs --no-log-prefix notification-worker | grep corr-CP06-124228
2026-10-30T17:42:29.500Z [notification-worker@c5e91622592e] Mensaje recibido orderId=ORD-E4B0124F correlationId=corr-CP06-12422
8 redelivered=false deliveryTag=2
2026-10-30T17:42:29.500Z [notification-worker@c5e91622592e] Contenido: {"type":"ProcessOrder","orderId":"ORD-E4B0124F","correla
tionId":"corr-CP06-124228","customerName":"Laura","product":"Curso de QA","quantity":2,"createdAt":"2026-10-30T17:42:29.498Z"}
2026-10-30T17:42:32.505Z [notification-worker@c5e91622592e] Notificación enviada. ACK confirmado orderId=ORD-E4B0124F correlati
onId=corr-CP06-124228
```

CP-07 · Responsive: la API responde sin esperar al consumidor RabbitMQ

ID del caso	CP-07
Arquitectura	RabbitMQ
Objetivo	Demostrar que el tiempo de respuesta HTTP no depende del tiempo de procesamiento del consumidor.
Precondiciones	• Stack `rabbitmq/docker-compose.yml` en ejecución (rabbitmq, orders-api, notification-worker). • Worker con `PROCESSING_MS=3000` (3 s por mensaje).
Datos de entrada	10 pedidos (Cliente1…Cliente10) enviados seguidos.
Pasos ejecutados	1. Enviar 10 pedidos y medir cada respuesta (`%{time_total}` de curl). 2. Consultar la cola inmediatamente. 3. Esperar a que la cola se vacíe y medir el tiempo.
Resultado esperado	Cada respuesta en milisegundos; la cola acumula mensajes que el worker procesa después (~30 s).
Resultado obtenido	Las 10 respuestas HTTP 202 tardaron entre 4 y 6 ms (428 ms en total, incluido el arranque de curl). Justo después la cola tenía Ready=9 y Unacked=1. El worker necesitó 30 s más para vaciarla.
Estado	**Cumple**
Análisis técnico	Si la API esperara al consumidor, cada respuesta tardaría al menos 3 s (30 s en total). La diferencia de tres órdenes de magnitud prueba que la respuesta es independiente del procesamiento. La captura de la consola muestra la cola acumulando (Ready 8, Unacked 1) mientras la API ya había respondido.

Evidencia



Consola RabbitMQ durante CP-07: Ready=8, Unacked=1, Total=9 (`rabbitmq/img/05-cp07-cola-acumulando.png`)

Ejecución CP-07

`evidencias/rabbitmq/CP-07.txt`

```
==================== CP-07 Responsive: la API responde sin esperar al consumidor ====================
# Ejecutado: 2026-10-30 12:42:41 (hora local) / 17:42:41 UTC

# El worker tarda 3 s por mensaje (PROCESSING_MS=3000). Se envían 10 pedidos seguidos.
Pedido  1:{"status":"accepted","message":"Pedido recibido y enviado a RabbitMQ","orderId":"ORD-9A4F8C35","correlationId":"corr-
0245ac08-d9cc-4efb-86d1-564b9a5817e5"}  -> HTTP 202 en 0.005559s
Pedido  2:{"status":"accepted","message":"Pedido recibido y enviado a RabbitMQ","orderId":"ORD-88E1413B","correlationId":"corr-
1f7371bc-4d13-4f44-8828-dbf0d0a5e526"}  -> HTTP 202 en 0.006073s
Pedido  3:{"status":"accepted","message":"Pedido recibido y enviado a RabbitMQ","orderId":"ORD-0CEB6E04","correlationId":"corr-
770bf818-c459-4fac-b288-fb73c0e72169"}  -> HTTP 202 en 0.005848s
Pedido  4:{"status":"accepted","message":"Pedido recibido y enviado a RabbitMQ","orderId":"ORD-2ACAE30F","correlationId":"corr-
0ba3c71f-afbf-464a-8f69-411dfd913846"}  -> HTTP 202 en 0.004742s
Pedido  5:{"status":"accepted","message":"Pedido recibido y enviado a RabbitMQ","orderId":"ORD-F9FA9D2A","correlationId":"corr-
cda65648-0057-4eff-b1a6-8d4f51fe269d"}  -> HTTP 202 en 0.004126s
Pedido  6:{"status":"accepted","message":"Pedido recibido y enviado a RabbitMQ","orderId":"ORD-B67E4B52","correlationId":"corr-
8d88435f-d1b6-435c-81fc-619001a477ae"}  -> HTTP 202 en 0.004604s
Pedido  7:{"status":"accepted","message":"Pedido recibido y enviado a RabbitMQ","orderId":"ORD-22885040","correlationId":"corr-
a33f5a18-fd2f-4ed2-bd08-8e863c3910df"}  -> HTTP 202 en 0.004309s
Pedido  8:{"status":"accepted","message":"Pedido recibido y enviado a RabbitMQ","orderId":"ORD-56D9C350","correlationId":"corr-
4a13f10f-6855-455c-927b-9f8aa6ddf9d6"}  -> HTTP 202 en 0.004554s
Pedido  9:{"status":"accepted","message":"Pedido recibido y enviado a RabbitMQ","orderId":"ORD-EF26C73D","correlationId":"corr-
74ac0d19-2c4a-4cdf-a7a6-676e4ef4a3d0"}  -> HTTP 202 en 0.005473s
Pedido 10:{"status":"accepted","message":"Pedido recibido y enviado a RabbitMQ","orderId":"ORD-6DE252F4","correlationId":"corr-
ef05cfd2-74c2-4219-b437-340798c73e97"}  -> HTTP 202 en 0.004591s
Tiempo total de las 10 respuestas HTTP: 428 ms

# Inmediatamente después: la mayoría de los mensajes siguen pendientes en la cola
[cola pedidos] durable=true Ready=9 Unacked=1 Total=10 consumers=1

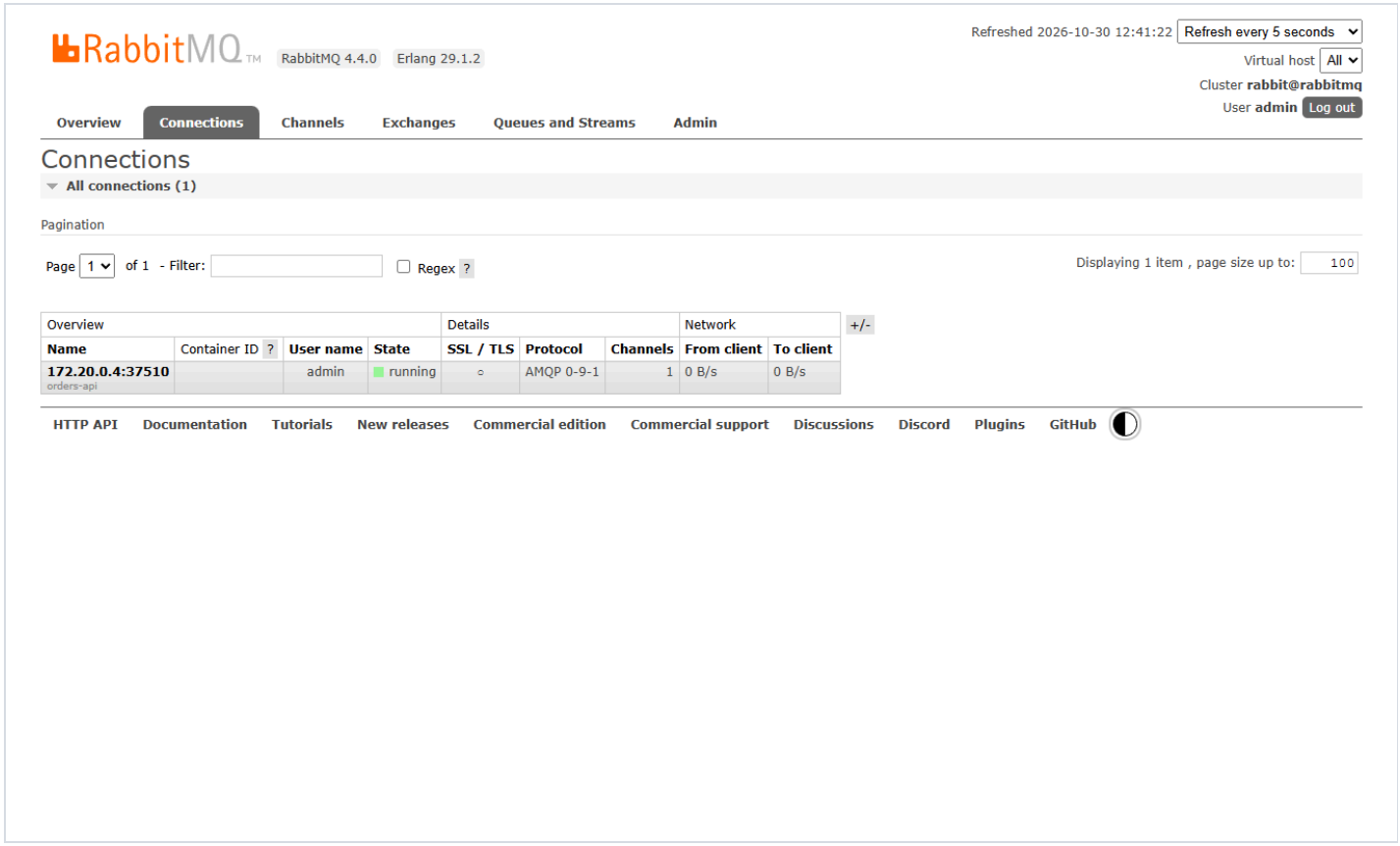
# El worker necesita ~30 s para terminarlos
```

```
Cola vacía después de 30 s adicionales
[cola pedidos] durable=true Ready=0 Unacked=0 Total=0 consumers=1
```

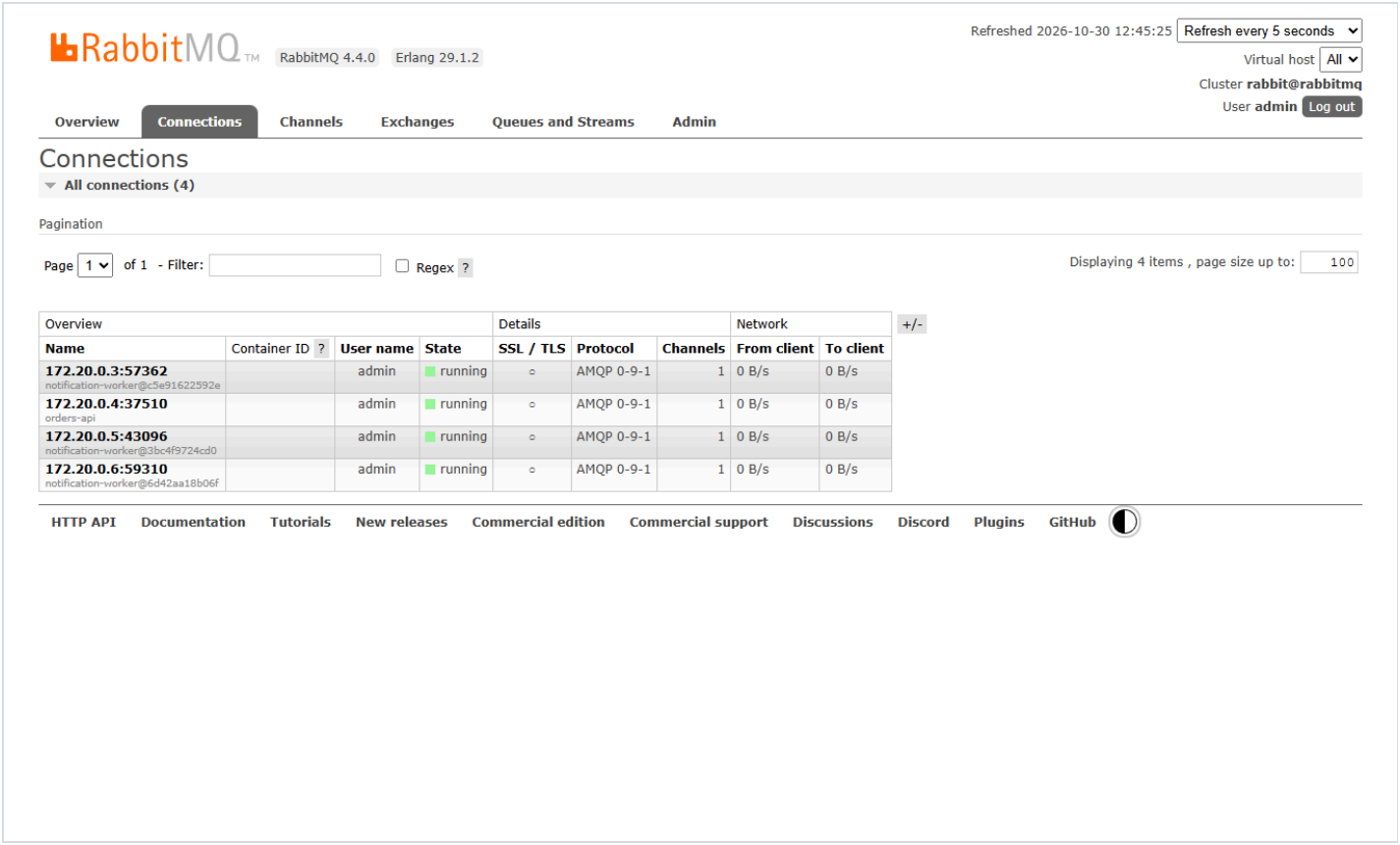
CP-08 · Message-driven: no existe comunicación directa RabbitMQ

ID del caso	CP-08
Arquitectura	RabbitMQ
Objetivo	Comprobar que la API y el worker solo se comunican a través de RabbitMQ.
Precondiciones	• Stack `rabbitmq/docker-compose.yml` en ejecución (rabbitmq, orders-api, notification-worker).
Datos de entrada	No aplica (inspección de red, puertos y código).
Pasos ejecutados	1. Listar los puertos publicados (`docker compose ps`). 2. Listar los sockets en escucha dentro del worker (`netstat -tln`). 3. Listar las conexiones TCP establecidas de la API y del worker. 4. Intentar una llamada HTTP de la API al worker. 5. Listar las conexiones registradas en el broker. 6. Buscar en el código llamadas HTTP salientes o servidores HTTP.
Resultado esperado	El worker no publica ni escucha puertos; ambos servicios solo tienen conexiones hacia rabbitmq:5672; la llamada HTTP directa falla.
Resultado obtenido	El worker no publica puertos y solo escucha en 127.0.0.11 (resolvedor DNS interno de Docker). La API (172.20.0.4) y el worker (172.20.0.3) solo tienen una conexión ESTABLISHED cada uno, hacia 172.20.0.2:5672 (rabbitmq). `wget http://notification-worker:3000` desde la API: "Connection refused". El broker lista las conexiones AMQP `orders-api` y `notification-worker@…`. En el código, el único servidor HTTP es el de orders-api.
Estado	**Cumple**
Análisis técnico	La comunicación es exclusivamente AMQP a través del broker. El worker ni siquiera tiene un servidor HTTP que la API pudiera llamar. Los nombres de conexión (`connection_name`) hacen visible en la consola qué servicio abrió cada conexión.

Evidencia



Consola RabbitMQ: conexión de orders-api (worker detenido en ese momento) `(rabbitmq/img/02-conexiones.png)`



Consola RabbitMQ: solo existen conexiones AMQP de orders-api y de los workers hacia el broker (`rabbitmq/img/08-cp10-conexiones-3-workers.png`)

Ejecución CP-08

`evidencias/rabbitmq/CP-08.txt`

```
==================== CP-08 Message-driven: no existe comunicación directa entre servicios ====================
# Ejecutado: 2026-10-30 12:44:24 (hora local) / 17:44:24 UTC

# El worker no publica puertos; solo orders-api (3000) y rabbitmq (5672, 15672)

$ docker compose ps --format 'table {{.Name}}\t{{.Ports}}'
NAME                                    PORTS
orders-api                              0.0.0.0:3010->3000/tcp, [::]:3010->3000/tcp
rabbitmq                                0.0.0.0:5672->5672/tcp, [::]:5672->5672/tcp, 0.0.0.0:15672->15672/tcp, [::]:15672->156
72/tcp
rabbitmq-pedidos-notification-worker-1

# Sockets en escucha dentro del worker: solo 127.0.0.11 (DNS interno de Docker), no hay servidor HTTP

$ docker compose exec notification-worker netstat -tln; echo '(fin de la lista)'
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 127.0.0.11:42933        0.0.0.0:*               LISTEN
(fin de la lista)

# Conexiones TCP establecidas: API y worker solo hablan con rabbitmq:5672

$ docker inspect -f '{{.Name}} {{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' rabbitmq orders-api $(docker compose p
s -q notification-worker)
/rabbitmq 172.20.0.2
/orders-api 172.20.0.4
/rabbitmq-pedidos-notification-worker-1 172.20.0.3

$ docker compose exec orders-api netstat -tn | grep ESTABLISHED
tcp        0      0 172.20.0.4:37510        172.20.0.2:5672         ESTABLISHED
```

```
$ docker compose exec notification-worker netstat -tn | grep ESTABLISHED
tcp        0      0 172.20.0.3:40144        172.20.0.2:5672         ESTABLISHED

# Intento de llamada HTTP directa de la API al worker: debe fallar

$ docker compose exec orders-api wget -q -T 3 -O- http://notification-worker:3000/ 2>&1 || echo 'Conexión rechazada: el worker
no expone HTTP'
wget: can't connect to remote host (172.20.0.3): Connection refused
Conexión rechazada: el worker no expone HTTP

# Conexiones registradas en el broker
notification-worker@c5e91622592e  172.20.0.3:40144 -> 172.20.0.2:5672  protocolo=AMQP 0-9-1
orders-api  172.20.0.4:37510 -> 172.20.0.2:5672  protocolo=AMQP 0-9-1

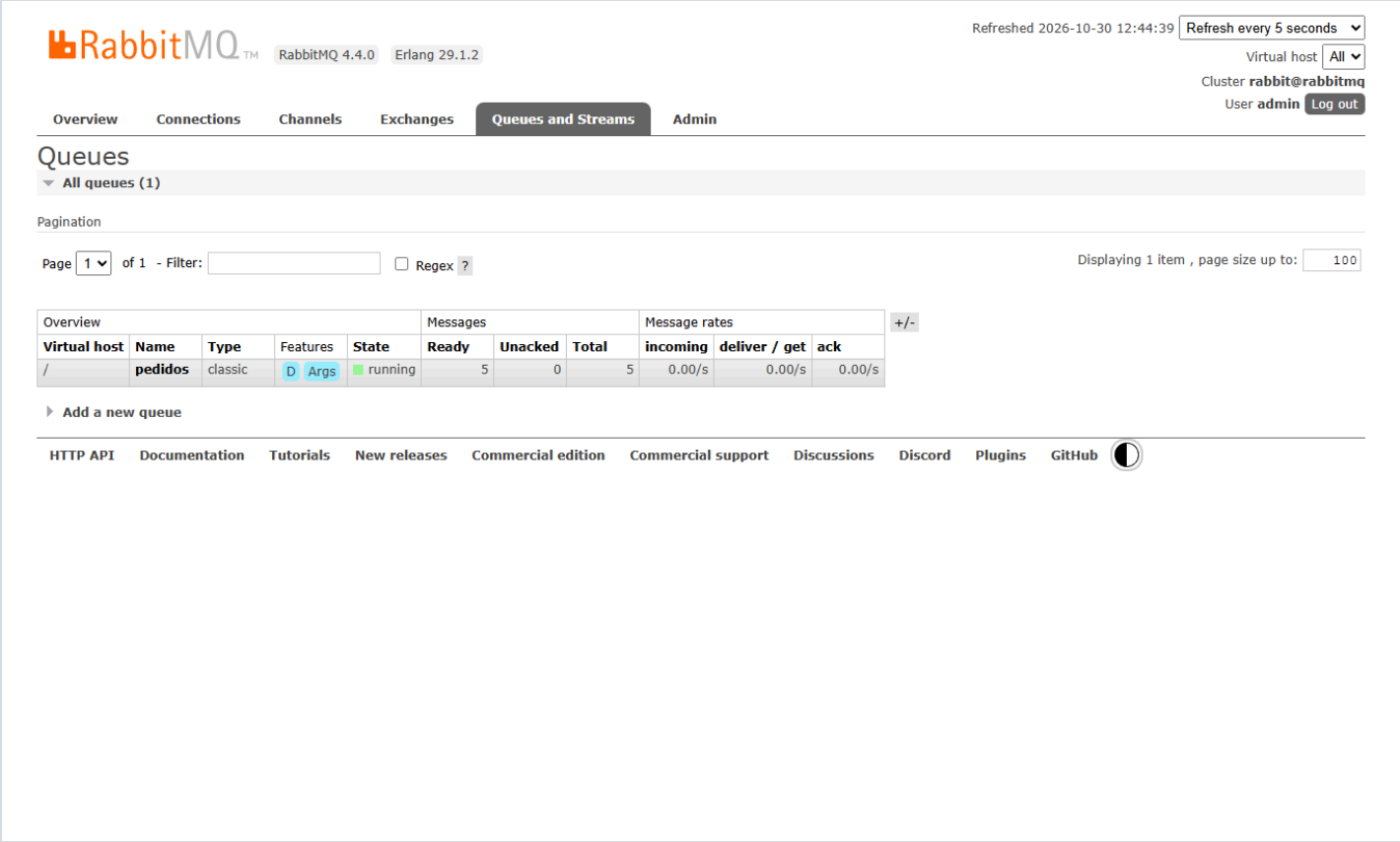
# Revisión del código: ninguna llamada HTTP saliente en la API ni servidor HTTP en el worker

$ grep -nE 'http\.request|[^e]fetch\(|axios|createServer|listen\(' orders-api/index.js notification-worker/index.js
orders-api/index.js:120:const server = http.createServer((req, res) => {
orders-api/index.js:133:server.listen(PORT, () => {
```

CP-09 · Resilient: el sistema soporta la caída del consumidor RabbitMQ

ID del caso	CP-09
Arquitectura	RabbitMQ
Objetivo	Verificar que con el consumidor caído la API sigue aceptando pedidos y los mensajes se conservan.
Precondiciones	• Stack `rabbitmq/docker-compose.yml` en ejecución (rabbitmq, orders-api, notification-worker).
Datos de entrada	5 pedidos (SinWorker1…SinWorker5).
Pasos ejecutados	1. `docker compose stop notification-worker`. 2. Enviar 5 pedidos. 3. Consultar `/health` de la API. 4. Consultar la cola.
Resultado esperado	Las 5 respuestas son HTTP 202; la API sigue UP; la cola acumula Ready=5 con consumers=0.
Resultado obtenido	5 × HTTP 202 (4–6 ms). `/health` = `{"status":"UP","broker":"connected"}`. Cola: Ready=5, Unacked=0, consumers=0. La consola muestra lo mismo.
Estado	**Cumple**
Análisis técnico	La caída del consumidor no se propaga al productor: el broker absorbe la carga en la cola. El fallo queda contenido en un solo componente.

Evidencia



Consola RabbitMQ: 5 mensajes en Ready con el worker apagado (`rabbitmq/img/06-cp09-worker-apagado-ready5.png`)

Ejecución CP-09

`evidencias/rabbitmq/CP-09.txt`

```
==================== CP-09 Resilient: el sistema soporta la caída del consumidor ====================
# Ejecutado: 2026-10-30 12:44:27 (hora local) / 17:44:27 UTC

$ docker compose stop notification-worker 2>&1
 Container rabbitmq-pedidos-notification-worker-1 Stopping
 Container rabbitmq-pedidos-notification-worker-1 Stopped

$ docker compose ps --format 'table {{.Name}}\t{{.Status}}'
NAME          STATUS
orders-api    Up 3 minutes
rabbitmq      Up 3 minutes (healthy)

# Con el worker caído la API sigue aceptando pedidos
Pedido 1:{"status":"accepted","message":"Pedido recibido y enviado a RabbitMQ","orderId":"ORD-64D9F560","correlationId":"corr-4
d480808-1ff8-4860-84af-eb4e10f21a42"}  -> HTTP 202 en 0.005917s
Pedido 2:{"status":"accepted","message":"Pedido recibido y enviado a RabbitMQ","orderId":"ORD-EA6E4A51","correlationId":"corr-3
8c37954-8b19-48a8-a65e-4680925ae469"}  -> HTTP 202 en 0.004328s
Pedido 3:{"status":"accepted","message":"Pedido recibido y enviado a RabbitMQ","orderId":"ORD-AD987C25","correlationId":"corr-4
1e77517-13a4-4871-8d29-569d61de30ec"}  -> HTTP 202 en 0.004483s
Pedido 4:{"status":"accepted","message":"Pedido recibido y enviado a RabbitMQ","orderId":"ORD-2CF8BF57","correlationId":"corr-2
21ceb5c-275b-4006-90e4-ebce374542ca"}  -> HTTP 202 en 0.004035s
Pedido 5:{"status":"accepted","message":"Pedido recibido y enviado a RabbitMQ","orderId":"ORD-C208AFA0","correlationId":"corr-f
cec9a3b-6c5d-4772-9dd8-5032f6f72963"}  -> HTTP 202 en 0.004695s

$ curl -s http://localhost:3010/health; echo
{"status":"UP","broker":"connected"}

# Los mensajes se acumulan en la cola (Ready sube, consumers=0)
[cola pedidos] durable=true Ready=5 Unacked=0 Total=5 consumers=0
```

CP-11 · Desacoplamiento temporal RabbitMQ

ID del caso	CP-11
Arquitectura	RabbitMQ
Objetivo	Verificar que los mensajes publicados con el consumidor apagado se procesan cuando este se inicia.
Precondiciones	• Stack `rabbitmq/docker-compose.yml` en ejecución (rabbitmq, orders-api, notification-worker). • 5 mensajes pendientes de CP-09 y worker detenido.
Datos de entrada	Los 5 mensajes de CP-09.
Pasos ejecutados	1. Consultar la cola (Ready=5). 2. `docker compose start notification-worker`. 3. Esperar a que la cola se vacíe y revisar el log del worker.
Resultado esperado	El worker procesa los 5 mensajes en orden FIFO y la cola queda en 0.
Resultado obtenido	El worker se conectó a las 17:44:49.897 y procesó los 5 pedidos (ORD-64D9F560 … ORD-C208AFA0), uno cada 3 s, en el mismo orden de publicación. Cola final: Total=0.
Estado	**Cumple**
Análisis técnico	Productor y consumidor no necesitan estar activos al mismo tiempo: la cola desacopla en el tiempo. Con `prefetch=1` los mensajes se procesan uno a uno.

Evidencia

Ejecución CP-11 — `evidencias/rabbitmq/CP-11.txt`

```
==================== CP-11 Desacoplamiento temporal: el worker procesa al iniciarse ====================
# Ejecutado: 2026-10-30 12:44:47 (hora local) / 17:44:47 UTC

# Estado antes de iniciar el worker
[cola pedidos] durable=true Ready=5 Unacked=0 Total=5 consumers=0

$ docker compose start notification-worker 2>&1
 Container rabbitmq Waiting
 Container rabbitmq Healthy
 Container rabbitmq-pedidos-notification-worker-1 Starting
 Container rabbitmq-pedidos-notification-worker-1 Started

# Estado después
[cola pedidos] durable=true Ready=0 Unacked=0 Total=0 consumers=1

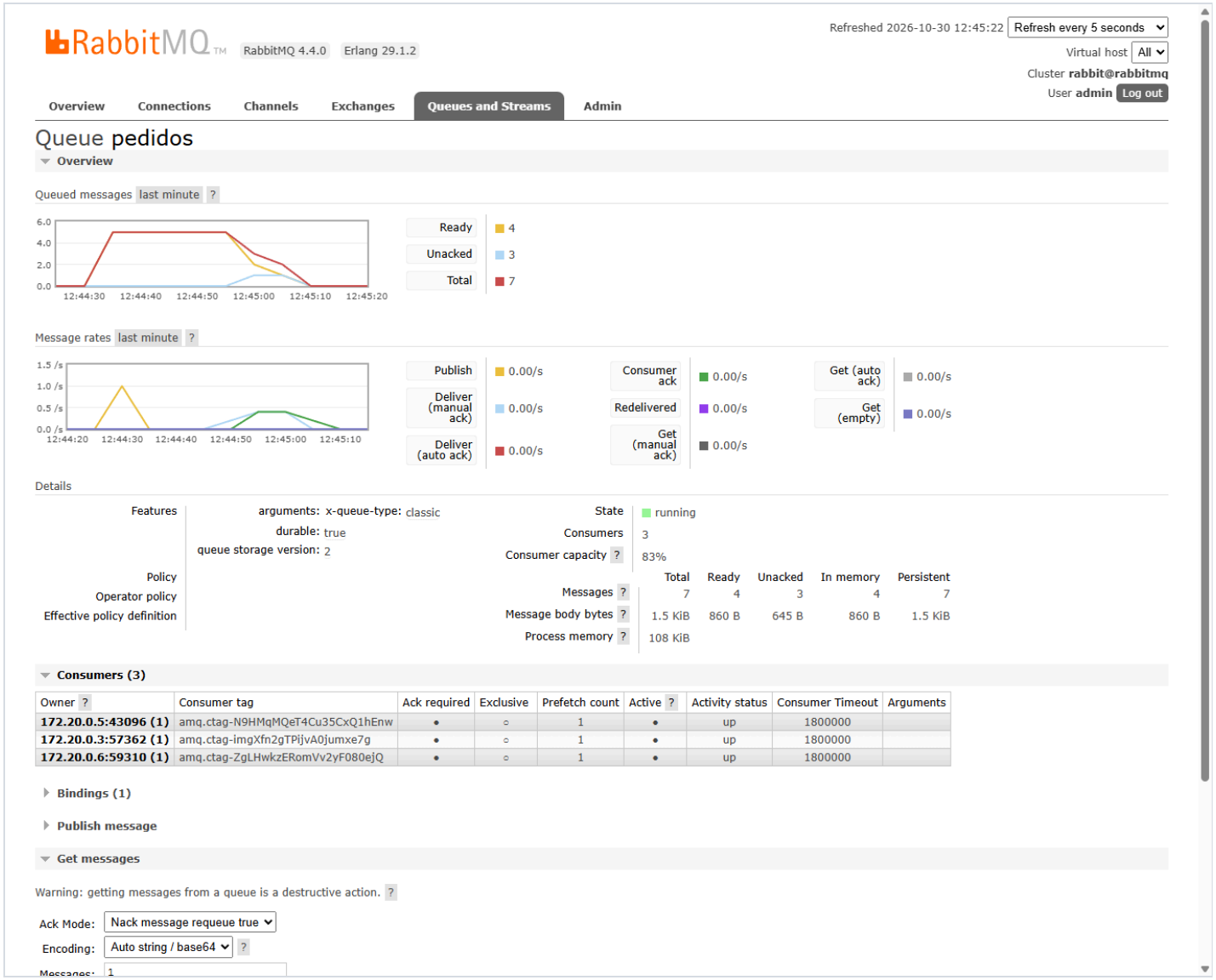
$ docker compose logs --no-log-prefix --since 2026-10-30T17:44:47Z notification-worker | grep -E 'Conectado|Mensaje recibido|AC
K'
2026-10-30T17:44:49.897Z [notification-worker@c5e91622592e] Conectado a RabbitMQ. Escuchando la cola "pedidos" (prefetch=1, ack
manual, procesamiento=3000ms)
2026-10-30T17:44:49.899Z [notification-worker@c5e91622592e] Mensaje recibido orderId=ORD-64D9F560 correlationId=corr-4d480808-1
ff8-4860-84af-eb4e10f21a42 redelivered=false deliveryTag=1
2026-10-30T17:44:52.903Z [notification-worker@c5e91622592e] Notificación enviada. ACK confirmado orderId=ORD-64D9F560 correlati
onId=corr-4d480808-1ff8-4860-84af-eb4e10f21a42
2026-10-30T17:44:52.904Z [notification-worker@c5e91622592e] Mensaje recibido orderId=ORD-EA6E4A51 correlationId=corr-38c37954-8
b19-48a8-a65e-4680925ae469 redelivered=false deliveryTag=2
```

```
2026-10-30T17:44:55.908Z [notification-worker@c5e91622592e] Notificación enviada. ACK confirmado orderId=ORD-EA6E4A51 correlati
onId=corr-38c37954-8b19-48a8-a65e-4680925ae469
2026-10-30T17:44:55.909Z [notification-worker@c5e91622592e] Mensaje recibido orderId=ORD-AD987C25 correlationId=corr-41e77517-1
3a4-4871-8d29-569d61de30ec redelivered=false deliveryTag=3
2026-10-30T17:44:58.910Z [notification-worker@c5e91622592e] Notificación enviada. ACK confirmado orderId=ORD-AD987C25 correlati
onId=corr-41e77517-13a4-4871-8d29-569d61de30ec
2026-10-30T17:44:58.911Z [notification-worker@c5e91622592e] Mensaje recibido orderId=ORD-2CF8BF57 correlationId=corr-221ceb5c-2
75b-4006-90e4-ebce374542ca redelivered=false deliveryTag=4
2026-10-30T17:45:01.915Z [notification-worker@c5e91622592e] Notificación enviada. ACK confirmado orderId=ORD-2CF8BF57 correlati
onId=corr-221ceb5c-275b-4006-90e4-ebce374542ca
2026-10-30T17:45:01.916Z [notification-worker@c5e91622592e] Mensaje recibido orderId=ORD-C208AFA0 correlationId=corr-fcec9a3b-6
c5d-4772-9dd8-5032f6f72963 redelivered=false deliveryTag=5
2026-10-30T17:45:04.919Z [notification-worker@c5e91622592e] Notificación enviada. ACK confirmado orderId=ORD-C208AFA0 correlati
onId=corr-fcec9a3b-6c5d-4772-9dd8-5032f6f72963
```

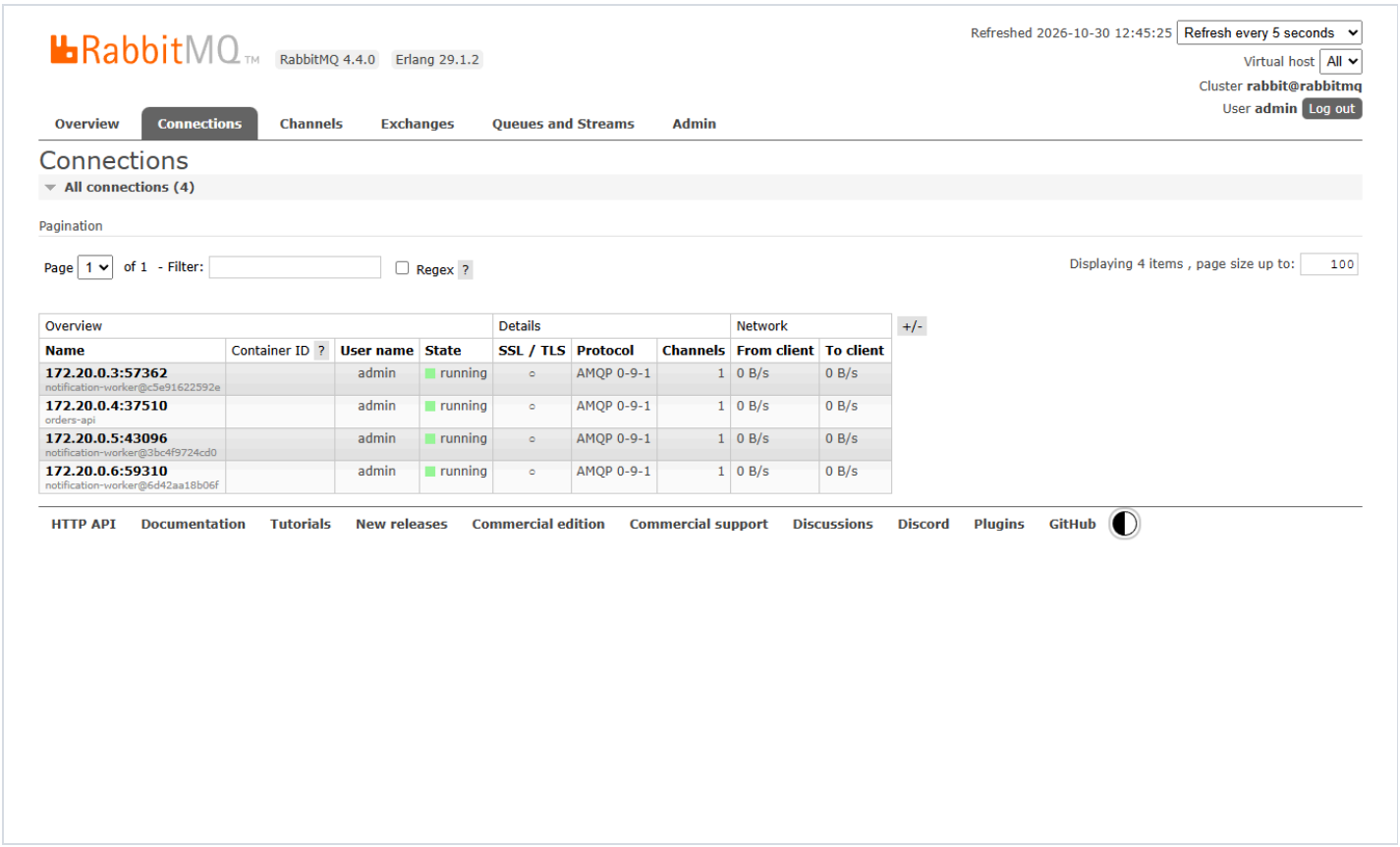
CP-10 · Elastic: distribución de carga con múltiples consumidores RabbitMQ

ID del caso	CP-10
Arquitectura	RabbitMQ
Objetivo	Comprobar que varias instancias del worker se reparten los mensajes de la cola.
Precondiciones	• Stack `rabbitmq/docker-compose.yml` en ejecución (rabbitmq, orders-api, notification-worker).
Datos de entrada	9 pedidos (Escala1…Escala9).
Pasos ejecutados	1. `docker compose up -d --scale notification-worker=3`. 2. Verificar consumers=3. 3. Enviar 9 pedidos. 4. Contar en los logs qué instancia procesó cada pedido.
Resultado esperado	Los 9 mensajes se reparten entre las 3 instancias; ninguno se procesa dos veces.
Resultado obtenido	consumers=3; durante el procesamiento Unacked=3 (uno por worker). Reparto final: 3 / 3 / 3 mensajes (instancias 3bc4f9724cd0, 6d42aa18b06f, c5e91622592e), sin duplicados.
Estado	**Cumple**
Análisis técnico	RabbitMQ entrega en *round-robin* entre consumidores de la misma cola. Con `prefetch(1)` ningún worker acapara mensajes: cada uno recibe el siguiente solo después de confirmar el anterior. Escalar no requirió cambios de código porque el servicio no declara `container_name`.

Evidencia



Consola RabbitMQ: 3 consumidores con prefetch 1 y Unacked=3 (`rabbitmq/img/07-cp10-tres-consumidores.png`)



Consola RabbitMQ: conexiones de las 3 instancias del worker `(rabbitmq/img/08-cp10-conexiones-3-workers.png)`

Ejecución CP-10

`evidencias/rabbitmq/CP-10.txt`

```
==================== CP-10 Elastic: distribución de carga con múltiples consumidores ====================
# Ejecutado: 2026-10-30 12:45:14 (hora local) / 17:45:14 UTC

$ docker compose up -d --scale notification-worker=3 2>&1 | grep -E 'Started|Running'
 Container rabbitmq Running
 Container rabbitmq-pedidos-notification-worker-1 Running
 Container orders-api Running
 Container rabbitmq-pedidos-notification-worker-3 Started
 Container rabbitmq-pedidos-notification-worker-2 Started

$ docker compose ps notification-worker --format 'table {{.Name}}\t{{.Status}}'
NAME                                      STATUS
rabbitmq-pedidos-notification-worker-1   Up 27 seconds
rabbitmq-pedidos-notification-worker-2   Up 1 second
rabbitmq-pedidos-notification-worker-3   Up 1 second
[cola pedidos] durable=true Ready=0 Unacked=0 Total=0 consumers=3

# Se envían 9 pedidos
Pedido 1:{"status":"accepted","message":"Pedido recibido y enviado a RabbitMQ","orderId":"ORD-00498E99","correlationId":"corr-0
2465bef-155b-4605-978b-5597628a0c25"}  -> HTTP 202 en 0.004498s
Pedido 2:{"status":"accepted","message":"Pedido recibido y enviado a RabbitMQ","orderId":"ORD-237E4B02","correlationId":"corr-b
00d31e6-b23b-4181-8437-305f515aad91"}  -> HTTP 202 en 0.004010s
Pedido 3:{"status":"accepted","message":"Pedido recibido y enviado a RabbitMQ","orderId":"ORD-51A1D4BB","correlationId":"corr-5
c2ed29e-d3d3-4fa0-8163-8f709d9bea3d"}  -> HTTP 202 en 0.004203s
Pedido 4:{"status":"accepted","message":"Pedido recibido y enviado a RabbitMQ","orderId":"ORD-A55ACF12","correlationId":"corr-0
a3ff298-b726-47fb-a0c5-15906d961620"}  -> HTTP 202 en 0.004696s
Pedido 5:{"status":"accepted","message":"Pedido recibido y enviado a RabbitMQ","orderId":"ORD-F1F2854B","correlationId":"corr-9
4ae1c8f-4124-468d-bf7e-61f231c6e9c1"}  -> HTTP 202 en 0.004136s
Pedido 6:{"status":"accepted","message":"Pedido recibido y enviado a RabbitMQ","orderId":"ORD-76063D78","correlationId":"corr-2
98391c8-135f-4c16-be6c-89f2511c97eb"}  -> HTTP 202 en 0.003951s
Pedido 7:{"status":"accepted","message":"Pedido recibido y enviado a RabbitMQ","orderId":"ORD-0B198BDA","correlationId":"corr-1
e9b8e3b-432a-4cb8-b027-9cbd575751d6"}  -> HTTP 202 en 0.004353s
```

```
Pedido 8:{"status":"accepted","message":"Pedido recibido y enviado a RabbitMQ","orderId":"ORD-1617EE61","correlationId":"corr-8
4253d57-b80b-4fd4-a0f3-e174036330ae"}  -> HTTP 202 en 0.004214s
Pedido 9:{"status":"accepted","message":"Pedido recibido y enviado a RabbitMQ","orderId":"ORD-C735BE67","correlationId":"corr-2
191242c-8981-48e8-b600-7d75a453a751"}  -> HTTP 202 en 0.004347s

# Con prefetch=1 cada worker toma un mensaje: Unacked=3
[cola pedidos] durable=true Ready=6 Unacked=3 Total=9 consumers=3

# Pedidos procesados por cada instancia

$ docker compose logs --no-log-prefix --since 2026-10-30T17:45:14Z notification-worker | grep 'ACK confirmado' | sed -E 's/.*\
[(notification-worker@[a-f0-9]+)\].*orderId=(.*)/\1 \2/' | sort
notification-worker@3bc4f9724cd0 ORD-1617EE61 correlationId=corr-84253d57-b80b-4fd4-a0f3-e174036330ae
notification-worker@3bc4f9724cd0 ORD-237E4B02 correlationId=corr-b00d31e6-b23b-4181-8437-305f515aad91
notification-worker@3bc4f9724cd0 ORD-F1F2854B correlationId=corr-94ae1c8f-4124-468d-bf7e-61f231c6e9c1
notification-worker@6d42aa18b06f ORD-51A1D4BB correlationId=corr-5c2ed29e-d3d3-4fa0-8163-8f709d9bea3d
notification-worker@6d42aa18b06f ORD-76063D78 correlationId=corr-298391c8-135f-4c16-be6c-89f2511c97eb
notification-worker@6d42aa18b06f ORD-C735BE67 correlationId=corr-2191242c-8981-48e8-b600-7d75a453a751
notification-worker@c5e91622592e ORD-00498E99 correlationId=corr-02465bef-155b-4605-978b-5597628a0c25
notification-worker@c5e91622592e ORD-0B198BDA correlationId=corr-1e9b8e3b-432a-4cb8-b027-9cbd575751d6
notification-worker@c5e91622592e ORD-A55ACF12 correlationId=corr-0a3ff298-b726-47fb-a0c5-15906d961620

$ docker compose logs --no-log-prefix --since 2026-10-30T17:45:14Z notification-worker | grep 'ACK confirmado' | grep -oE 'noti
fication-worker@[a-f0-9]+' | sort | uniq -c
      3 notification-worker@3bc4f9724cd0
      3 notification-worker@6d42aa18b06f
      3 notification-worker@c5e91622592e
```

CP-12 · Recuperación tras reinicio del consumidor y del broker RabbitMQ

ID del caso	CP-12
Arquitectura	RabbitMQ
Objetivo	Verificar que los mensajes no confirmados (consumidor) y los pendientes (broker) se procesan sin pérdida tras un reinicio.
Precondiciones	• Stack `rabbitmq/docker-compose.yml` en ejecución (rabbitmq, orders-api, notification-worker). • Una sola instancia del worker.
Datos de entrada	A) 3 pedidos (Recupera1…3). B) 3 pedidos (Broker1…3) + 1 pedido tras el reinicio (Broker4).
Pasos ejecutados	1. A) Enviar 3 pedidos; con uno en Unacked, matar el worker con `docker kill` (SIGKILL). 2. A) Consultar la cola y volver a levantar el worker. 3. B) Detener el worker, enviar 3 pedidos y reiniciar el broker (`docker compose restart rabbitmq`). 4. B) Consultar la cola tras el reinicio, enviar un pedido nuevo y levantar el worker.
Resultado esperado	A) El mensaje Unacked vuelve a Ready y se reprocesa (redelivered=true). B) Los 3 mensajes persisten tras reiniciar el broker, la API se reconecta sola y el worker procesa los 4.
Resultado obtenido	A) Antes del kill: Ready=2, Unacked=1. Después: Ready=3, Unacked=0. Al volver, el worker procesó ORD-B867DF6A con `redelivered=true` y luego los otros 2. B) Ready=3 antes y después del reinicio del broker. La API registró "Conexión con RabbitMQ cerrada. Reintentando..." y se reconectó 8 s después; el pedido Broker4 fue aceptado (202). El worker procesó los 4 mensajes.
Estado	**Cumple**
Análisis técnico	El ACK manual garantiza entrega *al menos una vez*: si el consumidor muere sin confirmar, el broker reentrega el mensaje. La cola durable, los mensajes persistentes, el volumen de datos y el `hostname` fijo del broker hacen que los mensajes sobrevivan a un reinicio. La reconexión automática de la API evita reiniciarla manualmente. Riesgo observado: el mensaje reentregado se procesa dos veces (aquí, notificación duplicada), por lo que en producción el consumidor debería ser idempotente.

Evidencia

Ejecución CP-12 — `evidencias/rabbitmq/CP-12.txt`

```
==================== CP-12 Recuperación tras reinicio del consumidor y del broker ====================
# Ejecutado: 2026-10-30 12:45:36 (hora local) / 17:45:36 UTC

$ docker compose up -d --scale notification-worker=1 2>&1 | grep -E 'Stopped|Removed|Running' | head -n 5
 Container rabbitmq Running
 Container orders-api Running
```

```
 Container rabbitmq-pedidos-notification-worker-1 Running
 Container rabbitmq-pedidos-notification-worker-3 Stopped
 Container rabbitmq-pedidos-notification-worker-2 Stopped

# A) Caída abrupta del worker con un mensaje entregado y no confirmado
Pedido 1:{"status":"accepted","message":"Pedido recibido y enviado a RabbitMQ","orderId":"ORD-B867DF6A","correlationId":"corr-d
c83c1ab-4bab-4dd1-bec9-3f5c9138f45c"}  -> HTTP 202 en 0.004878s
Pedido 2:{"status":"accepted","message":"Pedido recibido y enviado a RabbitMQ","orderId":"ORD-674353DA","correlationId":"corr-d
1788639-aa93-4ece-8bb4-e138c216c02c"}  -> HTTP 202 en 0.003881s
Pedido 3:{"status":"accepted","message":"Pedido recibido y enviado a RabbitMQ","orderId":"ORD-94C7FA35","correlationId":"corr-1
a1022ed-0081-4fc0-825a-f8729bd9b6ea"}  -> HTTP 202 en 0.003753s
[cola pedidos] durable=true Ready=2 Unacked=1 Total=3 consumers=1

$ docker kill $(docker compose ps -q notification-worker)
c5e91622592e2e5b8570be10b1aa111fb6630900bfe5b1a4c12a4ce1cce8b558

# El mensaje Unacked vuelve a Ready: no se pierde
[cola pedidos] durable=true Ready=3 Unacked=0 Total=3 consumers=0

$ docker compose up -d notification-worker 2>&1 | tail -n 1
 Container rabbitmq-pedidos-notification-worker-1 Started
[cola pedidos] durable=true Ready=0 Unacked=0 Total=0 consumers=1

$ docker compose logs --no-log-prefix --since 2026-10-30T17:45:45Z notification-worker | grep -E 'Mensaje recibido|ACK'
2026-10-30T17:45:46.828Z [notification-worker@c5e91622592e] Mensaje recibido orderId=ORD-B867DF6A correlationId=corr-dc83c1ab-4
bab-4dd1-bec9-3f5c9138f45c redelivered=true deliveryTag=1
2026-10-30T17:45:49.831Z [notification-worker@c5e91622592e] Notificación enviada. ACK confirmado orderId=ORD-B867DF6A correlati
onId=corr-dc83c1ab-4bab-4dd1-bec9-3f5c9138f45c
2026-10-30T17:45:49.833Z [notification-worker@c5e91622592e] Mensaje recibido orderId=ORD-674353DA correlationId=corr-d1788639-a
a93-4ece-8bb4-e138c216c02c redelivered=false deliveryTag=2
2026-10-30T17:45:52.836Z [notification-worker@c5e91622592e] Notificación enviada. ACK confirmado orderId=ORD-674353DA correlati
onId=corr-d1788639-aa93-4ece-8bb4-e138c216c02c
2026-10-30T17:45:52.837Z [notification-worker@c5e91622592e] Mensaje recibido orderId=ORD-94C7FA35 correlationId=corr-1a1022ed-0
081-4fc0-825a-f8729bd9b6ea redelivered=false deliveryTag=3
2026-10-30T17:45:55.838Z [notification-worker@c5e91622592e] Notificación enviada. ACK confirmado orderId=ORD-94C7FA35 correlati
onId=corr-1a1022ed-0081-4fc0-825a-f8729bd9b6ea

# B) Reinicio del broker con mensajes pendientes (cola durable + mensajes persistentes)

$ docker compose stop notification-worker 2>&1
 Container rabbitmq-pedidos-notification-worker-1 Stopping
 Container rabbitmq-pedidos-notification-worker-1 Stopped
Pedido 1:{"status":"accepted","message":"Pedido recibido y enviado a RabbitMQ","orderId":"ORD-CC50F3CA","correlationId":"corr-d
398f8f7-d7f5-401d-b1d6-88b2b87c29d0"}  -> HTTP 202 en 0.005546s
Pedido 2:{"status":"accepted","message":"Pedido recibido y enviado a RabbitMQ","orderId":"ORD-6064FCE9","correlationId":"corr-4
52aa9c9-9348-48e3-b67d-36aca396c522"}  -> HTTP 202 en 0.003913s
Pedido 3:{"status":"accepted","message":"Pedido recibido y enviado a RabbitMQ","orderId":"ORD-9B7ADC08","correlationId":"corr-7
95063f5-a364-4d95-bf7b-acf93e418711"}  -> HTTP 202 en 0.004024s
[cola pedidos] durable=true Ready=3 Unacked=0 Total=3 consumers=0

$ docker compose restart rabbitmq 2>&1
 Container rabbitmq Restarting
 Container rabbitmq Started

# Tras reiniciar el broker los 3 mensajes siguen en la cola
[cola pedidos] durable=true Ready=3 Unacked=0 Total=3 consumers=0

$ docker compose logs --no-log-prefix orders-api | tail -n 4
2026-10-30T17:46:02.566Z [orders-api] Conexión con RabbitMQ cerrada. Reintentando...
2026-10-30T17:46:04.570Z [orders-api] RabbitMQ no disponible (intento 1): connect ECONNREFUSED 172.20.0.2:5672
2026-10-30T17:46:07.572Z [orders-api] RabbitMQ no disponible (intento 2): connect ECONNREFUSED 172.20.0.2:5672
2026-10-30T17:46:10.627Z [orders-api] Conectado a RabbitMQ (amqp://***@rabbitmq:5672). Cola "pedidos" declarada (durable=true)

# La API se reconecta sola y sigue publicando
Pedido 4:{"status":"accepted","message":"Pedido recibido y enviado a RabbitMQ","orderId":"ORD-26B3A7ED","correlationId":"corr-4
5f848bc-ec7e-4a28-965d-566ab46a50b6"}  -> HTTP 202 en 0.005057s

$ docker compose start notification-worker 2>&1
 Container rabbitmq Waiting
 Container rabbitmq Healthy
```

```
 Container rabbitmq-pedidos-notification-worker-1 Starting
 Container rabbitmq-pedidos-notification-worker-1 Started
[cola pedidos] durable=true Ready=0 Unacked=0 Total=0 consumers=1

$ docker compose logs --no-log-prefix --since 2026-10-30T17:46:12Z notification-worker | grep -E 'Conectado|Mensaje recibido|ACK'
2026-10-30T17:46:13.461Z [notification-worker@c5e91622592e] Conectado a RabbitMQ. Escuchando la cola "pedidos" (prefetch=1, ack manual, procesamiento=3000ms)
2026-10-30T17:46:13.464Z [notification-worker@c5e91622592e] Mensaje recibido orderId=ORD-CC50F3CA correlationId=corr-d398f8f7-d7f5-401d-b1d6-88b2b87c29d0 redelivered=false deliveryTag=1
2026-10-30T17:46:16.468Z [notification-worker@c5e91622592e] Notificación enviada. ACK confirmado orderId=ORD-CC50F3CA correlationId=corr-d398f8f7-d7f5-401d-b1d6-88b2b87c29d0
2026-10-30T17:46:16.470Z [notification-worker@c5e91622592e] Mensaje recibido orderId=ORD-6064FCE9 correlationId=corr-452aa9c9-9348-48e3-b67d-36aca396c522 redelivered=false deliveryTag=2
2026-10-30T17:46:19.473Z [notification-worker@c5e91622592e] Notificación enviada. ACK confirmado orderId=ORD-6064FCE9 correlationId=corr-452aa9c9-9348-48e3-b67d-36aca396c522
2026-10-30T17:46:19.475Z [notification-worker@c5e91622592e] Mensaje recibido orderId=ORD-9B7ADC08 correlationId=corr-795063f5-a364-4d95-bf7b-acf93e418711 redelivered=false deliveryTag=3
2026-10-30T17:46:22.476Z [notification-worker@c5e91622592e] Notificación enviada. ACK confirmado orderId=ORD-9B7ADC08 correlationId=corr-795063f5-a364-4d95-bf7b-acf93e418711
2026-10-30T17:46:22.478Z [notification-worker@c5e91622592e] Mensaje recibido orderId=ORD-26B3A7ED correlationId=corr-45f848bc-ec7e-4a28-965d-566ab46a50b6 redelivered=false deliveryTag=4
2026-10-30T17:46:25.479Z [notification-worker@c5e91622592e] Notificación enviada. ACK confirmado orderId=ORD-26B3A7ED correlationId=corr-45f848bc-ec7e-4a28-965d-566ab46a50b6
```

5. Ejecución de pruebas: Kafka

Evidencias requeridas por la guía: **tópico creado** (sección 3.2 y CP-04), **evento publicado** (CP-04), **evento consumido** (CP-05). Pruebas obligatorias: **consumidor posterior** (CP-11 parte B), **múltiples consumidores** (CP-05 con grupos distintos y CP-10 con instancias del mismo grupo) y **desacoplamiento total** (CP-07, CP-08, CP-09, CP-11).

CP-01 · Creación de pedido vía API Kafka

ID del caso	CP-01
Arquitectura	Kafka
Objetivo	Validar que el endpoint `POST /orders` recibe el pedido, lo valida y responde correctamente.
Precondiciones	• Stack `kafka/docker-compose.yml` en ejecución (kafka, kafka-init, kafka-ui, orders-api, inventory/billing/notification-consumer). • Productor conectado a Kafka.
Datos de entrada	`{"customerName":"Carlos","product":"Libro de arquitectura","quantity":1,"unitPrice":85000}`; casos negativos con `product` vacío y `quantity:-1`, y texto no JSON.
Pasos ejecutados	1. Enviar el pedido válido con curl. 2. Enviar los pedidos inválidos. 3. Revisar el log de orders-api.
Resultado esperado	HTTP 202 con `status: accepted` y el mensaje "Pedido recibido y evento OrderCreated publicado en Kafka"; HTTP 400 para los inválidos.
Resultado obtenido	HTTP 202 en 0,034 s (primer envío, incluye la obtención de metadatos del tópico) con `orderId=ORD-702E978A`, `eventId` y `correlationId`. Los inválidos: HTTP 400 con los errores (incluido `unitPrice` faltante).
Estado	**Cumple**
Análisis técnico	La API valida también `unitPrice`, que en Kafka es obligatorio para calcular el `total` del evento. El primer envío es más lento porque el productor descarga los metadatos de particiones; los siguientes tardan 5–8 ms (ver CP-07).

Evidencia

Ejecución CP-01 — `evidencias/kafka/CP-01.txt`

```
==================== CP-01 Creación de pedido vía API ====================
# Ejecutado: 2026-10-30 12:47:48 (hora local) / 17:47:48 UTC

# Pedido válido

$ curl -s -i -X POST http://localhost:3010/orders -H 'Content-Type: application/json' -d '{"customerName":"Carlos","product":"L
ibro de arquitectura","quantity":1,"unitPrice":85000}' -w '\n(tiempo total: %{time_total}s)\n'
HTTP/1.1 202 Accepted
Content-Type: application/json; charset=utf-8
Date: Fri, 30 Oct 2026 17:47:48 GMT
Connection: keep-alive
Keep-Alive: timeout=5
Transfer-Encoding: chunked

{"status":"accepted","message":"Pedido recibido y evento OrderCreated publicado en Kafka","orderId":"ORD-702E978A","eventId":"e
vt-ae5a5cee-9cc2-42ea-b072-81e253581434","correlationId":"corr-77f34890-73a6-4aec-a7be-9cca34d7c8dd"}
(tiempo total: 0.033644s)
```

```
# Pedidos inválidos (validación de JSON básico)

$ curl -s -i -X POST http://localhost:3010/orders -H 'Content-Type: application/json' -d '{"customerName":"Carlos","produc
t":"","quantity":-1}'
HTTP/1.1 400 Bad Request
Content-Type: application/json; charset=utf-8
Date: Fri, 30 Oct 2026 17:47:48 GMT
Connection: keep-alive
Keep-Alive: timeout=5
Transfer-Encoding: chunked

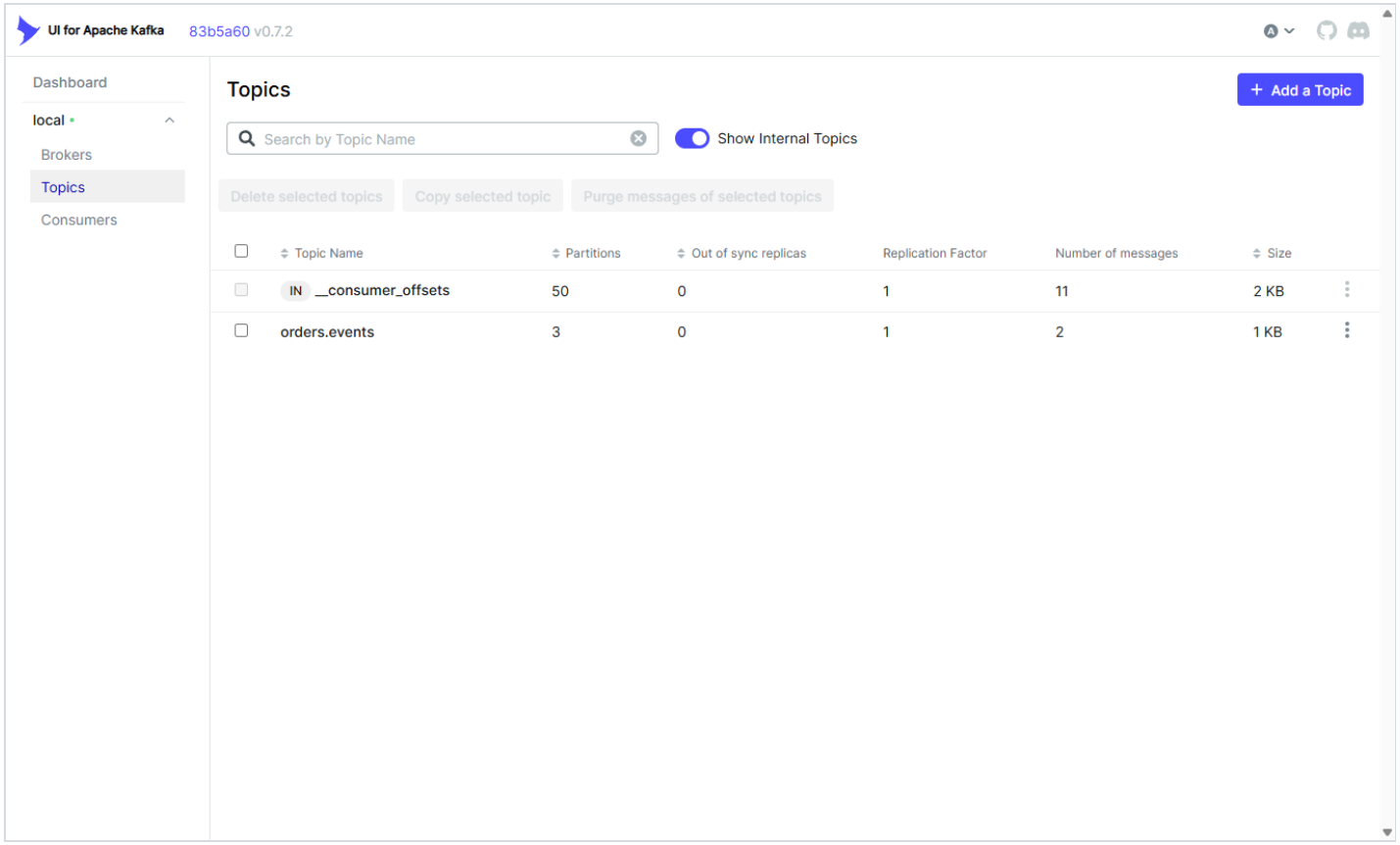
{"status":"rejected","errors":["product es obligatorio","quantity debe ser un entero positivo","unitPrice debe ser un número >=
0"]}
$ curl -s -i -X POST http://localhost:3010/orders -H 'Content-Type: application/json' -d 'esto no es json'
HTTP/1.1 400 Bad Request
Content-Type: application/json; charset=utf-8
Date: Fri, 30 Oct 2026 17:47:49 GMT
Connection: keep-alive
Keep-Alive: timeout=5
Transfer-Encoding: chunked

{"status":"rejected","errors":["JSON inválido"]}
$ docker compose logs --no-log-prefix orders-api | tail -n 3
2026-10-30T17:47:32.900Z [orders-api] Productor conectado a Kafka (kafka:9092). Tópico de publicación: orders.events
2026-10-30T17:47:48.868Z [orders-api] Evento OrderCreated publicado topic=orders.events partition=0 offset=0 orderId=ORD-702E97
8A eventId=evt-ae5a5cee-9cc2-42ea-b072-81e253581434 correlationId=corr-77f34890-73a6-4aec-a7be-9cca34d7c8dd
2026-10-30T17:47:48.942Z [orders-api] Pedido rechazado: product es obligatorio; quantity debe ser un entero positivo; unitPrice
debe ser un número >= 0
```

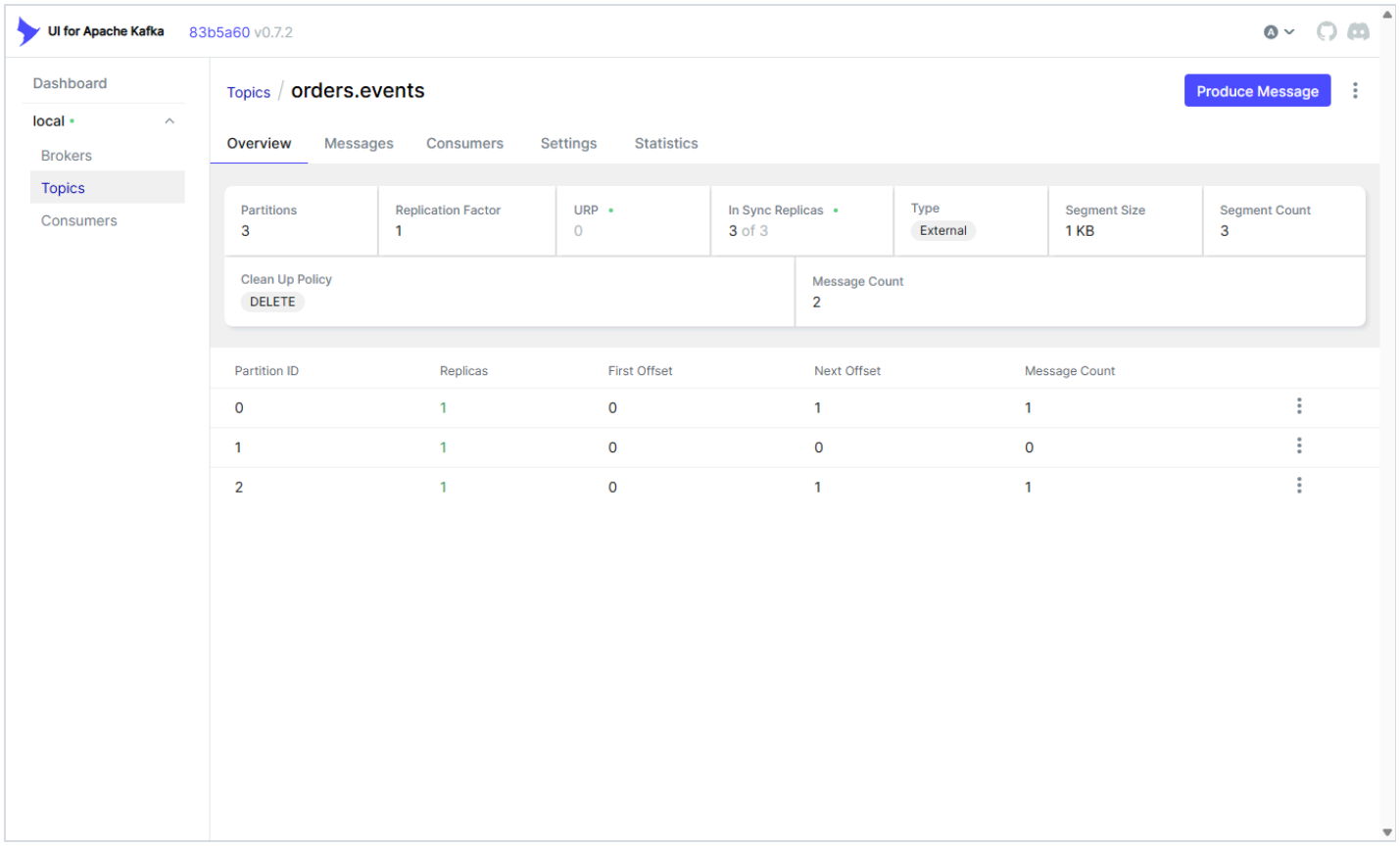
CP-04 · Publicación de evento en Kafka Kafka

ID del caso	CP-04
Arquitectura	Kafka
Objetivo	Verificar que cada pedido genera un evento `OrderCreated` con los metadatos mínimos en el tópico `orders.events`.
Precondiciones	• Stack `kafka/docker-compose.yml` en ejecución (kafka, kafka-init, kafka-ui, orders-api, inventory/billing/notification-consumer). • Tópico `orders.events` creado con 3 particiones por `kafka-init`.
Datos de entrada	`{"customerName":"Pedro","product":"Libro de arquitectura","quantity":1,"unitPrice":85000}`
Pasos ejecutados	1. Consultar los offsets finales del tópico. 2. Enviar el pedido. 3. Revisar el log de orders-api (partición y offset asignados). 4. Consultar de nuevo los offsets. 5. Leer el evento directamente del broker con `kafka-console-consumer.sh` y en Kafka UI.
Resultado esperado	Una partición avanza en 1. El evento tiene `eventId`, `eventType=OrderCreated`, `eventVersion`, `occurredAt`, `source`, `correlationId` y `data`, con key = orderId.
Resultado obtenido	Offsets 0:1 / 1:0 / 2:0 → 0:1 / 1:0 / 2:1. La API registró `partition=2 offset=0 orderId=ORD-D640D38E`. El evento leído del broker (Partition:2 Offset:0, key ORD-D640D38E) y en Kafka UI contiene todos los campos, con `total=85000`.
Estado	**Cumple**
Análisis técnico	El nombre en pasado (`OrderCreated`) expresa un hecho ocurrido, no una orden. Usar el orderId como key garantiza que los eventos de un mismo pedido vayan a la misma partición y conserven su orden. Las cabeceras Kafka también llevan `eventType` y `correlationId`.

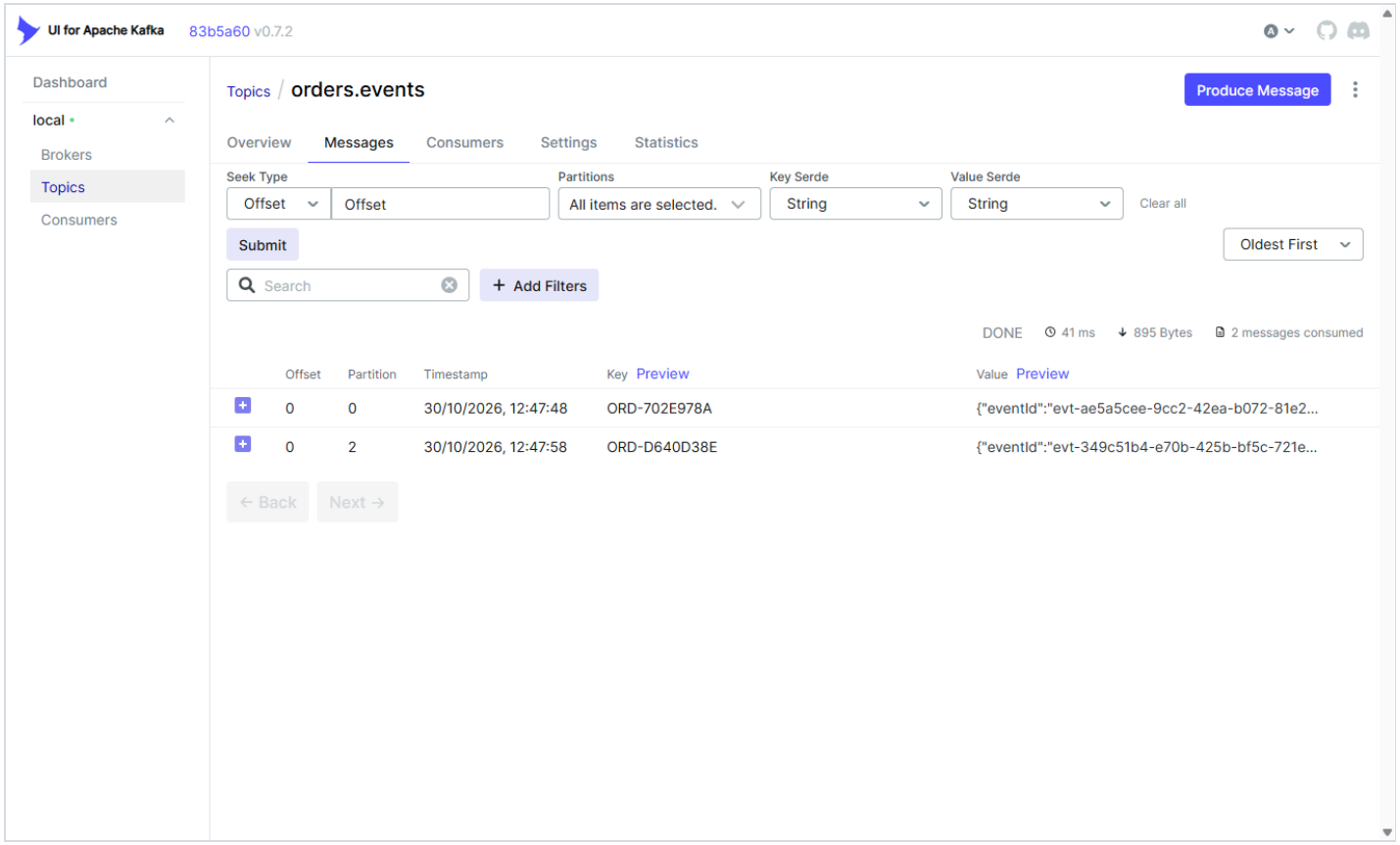
Evidencia



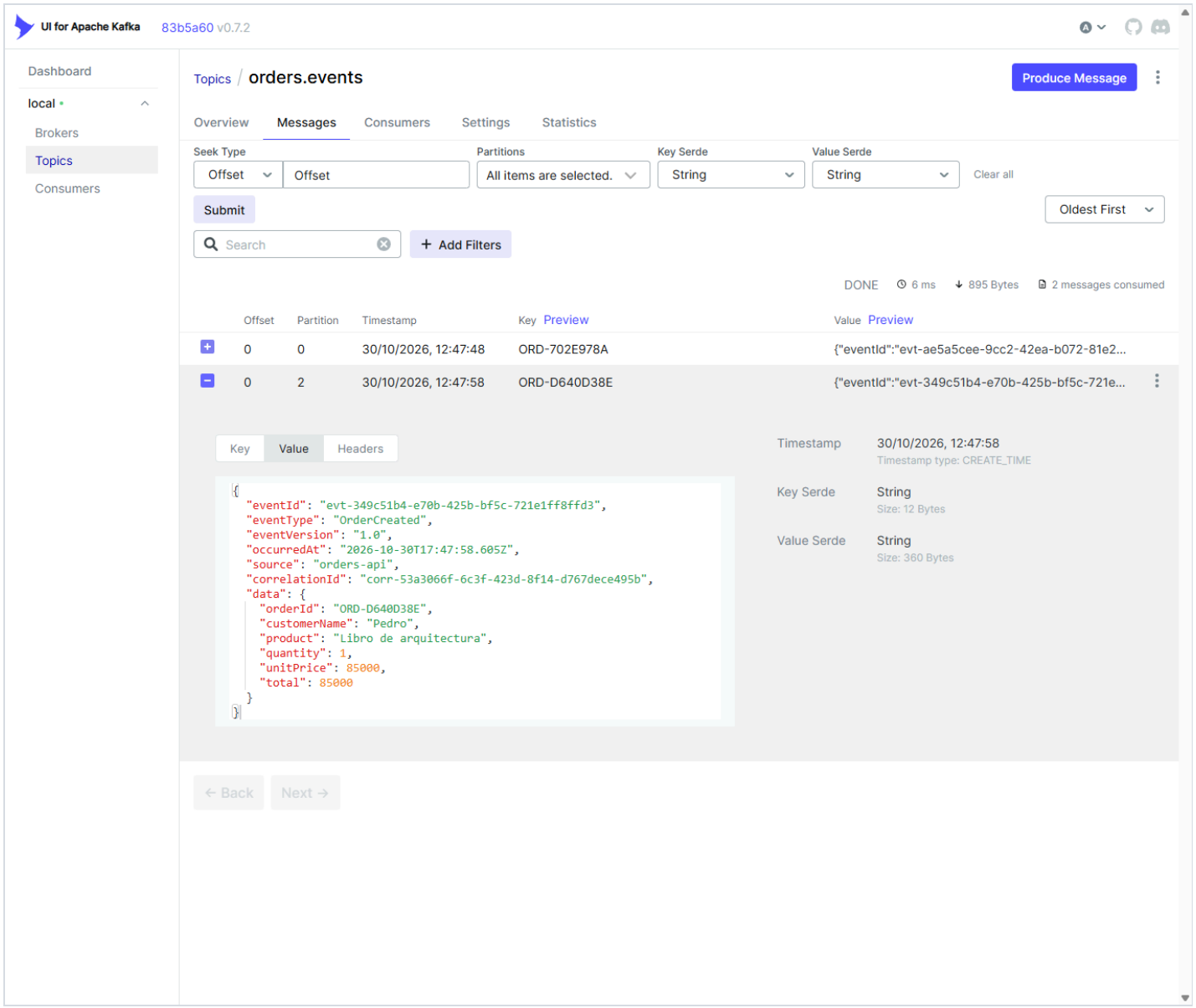
Kafka UI: tópico orders.events (`kafka/img/03-topicos.png`)



Kafka UI: 3 particiones y mensajes por partición (`kafka/img/04-topico-particiones.png`)



Kafka UI: mensajes del tópico con offset, partición y key (`kafka/img/05-cp04-mensajes.png`)



Kafka UI: contenido JSON del evento OrderCreated (`kafka/img/05-cp04-contenido-evento.png`)

Ejecución CP-04 — `evidencias/kafka/CP-04.txt`

```
==================== CP-04 Publicación de evento en Kafka ====================
# Ejecutado: 2026-10-30 12:47:57 (hora local) / 17:47:57 UTC

# Offsets del tópico antes de publicar (topic:partición:offset final)
orders.events:0:1
orders.events:1:0
orders.events:2:0

# Pedido
{"status":"accepted","message":"Pedido recibido y evento OrderCreated publicado en Kafka","orderId":"ORD-D640D38E","eventId":"e
vt-349c51b4-e70b-425b-bf5c-721e1ff8ffd3","correlationId":"corr-53a3066f-6c3f-423d-8f14-d767dece495b"}  -> HTTP 202 en 0.008312s

# Log de orders-api: evento publicado con partición y offset

$ docker compose logs --no-log-prefix orders-api | grep ORD-D640D38E
2026-10-30T17:47:58.608Z [orders-api] Evento OrderCreated publicado topic=orders.events partition=2 offset=0 orderId=ORD-D640D3
8E eventId=evt-349c51b4-e70b-425b-bf5c-721e1ff8ffd3 correlationId=corr-53a3066f-6c3f-423d-8f14-d767dece495b

# Offsets del tópico después de publicar: una partición avanzó en 1
orders.events:0:1
```

```
orders.events:1:0
orders.events:2:1

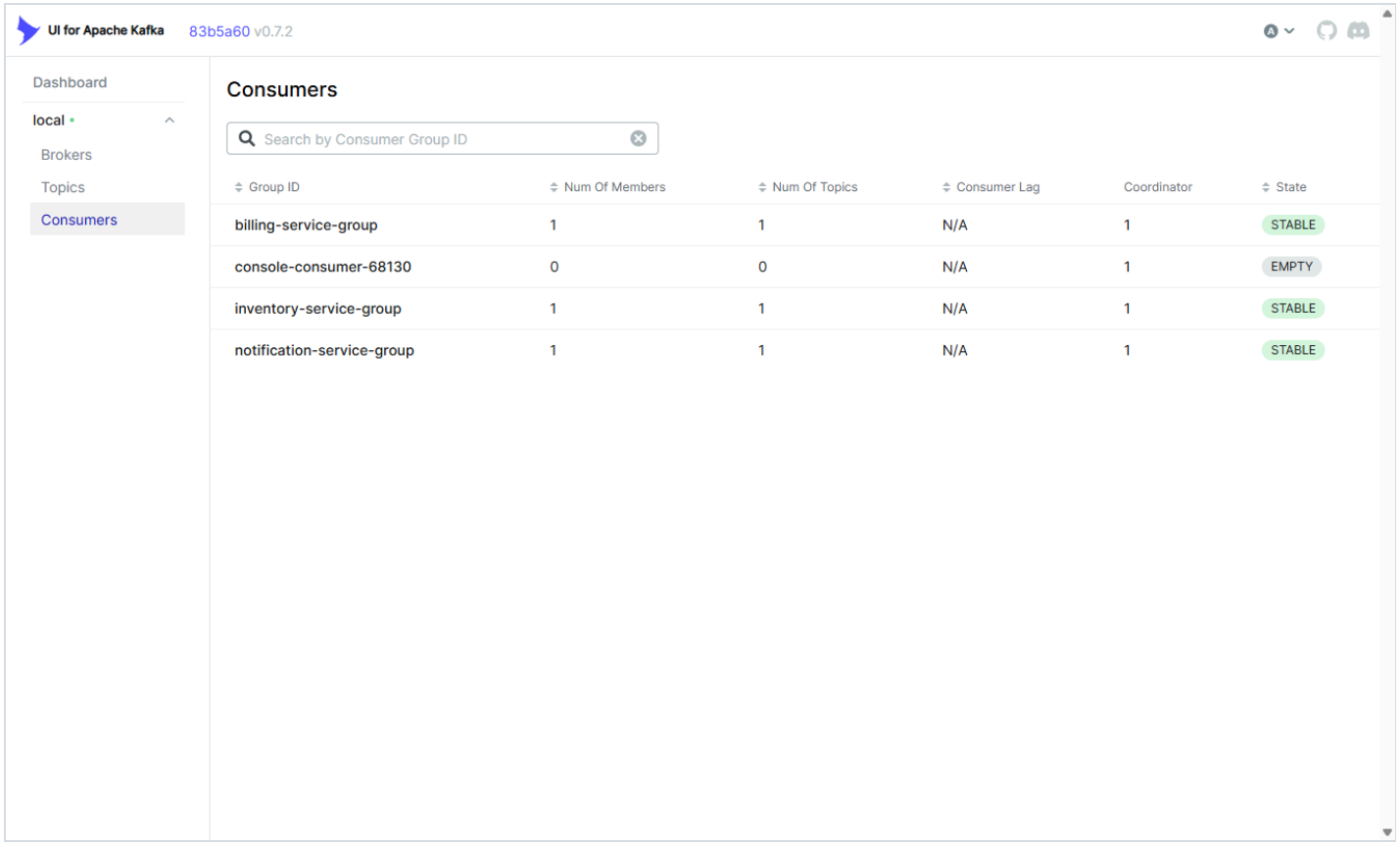
# Evento almacenado en el tópico (leído directamente del broker)

$ docker exec kafka kafka-console-consumer.sh --bootstrap-server kafka:9092 --topic orders.events --from-beginning --timeout-ms
6000 --property print.key=true --property print.partition=true --property print.offset=true 2>/dev/null | grep ORD-D640D38E
Partition:2     Offset:0        ORD-D640D38E    {"eventId":"evt-349c51b4-e70b-425b-bf5c-721e1ff8ffd3","eventType":"OrderCreate
d","eventVersion":"1.0","occurredAt":"2026-10-30T17:47:58.605Z","source":"orders-api","correlationId":"corr-53a3066f-6c3f-423d-
8f14-d767dece495b","data":{"orderId":"ORD-D640D38E","customerName":"Pedro","product":"Libro de arquitectura","quantity":1,"unit
Price":85000,"total":85000}}
```

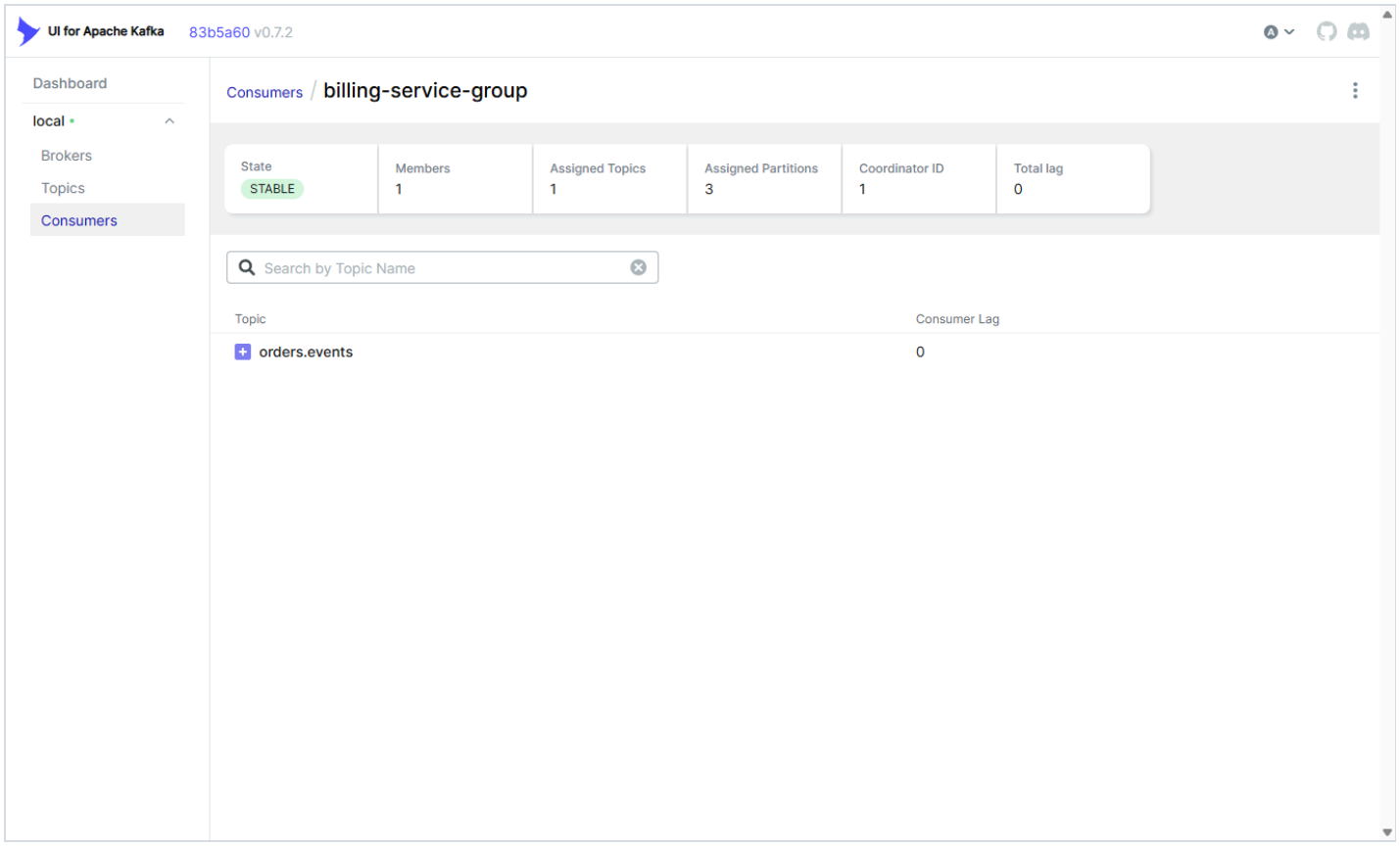
CP-05 · Consumo de evento (Kafka) Kafka

ID del caso	CP-05
Arquitectura	Kafka
Objetivo	Validar que cada consumidor, en su propio consumer group, recibe el mismo evento.
Precondiciones	• Stack `kafka/docker-compose.yml` en ejecución (kafka, kafka-init, kafka-ui, orders-api, inventory/billing/notification-consumer). • Evento ORD-D640D38E publicado en CP-04.
Datos de entrada	Evento `ORD-D640D38E` (partición 2, offset 0).
Pasos ejecutados	1. Esperar LAG=0 en los tres grupos. 2. Buscar el orderId en el log de inventory, billing y notification. 3. Listar y describir los consumer groups en el broker y en Kafka UI.
Resultado esperado	Los tres consumidores registran el mismo eventId con topic/partition/offset y ejecutan su acción; existen tres grupos distintos con LAG 0.
Resultado obtenido	inventory: "Inventario reservado: 1 x Libro de arquitectura"; billing: "Factura generada por $85000"; notification: "Notificación enviada a Pedro". Los tres registran `partition=2 offset=0` y el mismo eventId. Grupos inventory-, billing- y notification-service-group en estado STABLE con LAG 0.
Estado	**Cumple**
Análisis técnico	Cada grupo mantiene su propio offset, así que un evento se entrega una vez a cada grupo (publicación/suscripción). El grupo `console-consumer-68130` lo creó la herramienta de inspección usada en CP-04; no pertenece al sistema.

Evidencia



Kafka UI: tres consumer groups distintos (`kafka/img/06-cp05-consumer-groups.png`)



Kafka UI: billing-service-group, 3 particiones asignadas y LAG 0 (`kafka/img/07-cp05-grupo-billing-offsets.png`)

Ejecución CP-05 — `evidencias/kafka/CP-05.txt`

```
==================== CP-05 Consumo de evento (Kafka) ====================
# Ejecutado: 2026-10-30 12:48:10 (hora local) / 17:48:10 UTC

# Cada consumidor, en su propio consumer group, recibió el mismo evento (ORD-D640D38E)

$ docker compose logs --no-log-prefix inventory-consumer | grep -A 3 'orderId=ORD-D640D38E eventId' | head -n 4
2026-10-30T17:47:58.611Z [inventory-consumer@d5d9dfbc2cc6] Evento recibido: OrderCreated orderId=ORD-D640D38E eventId=evt-349c5
1b4-e70b-425b-bf5c-721e1ff8ffd3 correlationId=corr-53a3066f-6c3f-423d-8f14-d767dece495b
2026-10-30T17:47:58.611Z [inventory-consumer@d5d9dfbc2cc6] topic=orders.events partition=2 offset=0 key=ORD-D640D38E
2026-10-30T17:47:58.611Z [inventory-consumer@d5d9dfbc2cc6] Simulando reserva de inventario...
2026-10-30T17:47:59.611Z [inventory-consumer@d5d9dfbc2cc6] Inventario reservado: 1 x "Libro de arquitectura" (orderId=ORD-D640D
38E)

$ docker compose logs --no-log-prefix billing-consumer | grep -A 3 'orderId=ORD-D640D38E eventId' | head -n 4
2026-10-30T17:47:58.610Z [billing-consumer@286de68024c8] Evento recibido: OrderCreated orderId=ORD-D640D38E eventId=evt-349c51b
4-e70b-425b-bf5c-721e1ff8ffd3 correlationId=corr-53a3066f-6c3f-423d-8f14-d767dece495b
2026-10-30T17:47:58.610Z [billing-consumer@286de68024c8] topic=orders.events partition=2 offset=0 key=ORD-D640D38E
2026-10-30T17:47:58.611Z [billing-consumer@286de68024c8] Simulando generación de factura...
2026-10-30T17:47:59.610Z [billing-consumer@286de68024c8] Factura generada por $85000 (1 x $85000) (orderId=ORD-D640D38E)

$ docker compose logs --no-log-prefix notification-consumer | grep -A 3 'orderId=ORD-D640D38E eventId' | head -n 4
2026-10-30T17:47:58.610Z [notification-consumer@38db5bcd9c8a] Evento recibido: OrderCreated orderId=ORD-D640D38E eventId=evt-34
9c51b4-e70b-425b-bf5c-721e1ff8ffd3 correlationId=corr-53a3066f-6c3f-423d-8f14-d767dece495b
2026-10-30T17:47:58.610Z [notification-consumer@38db5bcd9c8a] topic=orders.events partition=2 offset=0 key=ORD-D640D38E
2026-10-30T17:47:58.610Z [notification-consumer@38db5bcd9c8a] Simulando envío de notificación al cliente...
2026-10-30T17:47:59.610Z [notification-consumer@38db5bcd9c8a] Notificación enviada a Pedro (orderId=ORD-D640D38E)

# Consumer groups registrados en el broker

$ docker exec kafka kafka-consumer-groups.sh --bootstrap-server kafka:9092 --list
console-consumer-68130
billing-service-group
```

```
inventory-service-group
notification-service-group

# Grupo inventory-service-group
GROUP                  TOPIC           PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG             CONSUMER-ID
HOST            CLIENT-ID
inventory-service-group orders.events   0          1               1               0               inventory-consumer-6103019b-
0c4b-42f3-923e-00096f3390bf /172.20.0.3     inventory-consumer
inventory-service-group orders.events   1          -               0               -               inventory-consumer-6103019b-
0c4b-42f3-923e-00096f3390bf /172.20.0.3     inventory-consumer
inventory-service-group orders.events   2          1               1               0               inventory-consumer-6103019b-
0c4b-42f3-923e-00096f3390bf /172.20.0.3     inventory-consumer

# Grupo billing-service-group
GROUP                TOPIC           PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG             CONSUMER-ID
HOST            CLIENT-ID
billing-service-group orders.events   0          1               1               0               billing-consumer-fce4b9fb-5ec6
-4077-9dde-885094a02779 /172.20.0.5     billing-consumer
billing-service-group orders.events   1          -               0               -               billing-consumer-fce4b9fb-5ec6
-4077-9dde-885094a02779 /172.20.0.5     billing-consumer
billing-service-group orders.events   2          1               1               0               billing-consumer-fce4b9fb-5ec6
-4077-9dde-885094a02779 /172.20.0.5     billing-consumer

# Grupo notification-service-group
GROUP                     TOPIC           PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG             CONSUMER-ID
HOST            CLIENT-ID
notification-service-group orders.events   0          1               1               0               notification-consumer-126
a305a-eefe-4e5a-9513-f6b3afab2ae3 /172.20.0.6     notification-consumer
notification-service-group orders.events   1          -               0               -               notification-consumer-126
a305a-eefe-4e5a-9513-f6b3afab2ae3 /172.20.0.6     notification-consumer
notification-service-group orders.events   2          1               1               0               notification-consumer-126
a305a-eefe-4e5a-9513-f6b3afab2ae3 /172.20.0.6     notification-consumer
```

CP-06 · Trazabilidad del flujo Kafka

ID del caso	CP-06
Arquitectura	Kafka
Objetivo	Seguir un mismo pedido (orderId / correlationId) en los logs de la API, en el broker y en los consumidores.
Precondiciones	• Stack `kafka/docker-compose.yml` en ejecución (kafka, kafka-init, kafka-ui, orders-api, inventory/billing/notification-consumer).
Datos de entrada	Cabecera `X-Correlation-Id: corr-CP06-124929`; pedido de Laura, 2 × Curso de QA a 120000.
Pasos ejecutados	1. Enviar el pedido con el correlationId. 2. Buscarlo en el log de orders-api. 3. Leer del broker el registro con partición, offset, cabeceras, key y valor. 4. Buscarlo en el log de los tres consumidores.
Resultado esperado	El mismo orderId/correlationId en la API, en el log del tópico y en los tres consumidores, con la misma partición y offset.
Resultado obtenido	`ORD-296E793F`: API "publicado partition=1 offset=0" (17:49:29.271); broker Partition:1 Offset:0 con cabeceras `eventType:OrderCreated,correlationId:corr-CP06-124929`; inventory, billing y notification lo recibieron en partition=1 offset=0 (17:49:29.273–.274) y completaron su acción 1 s después.
Estado	**Cumple**
Análisis técnico	La pareja (partición, offset) identifica de forma única el registro en el broker y coincide en los cuatro servicios. Junto con el correlationId permite reconstruir el flujo completo y ver que los tres consumidores reaccionaron en paralelo e independientemente.

Evidencia

Ejecución CP-06 — `evidencias/kafka/CP-06.txt`

```
==================== CP-06 Trazabilidad del flujo (correlationId) ====================
# Ejecutado: 2026-10-30 12:49:29 (hora local) / 17:49:29 UTC

# Pedido con cabecera X-Correlation-Id: corr-CP06-124929
{"status":"accepted","message":"Pedido recibido y evento OrderCreated publicado en Kafka","orderId":"ORD-296E793F","eventId":"e
vt-726d1ac0-ae11-465f-937d-e3ce02132eaf","correlationId":"corr-CP06-124929"}  -> HTTP 202 en 0.010045s

# 1) Log de la API (productor)

$ docker compose logs --no-log-prefix orders-api | grep corr-CP06-124929
2026-10-30T17:49:29.271Z [orders-api] Evento OrderCreated publicado topic=orders.events partition=1 offset=0 orderId=ORD-296E79
3F eventId=evt-726d1ac0-ae11-465f-937d-e3ce02132eaf correlationId=corr-CP06-124929

# 2) Broker: registro del evento en el log del tópico (partición, offset, headers, key y valor)

$ docker exec kafka kafka-console-consumer.sh --bootstrap-server kafka:9092 --topic orders.events --from-beginning --timeout-ms
6000 --property print.partition=true --property print.offset=true --property print.headers=true --property print.key=true 2>/de
v/null | grep corr-CP06-124929
```

```
Partition:1     Offset:0        eventType:OrderCreated,correlationId:corr-CP06-124929   ORD-296E793F    {"eventId":"evt-726d1ac
0-ae11-465f-937d-e3ce02132eaf","eventType":"OrderCreated","eventVersion":"1.0","occurredAt":"2026-10-30T17:49:29.267Z","sourc
e":"orders-api","correlationId":"corr-CP06-124929","data":{"orderId":"ORD-296E793F","customerName":"Laura","product":"Curso de
QA","quantity":2,"unitPrice":120000,"total":240000}}

# 3) Logs de los consumidores

$ docker compose logs --no-log-prefix inventory-consumer | grep -A 3 'correlationId=corr-CP06-124929' | head -n 4
2026-10-30T17:49:29.273Z [inventory-consumer@d5d9dfbc2cc6] Evento recibido: OrderCreated orderId=ORD-296E793F eventId=evt-726d1
ac0-ae11-465f-937d-e3ce02132eaf correlationId=corr-CP06-124929
2026-10-30T17:49:29.273Z [inventory-consumer@d5d9dfbc2cc6] topic=orders.events partition=1 offset=0 key=ORD-296E793F
2026-10-30T17:49:29.273Z [inventory-consumer@d5d9dfbc2cc6] Simulando reserva de inventario...
2026-10-30T17:49:30.272Z [inventory-consumer@d5d9dfbc2cc6] Inventario reservado: 2 x "Curso de QA" (orderId=ORD-296E793F)

$ docker compose logs --no-log-prefix billing-consumer | grep -A 3 'correlationId=corr-CP06-124929' | head -n 4
2026-10-30T17:49:29.274Z [billing-consumer@286de68024c8] Evento recibido: OrderCreated orderId=ORD-296E793F eventId=evt-726d1ac
0-ae11-465f-937d-e3ce02132eaf correlationId=corr-CP06-124929
2026-10-30T17:49:29.274Z [billing-consumer@286de68024c8] topic=orders.events partition=1 offset=0 key=ORD-296E793F
2026-10-30T17:49:29.274Z [billing-consumer@286de68024c8] Simulando generación de factura...
2026-10-30T17:49:30.274Z [billing-consumer@286de68024c8] Factura generada por $240000 (2 x $120000) (orderId=ORD-296E793F)

$ docker compose logs --no-log-prefix notification-consumer | grep -A 3 'correlationId=corr-CP06-124929' | head -n 4
2026-10-30T17:49:29.274Z [notification-consumer@38db5bcd9c8a] Evento recibido: OrderCreated orderId=ORD-296E793F eventId=evt-72
6d1ac0-ae11-465f-937d-e3ce02132eaf correlationId=corr-CP06-124929
2026-10-30T17:49:29.274Z [notification-consumer@38db5bcd9c8a] topic=orders.events partition=1 offset=0 key=ORD-296E793F
2026-10-30T17:49:29.274Z [notification-consumer@38db5bcd9c8a] Simulando envío de notificación al cliente...
2026-10-30T17:49:30.274Z [notification-consumer@38db5bcd9c8a] Notificación enviada a Laura (orderId=ORD-296E793F)
```

CP-07 · Responsive: la API responde sin esperar a los consumidores Kafka

ID del caso	CP-07
Arquitectura	Kafka
Objetivo	Demostrar que la respuesta HTTP no depende del procesamiento de los consumidores.
Precondiciones	• Stack `kafka/docker-compose.yml` en ejecución (kafka, kafka-init, kafka-ui, orders-api, inventory/billing/notification-consumer). • Consumidores con `PROCESSING_MS=1000`.
Datos de entrada	10 pedidos (Cliente1…Cliente10).
Pasos ejecutados	1. Enviar 10 pedidos midiendo cada respuesta. 2. Consultar el LAG de los tres grupos inmediatamente. 3. Esperar LAG=0 y medir el tiempo.
Resultado esperado	Respuestas en milisegundos con LAG > 0 justo después; los consumidores terminan segundos más tarde.
Resultado obtenido	10 × HTTP 202 (463 ms en total). LAG inmediato: inventory 9, billing 9, notification 6. Los tres grupos llegaron a LAG 0 11 s después.
Estado	**Cumple**
Análisis técnico	El LAG mide cuántos eventos publicados aún no procesó cada grupo: que sea mayor que 0 después de responder demuestra que la API no esperó. Cada grupo avanza a su propio ritmo (notification iba algo más adelantado).

Evidencia

Ejecución CP-07 — `evidencias/kafka/CP-07.txt`

```
==================== CP-07 Responsive: la API responde sin esperar a los consumidores ====================
# Ejecutado: 2026-10-30 12:49:45 (hora local) / 17:49:45 UTC

# Cada consumidor tarda 1 s por evento (PROCESSING_MS=1000). Se envían 10 pedidos seguidos.
Pedido  1:{"status":"accepted","message":"Pedido recibido y evento OrderCreated publicado en Kafka","orderId":"ORD-11746766","e
ventId":"evt-fd54b50f-2f81-47d4-a7c1-39e6d6a69979","correlationId":"corr-925bbef7-db8d-486b-8567-f24e31abc940"}  -> HTTP 202 en
0.007412s
Pedido  2:{"status":"accepted","message":"Pedido recibido y evento OrderCreated publicado en Kafka","orderId":"ORD-99EDE493","e
ventId":"evt-293c3752-9165-4402-ac44-2f67afe90dc9","correlationId":"corr-72926886-4dc5-4864-9d1b-b95a858647d8"}  -> HTTP 202 en
0.006670s
Pedido  3:{"status":"accepted","message":"Pedido recibido y evento OrderCreated publicado en Kafka","orderId":"ORD-EA5FF83E","e
ventId":"evt-5f315924-0c08-46b4-aa53-983a58243625","correlationId":"corr-40d026a1-682c-4dd6-a213-6840edec53d0"}  -> HTTP 202 en
0.006287s
Pedido  4:{"status":"accepted","message":"Pedido recibido y evento OrderCreated publicado en Kafka","orderId":"ORD-5EC14BDC","e
ventId":"evt-6cb21518-8293-4bc6-99f8-a1e15db3b9c1","correlationId":"corr-52c28e8f-5aa1-42f1-ab43-76f57cfbad7f"}  -> HTTP 202 en
0.006549s
Pedido  5:{"status":"accepted","message":"Pedido recibido y evento OrderCreated publicado en Kafka","orderId":"ORD-6AF90985","e
ventId":"evt-1458416b-f480-4995-8ff2-78de9185373c","correlationId":"corr-9c9a3852-331d-4d6d-9cdf-7443d0c35af7"}  -> HTTP 202 en
0.005248s
Pedido  6:{"status":"accepted","message":"Pedido recibido y evento OrderCreated publicado en Kafka","orderId":"ORD-8EEC0154","e
ventId":"evt-24ce7e8e-a83a-4a3b-b6a7-37186334f060","correlationId":"corr-c817d9bc-2eb4-4e34-bb7a-6d6acbd874be"}  -> HTTP 202 en
0.006085s
Pedido  7:{"status":"accepted","message":"Pedido recibido y evento OrderCreated publicado en Kafka","orderId":"ORD-F16975F4","e
ventId":"evt-bd47a8a1-905e-423f-bcea-41b8f3c6ead0","correlationId":"corr-f79003a5-0f5f-4f33-a787-ca643777ee2e"}  -> HTTP 202 en
```

```
0.005847s
Pedido  8:{"status":"accepted","message":"Pedido recibido y evento OrderCreated publicado en Kafka","orderId":"ORD-9B9438D9","e
ventId":"evt-3a55348e-11bb-4314-a1be-d709cd297074","correlationId":"corr-e7659d5f-e6b2-4508-b9ac-a2ebf38da312"}  -> HTTP 202 en
0.006006s
Pedido  9:{"status":"accepted","message":"Pedido recibido y evento OrderCreated publicado en Kafka","orderId":"ORD-F8C14D9C","e
ventId":"evt-825ec1df-164f-4aa6-98e8-fd6f13c3a365","correlationId":"corr-eae02125-dd3f-46c9-8ee7-b1a92ea5add4"}  -> HTTP 202 en
0.004869s
Pedido 10:{"status":"accepted","message":"Pedido recibido y evento OrderCreated publicado en Kafka","orderId":"ORD-2B12202C","e
ventId":"evt-52ac87be-ae7d-471c-9226-72e99f10bad7","correlationId":"corr-20961e37-28c3-4664-bb1b-8a60df9ec3a0"}  -> HTTP 202 en
0.005323s
Tiempo total de las 10 respuestas HTTP: 463 ms

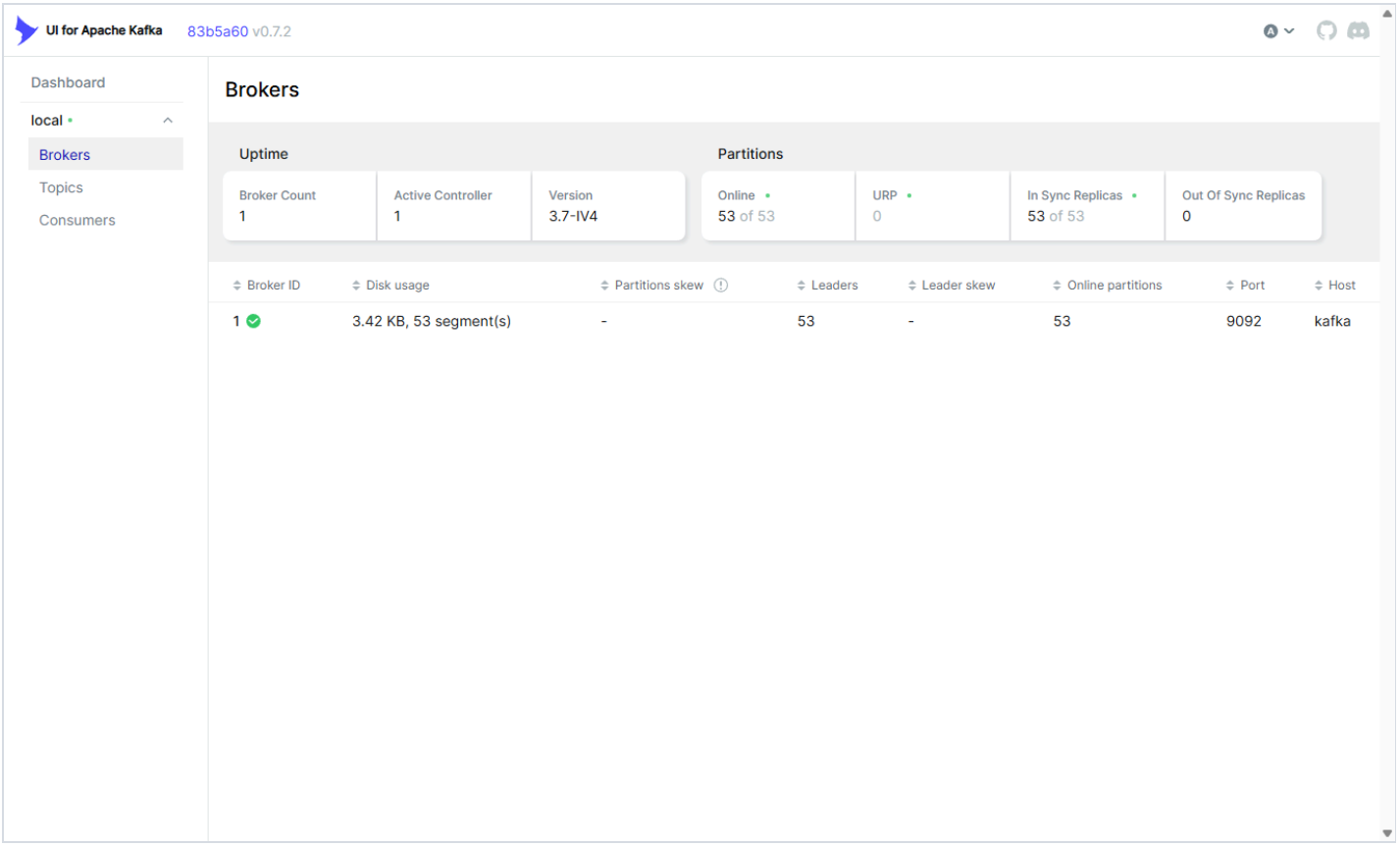
# Inmediatamente después: los consumidores aún tienen eventos pendientes (LAG > 0)
inventory-service-group LAG=9
billing-service-group LAG=9
notification-service-group LAG=6

# Todos los grupos alcanzaron LAG=0 después de 11 s adicionales
inventory-service-group LAG=0
billing-service-group LAG=0
notification-service-group LAG=0
```

CP-08 · Message-driven: no existe comunicación directa Kafka

ID del caso	CP-08
Arquitectura	Kafka
Objetivo	Comprobar que la API y los consumidores solo se comunican a través de Kafka.
Precondiciones	• Stack `kafka/docker-compose.yml` en ejecución (kafka, kafka-init, kafka-ui, orders-api, inventory/billing/notification-consumer).
Datos de entrada	No aplica (inspección de red, puertos y código).
Pasos ejecutados	1. Listar los puertos publicados. 2. Listar los sockets en escucha en cada consumidor. 3. Listar las conexiones establecidas de la API y de los consumidores. 4. Intentar una llamada HTTP de la API a inventory-consumer. 5. Buscar en el código llamadas HTTP salientes o servidores HTTP.
Resultado esperado	Los consumidores no publican ni escuchan puertos y solo se conectan a kafka:9092; la llamada HTTP directa falla.
Resultado obtenido	Solo publican puertos kafka (29092), kafka-ui (8080) y orders-api (3010→3000). Los consumidores solo escuchan en 127.0.0.11 (DNS de Docker). Todas las conexiones ESTABLISHED van a 172.20.0.2:9092 (kafka). `wget http://inventory-consumer:3000`: "Connection refused". El único servidor HTTP del código es el de orders-api.
Estado	**Cumple**
Análisis técnico	El productor no conoce a los consumidores: solo conoce el tópico. Por eso se pudo agregar un consumidor nuevo (audit-consumer, CP-11) sin tocar la API.

Evidencia



Kafka UI: broker activo (id 1, puerto 9092) (`kafka/img/02-broker-activo.png`)

Ejecución CP-08

`evidencias/kafka/CP-08.txt`

```
==================== CP-08 Message-driven: no existe comunicación directa entre servicios ====================
# Ejecutado: 2026-10-30 12:50:07 (hora local) / 17:50:07 UTC

# Los consumidores no publican puertos; solo orders-api (3000), kafka (29092) y kafka-ui (8080)

$ docker compose ps --format 'table {{.Name}}\t{{.Ports}}'
NAME                                    PORTS
kafka                                   0.0.0.0:29092->29092/tcp, [::]:29092->29092/tcp
kafka-pedidos-billing-consumer-1
kafka-pedidos-inventory-consumer-1
kafka-pedidos-notification-consumer-1
kafka-ui                                0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
orders-api                              0.0.0.0:3010->3000/tcp, [::]:3010->3000/tcp

# Sockets en escucha dentro de cada consumidor: solo 127.0.0.11 (DNS interno de Docker), no hay servidor HTTP

$ docker compose exec inventory-consumer netstat -tln; echo '(fin de la lista de inventory-consumer)'
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 127.0.0.11:36757        0.0.0.0:*               LISTEN
(fin de la lista de inventory-consumer)

$ docker compose exec billing-consumer netstat -tln; echo '(fin de la lista de billing-consumer)'
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 127.0.0.11:33923        0.0.0.0:*               LISTEN
(fin de la lista de billing-consumer)

$ docker compose exec notification-consumer netstat -tln; echo '(fin de la lista de notification-consumer)'
Active Internet connections (only servers)
```

```
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 127.0.0.11:45019        0.0.0.0:*               LISTEN
(fin de la lista de notification-consumer)

# Conexiones TCP establecidas: todos hablan solo con kafka:9092

$ docker inspect -f '{{.Name}} {{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' kafka orders-api $(docker compose ps -
q inventory-consumer billing-consumer notification-consumer)
/kafka 172.20.0.2
/orders-api 172.20.0.7
/kafka-pedidos-billing-consumer-1 172.20.0.5
/kafka-pedidos-inventory-consumer-1 172.20.0.3
/kafka-pedidos-notification-consumer-1 172.20.0.6

$ docker compose exec orders-api netstat -tn | grep ESTABLISHED
tcp        0      0 172.20.0.7:38998        172.20.0.2:9092         ESTABLISHED

$ docker compose exec inventory-consumer netstat -tn | grep ESTABLISHED
tcp        0      0 172.20.0.3:32962        172.20.0.2:9092         ESTABLISHED
tcp        0      0 172.20.0.3:32976        172.20.0.2:9092         ESTABLISHED

$ docker compose exec billing-consumer netstat -tn | grep ESTABLISHED
tcp        0      0 172.20.0.5:48568        172.20.0.2:9092         ESTABLISHED
tcp        0      0 172.20.0.5:48576        172.20.0.2:9092         ESTABLISHED

$ docker compose exec notification-consumer netstat -tn | grep ESTABLISHED
tcp        0      0 172.20.0.6:57806        172.20.0.2:9092         ESTABLISHED
tcp        0      0 172.20.0.6:57804        172.20.0.2:9092         ESTABLISHED

# Intento de llamada HTTP directa de la API a un consumidor: debe fallar

$ docker compose exec orders-api wget -q -T 3 -O- http://inventory-consumer:3000/ 2>&1 || echo 'Conexión rechazada: el consumid
or no expone HTTP'
wget: can't connect to remote host (172.20.0.3): Connection refused
Conexión rechazada: el consumidor no expone HTTP

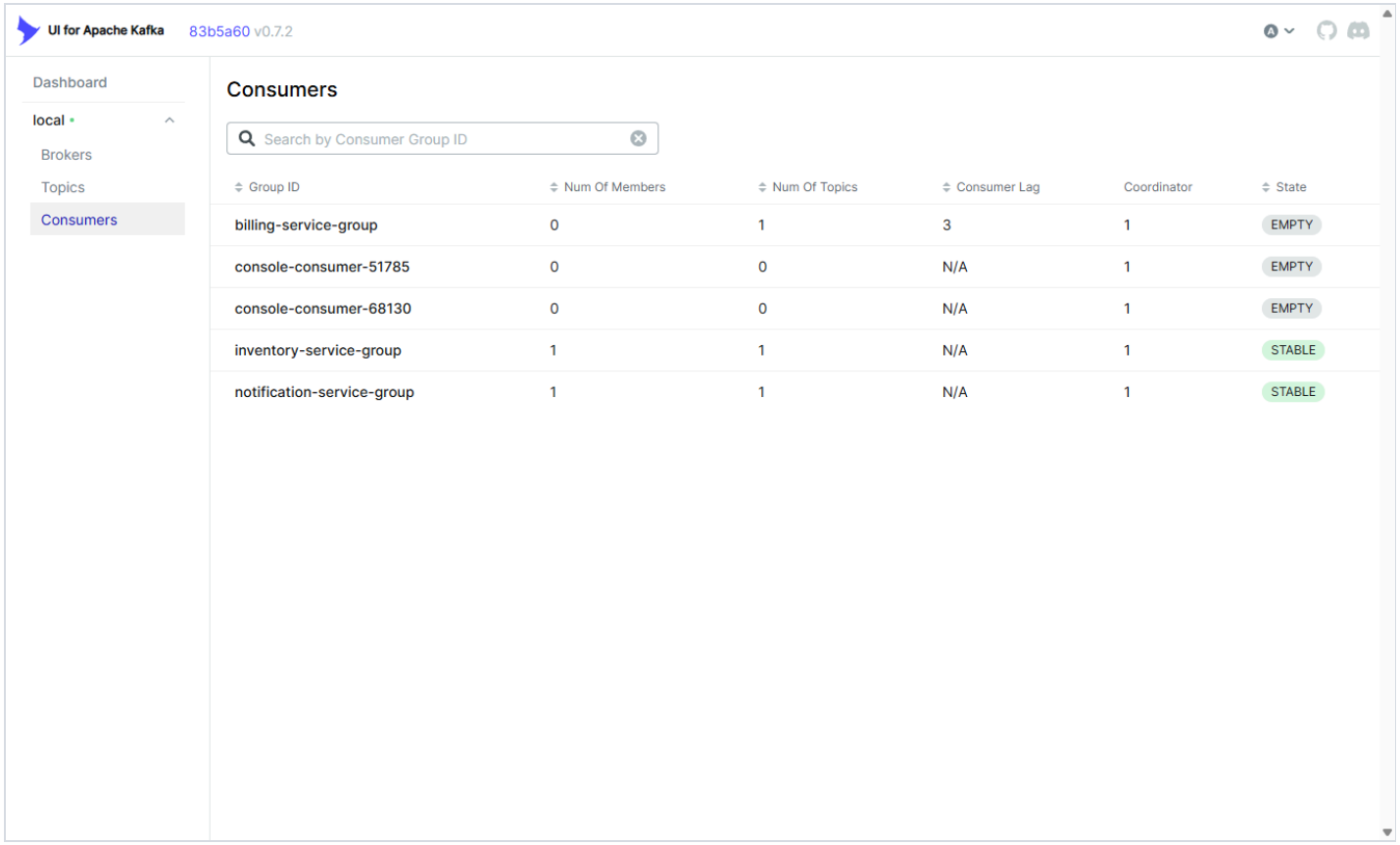
# Revisión del código: sin llamadas HTTP salientes en la API ni servidores HTTP en los consumidores

$ grep -nE 'http\.request|[^e]fetch\(|axios|createServer|listen\(' orders-api/index.js */index.js | sort -u
orders-api/index.js:114:const server = http.createServer((req, res) => {
orders-api/index.js:127:server.listen(PORT, () => {
```

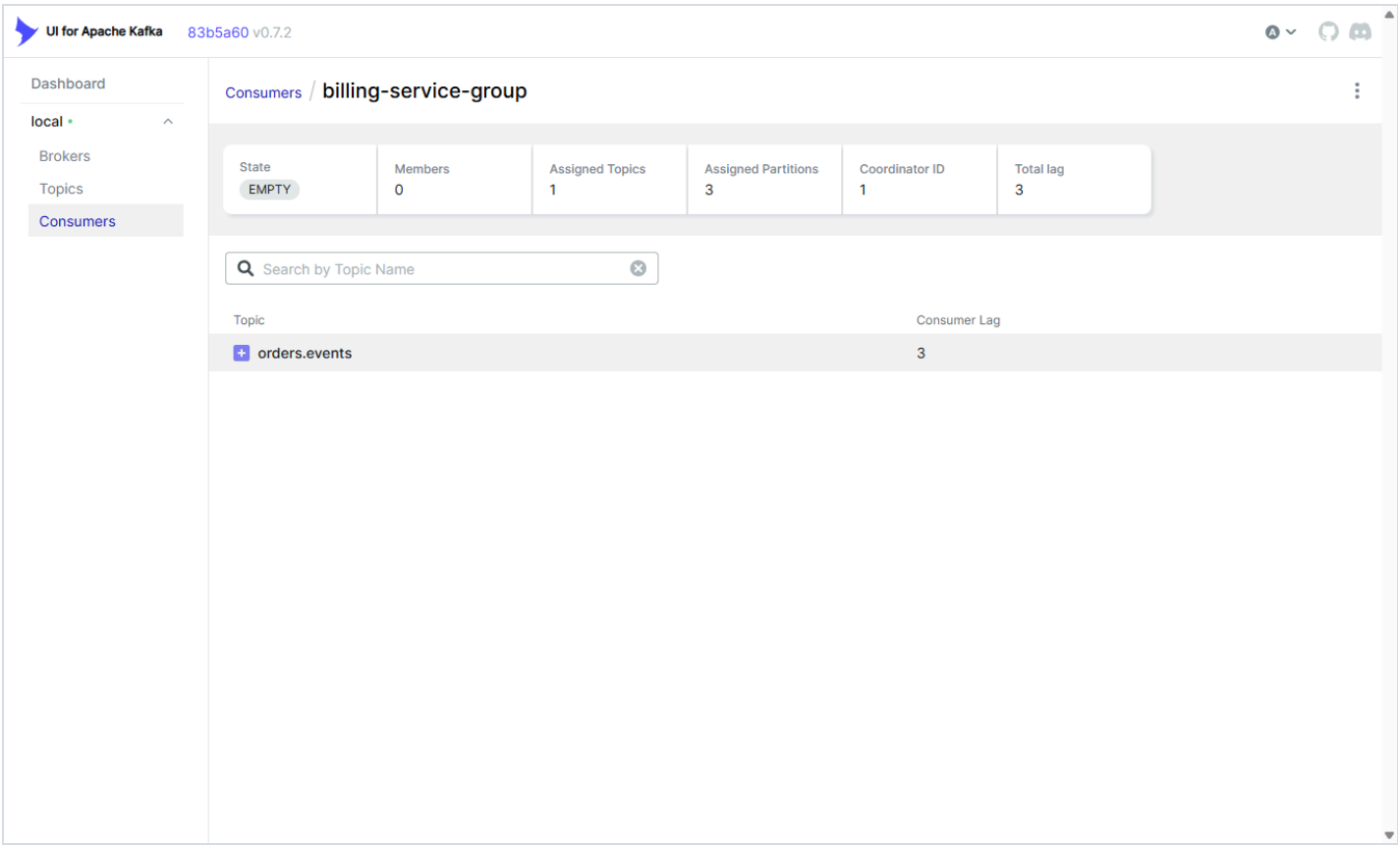
CP-09 · Resilient: el sistema soporta la caída de un consumidor Kafka

ID del caso	CP-09
Arquitectura	Kafka
Objetivo	Verificar que con un consumidor caído la API y los demás consumidores siguen funcionando.
Precondiciones	• Stack `kafka/docker-compose.yml` en ejecución (kafka, kafka-init, kafka-ui, orders-api, inventory/billing/notification-consumer).
Datos de entrada	3 pedidos (SinBilling1…3).
Pasos ejecutados	1. `docker compose stop billing-consumer`. 2. Enviar 3 pedidos. 3. Consultar `/health`. 4. Verificar que inventory y notification los procesaron. 5. Describir el grupo billing-service-group.
Resultado esperado	Las 3 respuestas son 202; inventory y notification procesan los 3 eventos; billing acumula LAG=3 sin miembros activos.
Resultado obtenido	3 × HTTP 202; API UP. inventory reservó y notification notificó los 3 pedidos; billing registró 0 eventos. Grupo billing: LAG 1 + 0 + 2 = 3, sin CONSUMER-ID (estado EMPTY en Kafka UI).
Estado	**Cumple**
Análisis técnico	La caída de un consumidor solo afecta a su propio grupo. Los eventos no se pierden: quedan en el log y el offset confirmado del grupo marca dónde retomar.

Evidencia



Kafka UI: billing-service-group con 0 miembros (EMPTY) `(kafka/img/08-cp09-grupos-billing-sin-miembros.png)`



Kafka UI: billing-service-group con LAG total 3 (`kafka/img/09-cp09-billing-lag3.png`)

Ejecución CP-09 — `evidencias/kafka/CP-09.txt`

```
==================== CP-09 Resilient: el sistema soporta la caída de un consumidor ====================
# Ejecutado: 2026-10-30 12:51:07 (hora local) / 17:51:07 UTC

$ docker compose stop billing-consumer 2>&1
 Container kafka-pedidos-billing-consumer-1 Stopping
 Container kafka-pedidos-billing-consumer-1 Stopped

$ docker compose ps --format 'table {{.Name}}\t{{.Status}}'
NAME                                STATUS
kafka                               Up 3 minutes (healthy)
kafka-pedidos-inventory-consumer-1      Up 3 minutes
kafka-pedidos-notification-consumer-1   Up 3 minutes
kafka-ui                            Up 3 minutes
orders-api                          Up 3 minutes

# Con billing-consumer caído la API sigue aceptando pedidos
Pedido 1:{"status":"accepted","message":"Pedido recibido y evento OrderCreated publicado en Kafka","orderId":"ORD-41652E7F","ev
entId":"evt-d14c141c-63a3-4b9f-b497-f8b55cacf9ad","correlationId":"corr-46b722b8-97de-440b-af5a-d006c2304572"}  -> HTTP 202 en
0.006118s
Pedido 2:{"status":"accepted","message":"Pedido recibido y evento OrderCreated publicado en Kafka","orderId":"ORD-2BAF2935","ev
entId":"evt-b93a18ea-ed93-425e-bb93-54f1b12ddd76","correlationId":"corr-68583954-01c9-4125-81e2-5a4f2f975c46"}  -> HTTP 202 en
0.004939s
Pedido 3:{"status":"accepted","message":"Pedido recibido y evento OrderCreated publicado en Kafka","orderId":"ORD-40AECA0F","ev
entId":"evt-21aa2804-73ec-48da-9067-92df12513838","correlationId":"corr-0b7e56a0-8b09-441c-810a-6c4e4d6cb070"}  -> HTTP 202 en
0.008101s

$ curl -s http://localhost:3010/health; echo
{"status":"UP","broker":"connected"}

# Los demás consumidores siguen procesando normalmente

$ docker compose logs --no-log-prefix --since 2026-10-30T17:51:07Z inventory-consumer notification-consumer | grep -E 'Inventar
```

```
io reservado|Notificación enviada' | sort
2026-10-30T17:51:10.724Z [inventory-consumer@d5d9dfbc2cc6] Inventario reservado: 1 x "Producto offline 1" (orderId=ORD-41652E7
F)
2026-10-30T17:51:10.725Z [notification-consumer@38db5bcd9c8a] Notificación enviada a SinBilling1 (orderId=ORD-41652E7F)
2026-10-30T17:51:11.729Z [inventory-consumer@d5d9dfbc2cc6] Inventario reservado: 3 x "Producto offline 3" (orderId=ORD-40AECA0
F)
2026-10-30T17:51:11.729Z [notification-consumer@38db5bcd9c8a] Notificación enviada a SinBilling2 (orderId=ORD-2BAF2935)
2026-10-30T17:51:12.732Z [inventory-consumer@d5d9dfbc2cc6] Inventario reservado: 2 x "Producto offline 2" (orderId=ORD-2BAF293
5)
2026-10-30T17:51:12.732Z [notification-consumer@38db5bcd9c8a] Notificación enviada a SinBilling3 (orderId=ORD-40AECA0F)

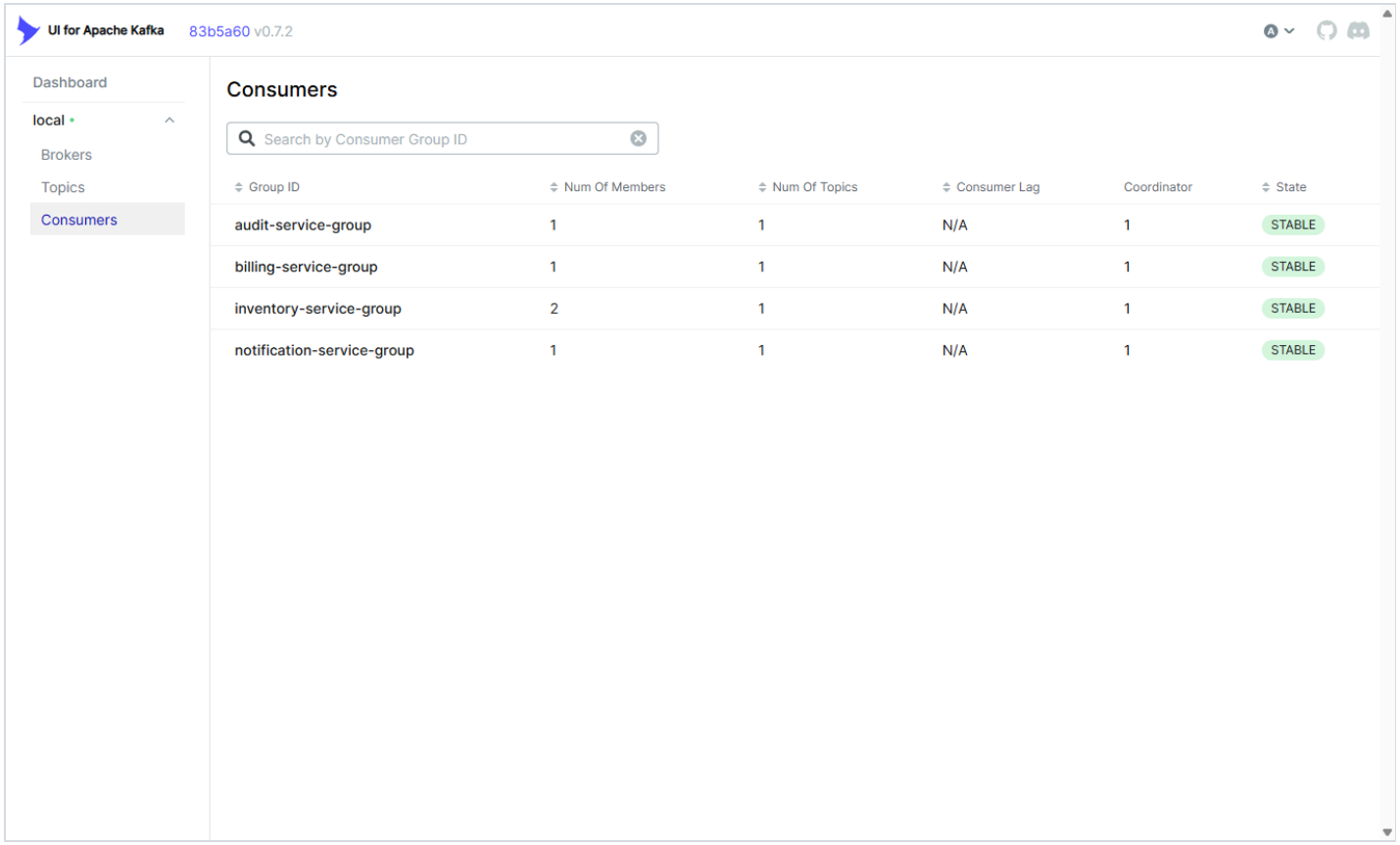
$ docker compose logs --no-log-prefix --since 2026-10-30T17:51:07Z billing-consumer | grep -c 'Evento recibido'
0

# El grupo de facturación no tiene miembros activos y acumula LAG (eventos pendientes en el log)
GROUP                TOPIC           PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG             CONSUMER-ID     HOST
CLIENT-ID
billing-service-group orders.events   0          7               8               1               -               -
-
billing-service-group orders.events   1          5               5               0               -               -
-
billing-service-group orders.events   2          4               6               2               -               -
-
```

CP-11 · Desacoplamiento temporal y consumidor posterior Kafka

ID del caso	CP-11
Arquitectura	Kafka
Objetivo	Verificar que los eventos publicados con un consumidor apagado se procesan al iniciarlo, y que un consumidor creado después de publicar los eventos los lee desde el log.
Precondiciones	• Stack `kafka/docker-compose.yml` en ejecución (kafka, kafka-init, kafka-ui, orders-api, inventory/billing/notification-consumer). • A) 3 eventos pendientes para billing (CP-09). B) El grupo audit-service-group no existe.
Datos de entrada	A) Eventos SinBilling1…3. B) Todo el log del tópico (19 eventos: 8 + 5 + 6 por partición).
Pasos ejecutados	1. A) `docker compose start billing-consumer` y esperar LAG=0. 2. B) Listar los grupos existentes y los offsets finales del tópico. 3. B) `docker compose --profile late up -d audit-consumer` (grupo nuevo, `fromBeginning: true`). 4. B) Contar los eventos leídos y describir el grupo.
Resultado esperado	A) billing procesa los 3 eventos pendientes desde su último offset. B) audit-consumer lee los 19 eventos desde el offset 0 de cada partición.
Resultado obtenido	A) billing se unió al grupo y facturó ORD-40AECA0F (p0 o7), ORD-41652E7F (p2 o4) y ORD-2BAF2935 (p2 o5); LAG final 0. B) audit-consumer leyó las particiones 2 (offsets 0–5), 1 (0–4) y 0 (0–7): 19 eventos, LAG final 0.
Estado	**Cumple**
Análisis técnico	Kafka conserva los eventos aunque ya los hayan leído otros grupos (log persistente con retención), a diferencia de una cola. Un grupo nuevo puede reconstruir toda la historia. Nota de ejecución: la primera corrida de la parte B se detuvo antes de tiempo por un error en la función `lag()` del script (no contaba el LAG de un grupo sin offsets confirmados); se corrigió, se eliminó el grupo de auditoría y la parte B se repitió completa (archivo `CP-11-posterior.txt`).

Evidencia



Kafka UI: aparece audit-service-group junto a los grupos funcionales (`kafka/img/10-cp10-cp11-grupos.png`)

Ejecución CP-11 (A): billing retoma los eventos pendientes — `evidencias/kafka/CP-11.txt`

```
==================== CP-11 Desacoplamiento temporal: eventos publicados con el consumidor apagado ====================
# Ejecutado: 2026-10-30 12:51:39 (hora local) / 17:51:39 UTC

# A) billing-consumer vuelve y procesa lo que se publicó mientras estaba apagado
LAG antes: 3

$ docker compose start billing-consumer 2>&1
 Container kafka Waiting
 Container kafka Healthy
 Container kafka-init Starting
 Container kafka-init Started
 Container kafka-init Waiting
 Container kafka-init Exited
 Container kafka-pedidos-billing-consumer-1 Starting
 Container kafka-pedidos-billing-consumer-1 Started

$ docker compose logs --no-log-prefix --since 2026-10-30T17:51:41Z billing-consumer | grep -E 'Unido|Evento recibido|offset=|Fa
ctura'
2026-10-30T17:51:45.778Z [billing-consumer@286de68024c8] Unido al consumer group "billing-service-group". Particiones asignadas
de orders.events: [0,1,2]
2026-10-30T17:51:45.791Z [billing-consumer@286de68024c8] Evento recibido: OrderCreated orderId=ORD-40AECA0F eventId=evt-21aa280
4-73ec-48da-9067-92df12513838 correlationId=corr-0b7e56a0-8b09-441c-810a-6c4e4d6cb070
2026-10-30T17:51:45.791Z [billing-consumer@286de68024c8] topic=orders.events partition=0 offset=7 key=ORD-40AECA0F
2026-10-30T17:51:46.791Z [billing-consumer@286de68024c8] Factura generada por $150000 (3 x $50000) (orderId=ORD-40AECA0F)
2026-10-30T17:51:46.795Z [billing-consumer@286de68024c8] Evento recibido: OrderCreated orderId=ORD-41652E7F eventId=evt-d14c141
c-63a3-4b9f-b497-f8b55cacf9ad correlationId=corr-46b722b8-97de-440b-af5a-d006c2304572
2026-10-30T17:51:46.796Z [billing-consumer@286de68024c8] topic=orders.events partition=2 offset=4 key=ORD-41652E7F
2026-10-30T17:51:47.794Z [billing-consumer@286de68024c8] Factura generada por $50000 (1 x $50000) (orderId=ORD-41652E7F)
2026-10-30T17:51:47.794Z [billing-consumer@286de68024c8] Evento recibido: OrderCreated orderId=ORD-2BAF2935 eventId=evt-b93a18e
a-ed93-425e-bb93-54f1b12ddd76 correlationId=corr-68583954-01c9-4125-81e2-5a4f2f975c46
```

```
2026-10-30T17:51:47.795Z [billing-consumer@286de68024c8] topic=orders.events partition=2 offset=5 key=ORD-2BAF2935
2026-10-30T17:51:48.795Z [billing-consumer@286de68024c8] Factura generada por $100000 (2 x $50000) (orderId=ORD-2BAF2935)
GROUP                 TOPIC           PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG             CONSUMER-ID
HOST            CLIENT-ID
billing-service-group orders.events   0          8               8               0               billing-consumer-7a329260-427a
-40a3-b824-eea7e5a3f3d2 /172.20.0.5     billing-consumer
billing-service-group orders.events   1          5               5               0               billing-consumer-7a329260-427a
-40a3-b824-eea7e5a3f3d2 /172.20.0.5     billing-consumer
billing-service-group orders.events   2          6               6               0               billing-consumer-7a329260-427a
-40a3-b824-eea7e5a3f3d2 /172.20.0.5     billing-consumer
```

Ejecución CP-11 (B): consumidor posterior lee el log desde el inicio

evidencias/kafka/CP-11-posterior.txt

```
==================== CP-11 (B) Consumidor posterior: iniciado después de publicar los eventos, lee desde el log ==============
=====
# Ejecutado: 2026-10-30 12:58:33 (hora local) / 17:58:33 UTC

# audit-consumer (grupo audit-service-group) no existía cuando se publicaron los eventos

$ docker exec kafka kafka-consumer-groups.sh --bootstrap-server kafka:9092 --list
billing-service-group
inventory-service-group
notification-service-group

# Total de eventos en el log del tópico (topic:partición:offset final):
orders.events:0:8
orders.events:1:5
orders.events:2:6

$ docker compose --profile late up -d audit-consumer 2>&1 | grep -E 'Created|Started'
 Container kafka-pedidos-audit-consumer-1 Created
 Container kafka-init Started
 Container kafka-pedidos-audit-consumer-1 Started

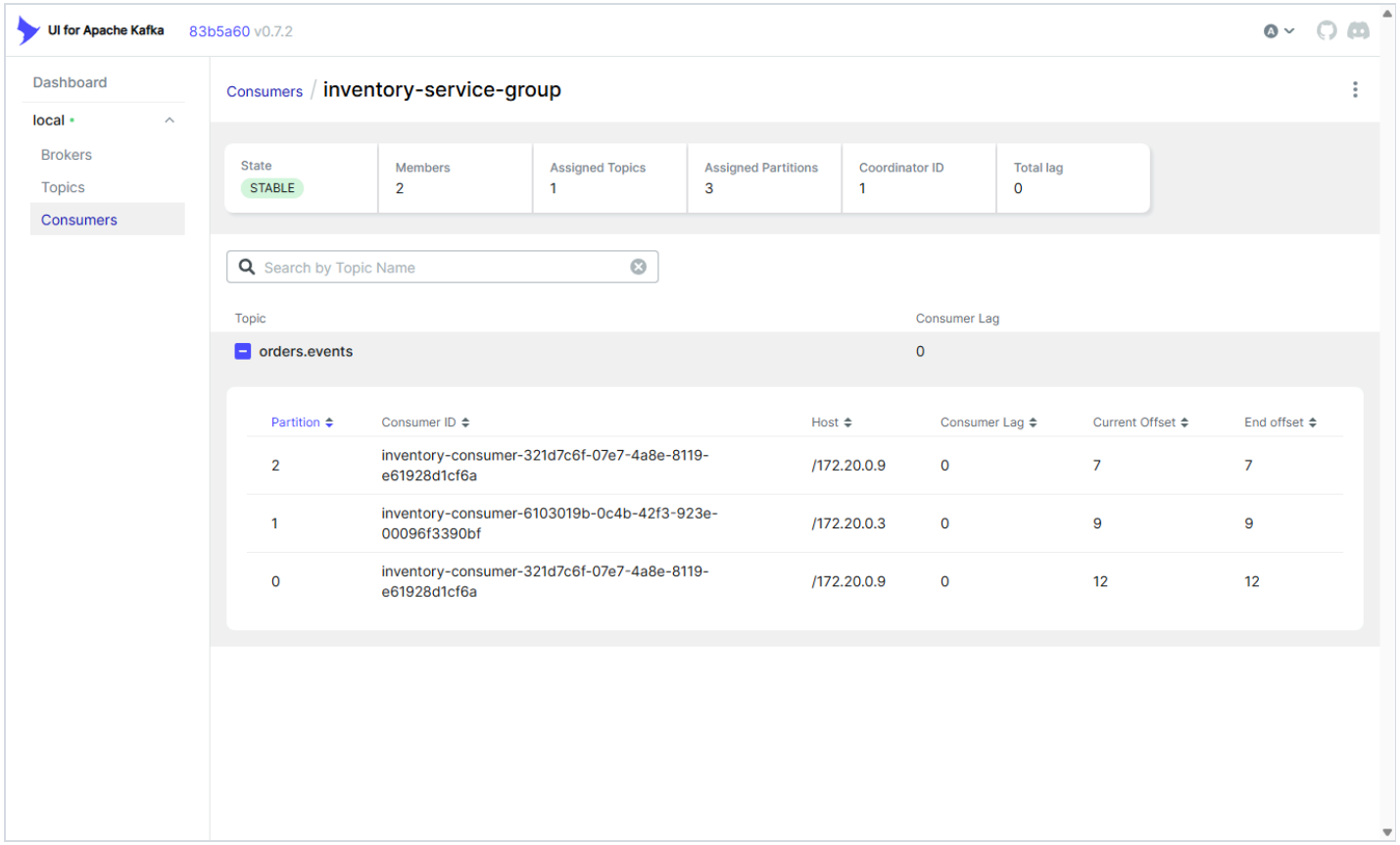
$ docker compose logs --no-log-prefix audit-consumer | grep -E 'Unido|offset='
2026-10-30T17:58:41.037Z [audit-consumer@8cd983ae13f5] Unido al consumer group "audit-service-group". Particiones asignadas de
orders.events: [0,1,2]
2026-10-30T17:58:41.056Z [audit-consumer@8cd983ae13f5] topic=orders.events partition=2 offset=0 key=ORD-D640D38E
2026-10-30T17:58:42.056Z [audit-consumer@8cd983ae13f5] topic=orders.events partition=2 offset=1 key=ORD-5EC14BDC
2026-10-30T17:58:43.056Z [audit-consumer@8cd983ae13f5] topic=orders.events partition=2 offset=2 key=ORD-F8C14D9C
2026-10-30T17:58:44.059Z [audit-consumer@8cd983ae13f5] topic=orders.events partition=2 offset=3 key=ORD-2B12202C
2026-10-30T17:58:45.059Z [audit-consumer@8cd983ae13f5] topic=orders.events partition=2 offset=4 key=ORD-41652E7F
2026-10-30T17:58:46.059Z [audit-consumer@8cd983ae13f5] topic=orders.events partition=2 offset=5 key=ORD-2BAF2935
2026-10-30T17:58:47.064Z [audit-consumer@8cd983ae13f5] topic=orders.events partition=1 offset=0 key=ORD-296E793F
2026-10-30T17:58:48.064Z [audit-consumer@8cd983ae13f5] topic=orders.events partition=1 offset=1 key=ORD-11746766
2026-10-30T17:58:49.064Z [audit-consumer@8cd983ae13f5] topic=orders.events partition=1 offset=2 key=ORD-99EDE493
2026-10-30T17:58:50.066Z [audit-consumer@8cd983ae13f5] topic=orders.events partition=1 offset=3 key=ORD-6AF90985
2026-10-30T17:58:51.066Z [audit-consumer@8cd983ae13f5] topic=orders.events partition=1 offset=4 key=ORD-8EEC0154
2026-10-30T17:58:52.068Z [audit-consumer@8cd983ae13f5] topic=orders.events partition=0 offset=0 key=ORD-702E978A
2026-10-30T17:58:53.070Z [audit-consumer@8cd983ae13f5] topic=orders.events partition=0 offset=1 key=ORD-EA5FF83E
2026-10-30T17:58:54.070Z [audit-consumer@8cd983ae13f5] topic=orders.events partition=0 offset=2 key=ORD-F16975F4
2026-10-30T17:58:55.071Z [audit-consumer@8cd983ae13f5] topic=orders.events partition=0 offset=3 key=ORD-9B9438D9
2026-10-30T17:58:56.072Z [audit-consumer@8cd983ae13f5] topic=orders.events partition=0 offset=4 key=ORD-087CE414
2026-10-30T17:58:57.072Z [audit-consumer@8cd983ae13f5] topic=orders.events partition=0 offset=5 key=ORD-745BE995
2026-10-30T17:58:58.073Z [audit-consumer@8cd983ae13f5] topic=orders.events partition=0 offset=6 key=ORD-F61A3093
2026-10-30T17:58:59.075Z [audit-consumer@8cd983ae13f5] topic=orders.events partition=0 offset=7 key=ORD-40AECA0F

$ echo "Eventos leídos por audit-consumer: $(docker compose logs --no-log-prefix audit-consumer | grep -c 'Evento recibido')"
Eventos leídos por audit-consumer: 19
GROUP                 TOPIC           PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG             CONSUMER-ID
HOST            CLIENT-ID
audit-service-group orders.events   0          8               8               0               audit-consumer-09382be2-368b-435
9-9f43-f73017de835a /172.20.0.8     audit-consumer
audit-service-group orders.events   1          5               5               0               audit-consumer-09382be2-368b-435
9-9f43-f73017de835a /172.20.0.8     audit-consumer
audit-service-group orders.events   2          6               6               0               audit-consumer-09382be2-368b-435
9-9f43-f73017de835a /172.20.0.8     audit-consumer
```

CP-10 · Elastic: distribución de carga con varias instancias del mismo grupo Kafka

ID del caso	CP-10
Arquitectura	Kafka
Objetivo	Comprobar que dos instancias del mismo consumidor (mismo grupo) se reparten las particiones.
Precondiciones	• Stack `kafka/docker-compose.yml` en ejecución (kafka, kafka-init, kafka-ui, orders-api, inventory/billing/notification-consumer). • Tópico con 3 particiones.
Datos de entrada	9 pedidos (Escala1…9), cada uno con su orderId como key.
Pasos ejecutados	1. `docker compose up -d --scale inventory-consumer=2`. 2. Describir los miembros del grupo. 3. Enviar 9 pedidos. 4. Contar los eventos procesados por cada instancia y partición. 5. Verificar que billing y notification recibieron los 9.
Resultado esperado	Las 3 particiones se reparten entre las 2 instancias; cada evento lo procesa una sola instancia del grupo; los demás grupos reciben los 9.
Resultado obtenido	Miembros: una instancia con 2 particiones (0 y 2) y otra con 1 (partición 1). Reparto: a2c481ced28e procesó 4 eventos de la partición 0 y 1 de la partición 2; d5d9dfbc2cc6 procesó 4 de la partición 1. Total 9, sin duplicados. billing y notification: 9 cada uno.
Estado	**Cumple**
Análisis técnico	En Kafka el paralelismo de un grupo está limitado por el número de particiones (por eso el tópico tiene 3). El reparto es por partición, no por mensaje: depende del hash de la key, por lo que no es exactamente equitativo (5 frente a 4).

Evidencia



Kafka UI: inventory-service-group con 2 miembros y las particiones repartidas (`kafka/img/11-cp10-inventory-2-miembros.png`)

Ejecución CP-10

`evidencias/kafka/CP-10.txt`

```
==================== CP-10 Elastic: distribución de carga con varias instancias del mismo grupo ====================
# Ejecutado: 2026-10-30 12:59:09 (hora local) / 17:59:09 UTC

$ docker compose up -d --scale inventory-consumer=2 2>&1 | grep -E 'Started|Running' | grep inventory
 Container kafka-pedidos-inventory-consumer-1 Running
 Container kafka-pedidos-inventory-consumer-2 Started

$ docker compose ps inventory-consumer --format 'table {{.Name}}\t{{.Status}}'
NAME                                STATUS
kafka-pedidos-inventory-consumer-1   Up 11 minutes
kafka-pedidos-inventory-consumer-2   Up 8 seconds

# Particiones repartidas entre las 2 instancias del grupo inventory-service-group
GROUP                  CONSUMER-ID                                                HOST            CLIENT-ID          #PARTITIONS
inventory-service-group inventory-consumer-6103019b-0c4b-42f3-923e-00096f3390bf /172.20.0.3     inventory-consumer 1
inventory-service-group inventory-consumer-321d7c6f-07e7-4a8e-8119-e61928d1cf6a /172.20.0.9     inventory-consumer 2

# Se envían 9 pedidos (cada uno con su propia key = orderId)
Pedido 1:{"status":"accepted","message":"Pedido recibido y evento OrderCreated publicado en Kafka","orderId":"ORD-F8F5CCA3","ev
entId":"evt-1cabcf57-daf8-4f62-9730-ca25ef2e0c8f","correlationId":"corr-7ec9e852-e83d-41f2-bc98-70d255ffe66c"}  -> HTTP 202 en
0.007800s
Pedido 2:{"status":"accepted","message":"Pedido recibido y evento OrderCreated publicado en Kafka","orderId":"ORD-0A46DCA1","ev
entId":"evt-da2f2140-0ce9-4638-9a2d-a2f30118b4cb","correlationId":"corr-cd7073aa-7acb-4048-9c94-9bc986254dc8"}  -> HTTP 202 en
0.005156s
Pedido 3:{"status":"accepted","message":"Pedido recibido y evento OrderCreated publicado en Kafka","orderId":"ORD-C4CFA81A","ev
entId":"evt-4d3c8d2b-a31b-4934-afae-f6425d70b74d","correlationId":"corr-24f6e894-5481-4901-b224-a0d1e805c7bf"}  -> HTTP 202 en
0.004919s
Pedido 4:{"status":"accepted","message":"Pedido recibido y evento OrderCreated publicado en Kafka","orderId":"ORD-7F0D2378","ev
entId":"evt-193bfd8e-7ce0-46d8-b64c-1cbcc45e626a","correlationId":"corr-3222939e-c912-4b7c-96ae-8dca8e61d63c"}  -> HTTP 202 en
0.004705s
```

```
Pedido 5:{"status":"accepted","message":"Pedido recibido y evento OrderCreated publicado en Kafka","orderId":"ORD-9A0DEF8E","ev
entId":"evt-18a28828-40d9-4f16-8ddd-9b73e8f1929d","correlationId":"corr-cb51dc80-d54b-478b-8626-c3dfd6c2b8ca"}  -> HTTP 202 en
0.004493s
Pedido 6:{"status":"accepted","message":"Pedido recibido y evento OrderCreated publicado en Kafka","orderId":"ORD-64C15D6E","ev
entId":"evt-5307edfa-398d-4a55-8bda-ec16cdcc8287","correlationId":"corr-46540780-1c41-4fd8-b72d-7b2aaf83ac74"}  -> HTTP 202 en
0.004548s
Pedido 7:{"status":"accepted","message":"Pedido recibido y evento OrderCreated publicado en Kafka","orderId":"ORD-24C0B7FB","ev
entId":"evt-db87ceb5-4681-40ee-bcf9-15b886d4006b","correlationId":"corr-1f3999f6-7dc8-45fd-b600-180a5b8fd3e9"}  -> HTTP 202 en
0.004660s
Pedido 8:{"status":"accepted","message":"Pedido recibido y evento OrderCreated publicado en Kafka","orderId":"ORD-86B0E479","ev
entId":"evt-0f2c6818-8544-4887-a5a0-a06b69c873a0","correlationId":"corr-b6c92637-ffdd-4912-b888-e13b3a6a0cc5"}  -> HTTP 202 en
0.004641s
Pedido 9:{"status":"accepted","message":"Pedido recibido y evento OrderCreated publicado en Kafka","orderId":"ORD-4FE785C9","ev
entId":"evt-d3bb7178-de0d-46ba-9b3d-9d6c422b4af2","correlationId":"corr-a93f2233-25ce-4fc1-af0b-9ba69c7f9c24"}  -> HTTP 202 en
0.004649s

# Eventos procesados por cada instancia (instancia, partición)

$ docker compose logs --no-log-prefix --since 2026-10-30T17:59:24Z inventory-consumer | grep 'topic=' | sed -E 's/.*\[(inventor
y-consumer@[a-f0-9]+)\].*partition=([0-9]+).*/\1 partición \2/' | sort | uniq -c
      4 inventory-consumer@a2c481ced28e partición 0
      1 inventory-consumer@a2c481ced28e partición 2
      4 inventory-consumer@d5d9dfbc2cc6 partición 1

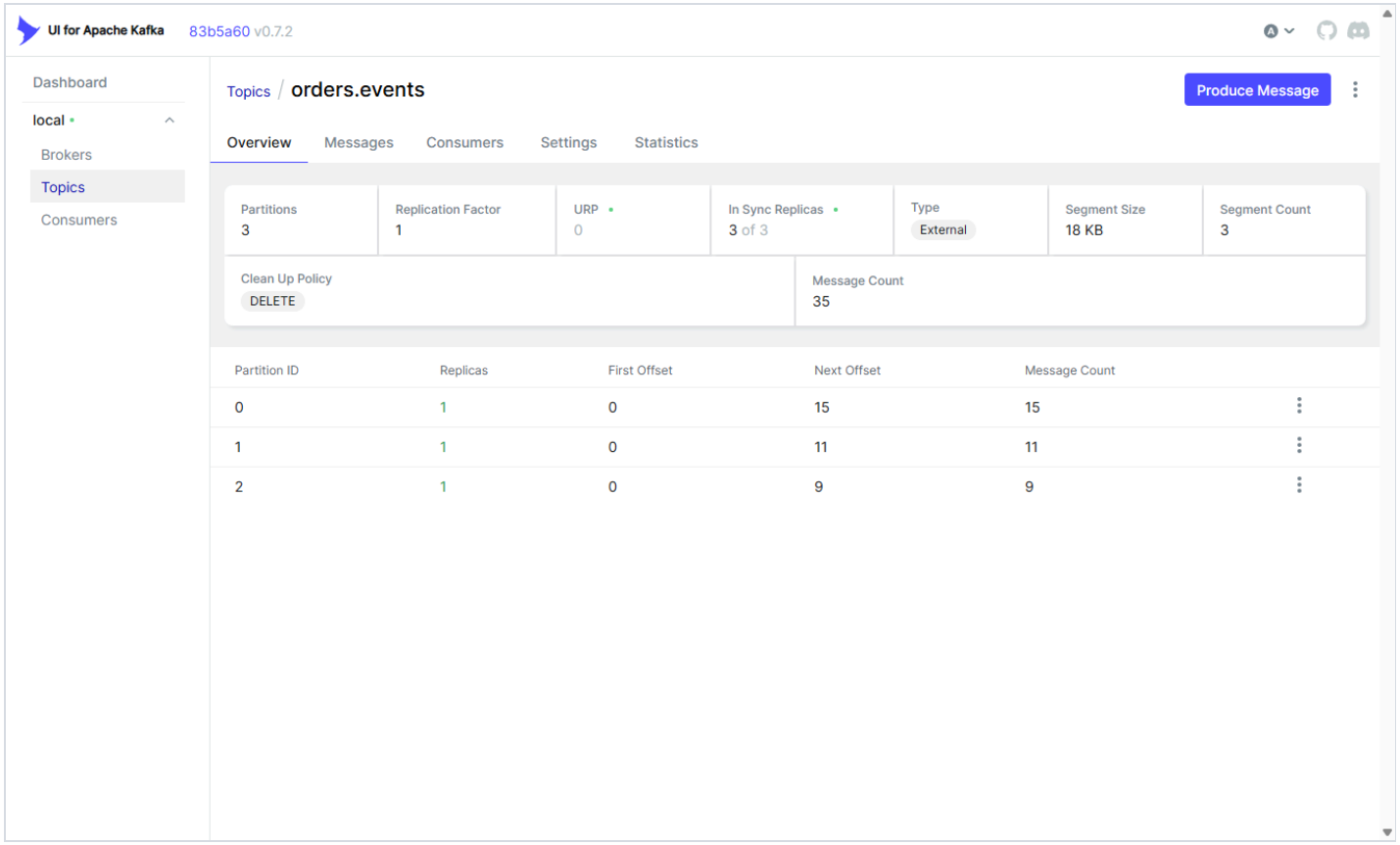
# Los demás grupos (billing, notification) recibieron los 9 eventos completos

$ docker compose logs --no-log-prefix --since 2026-10-30T17:59:24Z billing-consumer | grep -c 'Evento recibido'; docker compose
logs --no-log-prefix --since 2026-10-30T17:59:24Z notification-consumer | grep -c 'Evento recibido'
9
9
GROUP                   TOPIC           PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG             CONSUMER-ID
HOST            CLIENT-ID
inventory-service-group orders.events   0          12              12              0               inventory-consumer-321d7c6f-
07e7-4a8e-8119-e61928d1cf6a /172.20.0.9     inventory-consumer
inventory-service-group orders.events   2          7               7               0               inventory-consumer-321d7c6f-
07e7-4a8e-8119-e61928d1cf6a /172.20.0.9     inventory-consumer
inventory-service-group orders.events   1          9               9               0               inventory-consumer-6103019b-
0c4b-42f3-923e-00096f3390bf /172.20.0.3     inventory-consumer
```

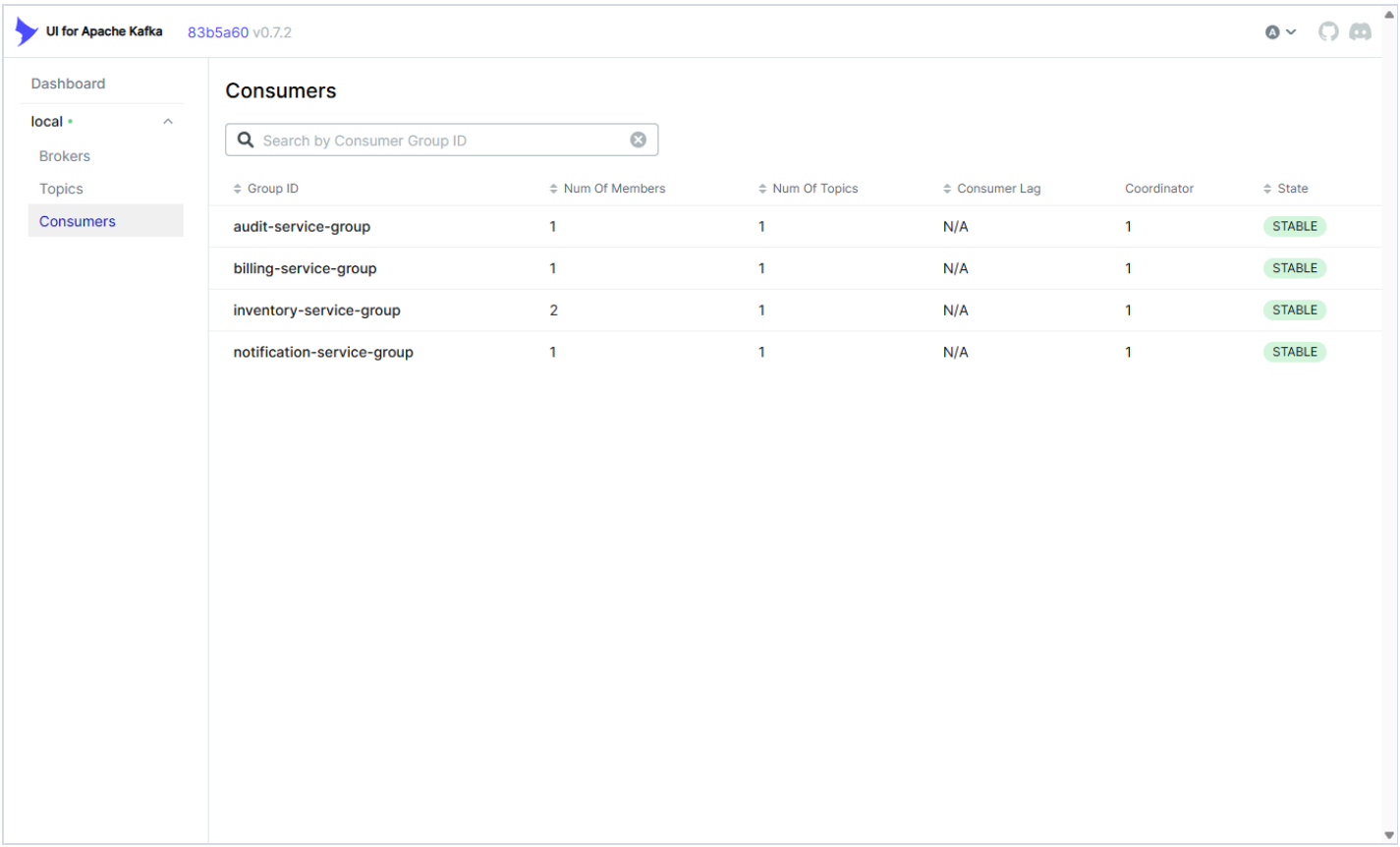
CP-12 · Recuperación tras reinicio del consumidor y del broker Kafka

ID del caso	CP-12
Arquitectura	Kafka
Objetivo	Verificar que tras una caída abrupta del consumidor o un reinicio del broker los eventos pendientes se procesan sin pérdida.
Precondiciones	• Stack `kafka/docker-compose.yml` en ejecución (kafka, kafka-init, kafka-ui, orders-api, inventory/billing/notification-consumer). • inventory-consumer escalado a 2 instancias (CP-10).
Datos de entrada	A) 3 pedidos (Recupera1…3). B) 3 pedidos (Broker1…3) + 1 tras el reinicio (Broker4).
Pasos ejecutados	1. A) `docker kill` de notification-consumer, enviar 3 pedidos, describir el grupo y volver a levantarlo. 2. B) Detener billing-consumer, enviar 3 pedidos, guardar los offsets y reiniciar el broker (`docker compose restart kafka`). 3. B) Comparar los offsets antes y después, enviar un pedido nuevo y levantar billing. 4. C) Analizar el LAG residual de inventory-service-group.
Resultado esperado	A) notification procesa los 3 eventos al volver. B) El log y los offsets confirmados se conservan; billing procesa los 4 eventos. Ningún evento se pierde.
Resultado obtenido	A) LAG 3 mientras estaba caído; al volver procesó los 3 (p2 o7, p2 o8, p0 o12). Tardó ~26 s en unirse porque el broker esperó a que expirara la sesión del miembro anterior, que murió sin despedirse. B) Offsets 15/10/9 antes y después del reinicio; el grupo billing conservó sus offsets confirmados (LAG 3); la API publicó Broker4 tras el reinicio; billing procesó los 4. C) Una instancia de inventario procesó el evento p0 o14 (ORD-05CEDD1E) 0,2 s antes de que el broker se detuviera, pero el commit de su offset no llegó: el grupo quedó con LAG=1. Al reiniciar esa instancia, el evento se volvió a entregar y procesar (duplicado) y el LAG quedó en 0.
Estado	**Cumple**
Análisis técnico	No hubo pérdida de eventos: el log es persistente (volumen `kafka-data`) y cada grupo retoma desde su último offset confirmado. La parte C muestra la semántica *al menos una vez* de Kafka con commit posterior al procesamiento: si el commit se pierde, el evento se reprocesa. Es un hallazgo relevante de QA: los consumidores deben ser idempotentes (por ejemplo, registrando los `eventId` ya procesados) para que un duplicado no reserve inventario dos veces.

Evidencia



Kafka UI: el tópico conserva sus mensajes tras el reinicio del broker `(kafka/img/13-cp12-topico-tras-reinicio.png)`



Kafka UI: estado final de los cuatro grupos (STABLE) (`kafka/img/12-cp12-estado-final-grupos.png`)

Ejecución CP-12 (A y B) — `evidencias/kafka/CP-12.txt`

```
==================== CP-12 Recuperación tras reinicio del consumidor y del broker ====================
# Ejecutado: 2026-10-30 12:59:55 (hora local) / 17:59:55 UTC

# A) Caída abrupta (SIGKILL) de notification-consumer

$ docker kill $(docker compose ps -q notification-consumer)
38db5bcd9c8ac61d8f5adfbc431089786fc9e8a30f913443674e32339f3b0602
Pedido 1:{"status":"accepted","message":"Pedido recibido y evento OrderCreated publicado en Kafka","orderId":"ORD-1114F872","ev
entId":"evt-e4ae1539-6735-4c50-adee-ff7e7ee0f131","correlationId":"corr-2cebb4c0-e2bb-4dd6-98ac-633067526580"}  -> HTTP 202 en
0.005357s
Pedido 2:{"status":"accepted","message":"Pedido recibido y evento OrderCreated publicado en Kafka","orderId":"ORD-B642EFB1","ev
entId":"evt-36200661-17a8-4a5e-848f-eb2fc03575bd","correlationId":"corr-4fe26336-ba8f-428f-9141-6611edee9a08"}  -> HTTP 202 en
0.004711s
Pedido 3:{"status":"accepted","message":"Pedido recibido y evento OrderCreated publicado en Kafka","orderId":"ORD-F6E620BE","ev
entId":"evt-7f38c655-b9f4-4435-ac1a-67fab26ffb2c","correlationId":"corr-8fee9323-c8e0-4327-b5ae-efa329060997"}  -> HTTP 202 en
0.004635s

# Eventos pendientes para el grupo de notificaciones
GROUP                     TOPIC           PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG             CONSUMER-ID
HOST            CLIENT-ID
notification-service-group orders.events   0          12              13              1               notification-consumer-126
a305a-eefe-4e5a-9513-f6b3afab2ae3 /172.20.0.6     notification-consumer
notification-service-group orders.events   1          9               9               0               notification-consumer-126
a305a-eefe-4e5a-9513-f6b3afab2ae3 /172.20.0.6     notification-consumer
notification-service-group orders.events   2          7               9               2               notification-consumer-126
a305a-eefe-4e5a-9513-f6b3afab2ae3 /172.20.0.6     notification-consumer

$ docker compose up -d notification-consumer 2>&1 | tail -n 1
 Container kafka-pedidos-notification-consumer-1 Started

$ docker compose logs --no-log-prefix --since 2026-10-30T17:59:57Z notification-consumer | grep -E 'Unido|Evento recibido|offse
t='
```

```
2026-10-30T18:00:23.557Z [notification-consumer@38db5bcd9c8a] Unido al consumer group "notification-service-group". Particiones
asignadas de orders.events: [0,1,2]
2026-10-30T18:00:23.573Z [notification-consumer@38db5bcd9c8a] Evento recibido: OrderCreated orderId=ORD-1114F872 eventId=evt-e4
ae1539-6735-4c50-adee-ff7e7ee0f131 correlationId=corr-2cebb4c0-e2bb-4dd6-98ac-633067526580
2026-10-30T18:00:23.573Z [notification-consumer@38db5bcd9c8a] topic=orders.events partition=2 offset=7 key=ORD-1114F872
2026-10-30T18:00:24.576Z [notification-consumer@38db5bcd9c8a] Evento recibido: OrderCreated orderId=ORD-B642EFB1 eventId=evt-36
200661-17a8-4a5e-848f-eb2fc03575bd correlationId=corr-4fe26336-ba8f-428f-9141-6611edee9a08
2026-10-30T18:00:24.576Z [notification-consumer@38db5bcd9c8a] topic=orders.events partition=2 offset=8 key=ORD-B642EFB1
2026-10-30T18:00:25.581Z [notification-consumer@38db5bcd9c8a] Evento recibido: OrderCreated orderId=ORD-F6E620BE eventId=evt-7f
38c655-b9f4-4435-ac1a-67fab26ffb2c correlationId=corr-8fee9323-c8e0-4327-b5ae-efa329060997
2026-10-30T18:00:25.581Z [notification-consumer@38db5bcd9c8a] topic=orders.events partition=0 offset=12 key=ORD-F6E620BE
GROUP                     TOPIC           PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG             CONSUMER-ID
HOST            CLIENT-ID
notification-service-group orders.events   0          13              13              0               notification-consumer-8d9
26066-863b-406c-86e3-abe13fe9884b /172.20.0.6     notification-consumer
notification-service-group orders.events   1          9               9               0               notification-consumer-8d9
26066-863b-406c-86e3-abe13fe9884b /172.20.0.6     notification-consumer
notification-service-group orders.events   2          9               9               0               notification-consumer-8d9
26066-863b-406c-86e3-abe13fe9884b /172.20.0.6     notification-consumer

# B) Reinicio del broker con eventos pendientes para billing-consumer

$ docker compose stop billing-consumer 2>&1
 Container kafka-pedidos-billing-consumer-1 Stopping
 Container kafka-pedidos-billing-consumer-1 Stopped
Pedido 1:{"status":"accepted","message":"Pedido recibido y evento OrderCreated publicado en Kafka","orderId":"ORD-B6DE9271","ev
entId":"evt-9eb03633-fc54-4ca7-97b7-54be0b7acb18","correlationId":"corr-665e14be-4451-4417-841d-5eaf70e9ce35"}  -> HTTP 202 en
0.004678s
Pedido 2:{"status":"accepted","message":"Pedido recibido y evento OrderCreated publicado en Kafka","orderId":"ORD-05CEDD1E","ev
entId":"evt-0d8f0b45-f1a1-4866-869f-5dd41944c3bd","correlationId":"corr-6ed1e22e-14a6-4ad9-8c94-f2013f32a276"}  -> HTTP 202 en
0.004131s
Pedido 3:{"status":"accepted","message":"Pedido recibido y evento OrderCreated publicado en Kafka","orderId":"ORD-103609A4","ev
entId":"evt-062d3503-43ea-4a37-bf60-9e8484912151","correlationId":"corr-31521dc2-d866-44eb-ae99-750804b12484"}  -> HTTP 202 en
0.004450s

# Offsets del tópico antes del reinicio
orders.events:0:15
orders.events:1:10
orders.events:2:9

$ docker compose restart kafka 2>&1
 Container kafka Restarting
 Container kafka Started

# Offsets del tópico después del reinicio: el log se conservó
orders.events:0:15
orders.events:1:10
orders.events:2:9

# Offsets confirmados del grupo de facturación tras el reinicio (eventos pendientes intactos)
GROUP                TOPIC           PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG             CONSUMER-ID
HOST            CLIENT-ID
billing-service-group orders.events   0          13              15              2               billing-consumer-7a329260-427a
-40a3-b824-eea7e5a3f3d2 /172.20.0.5     billing-consumer
billing-service-group orders.events   1          9               10              1               billing-consumer-7a329260-427a
-40a3-b824-eea7e5a3f3d2 /172.20.0.5     billing-consumer
billing-service-group orders.events   2          9               9               0               billing-consumer-7a329260-427a
-40a3-b824-eea7e5a3f3d2 /172.20.0.5     billing-consumer

# La API sigue publicando después del reinicio
Pedido 4:{"status":"accepted","message":"Pedido recibido y evento OrderCreated publicado en Kafka","orderId":"ORD-8034DC94","ev
entId":"evt-3e0de9a7-e47e-4a96-9410-06fac8caeeea","correlationId":"corr-e0ae2127-35b2-4c7d-98e8-e8d299168e7f"}  -> HTTP 202 en
0.011473s

$ docker compose start billing-consumer 2>&1
 Container kafka Waiting
 Container kafka Healthy
 Container kafka-init Starting
 Container kafka-init Started
 Container kafka-init Waiting
```

```
 Container kafka-init Exited
 Container kafka-pedidos-billing-consumer-1 Starting
 Container kafka-pedidos-billing-consumer-1 Started

$ docker compose logs --no-log-prefix --since 2026-10-30T18:00:48Z billing-consumer | grep -E 'Unido|Evento recibido'
2026-10-30T18:01:09.225Z [billing-consumer@286de68024c8] Unido al consumer group "billing-service-group". Particiones asignadas
de orders.events: [0,1,2]
2026-10-30T18:01:09.248Z [billing-consumer@286de68024c8] Evento recibido: OrderCreated orderId=ORD-103609A4 eventId=evt-062d350
3-43ea-4a37-bf60-9e8484912151 correlationId=corr-31521dc2-d866-44eb-ae99-750804b12484
2026-10-30T18:01:10.251Z [billing-consumer@286de68024c8] Evento recibido: OrderCreated orderId=ORD-8034DC94 eventId=evt-3e0de9a
7-e47e-4a96-9410-06fac8caeeea correlationId=corr-e0ae2127-35b2-4c7d-98e8-e8d299168e7f
2026-10-30T18:01:11.257Z [billing-consumer@286de68024c8] Evento recibido: OrderCreated orderId=ORD-B6DE9271 eventId=evt-9eb0363
3-fc54-4ca7-97b7-54be0b7acb18 correlationId=corr-665e14be-4451-4417-841d-5eaf70e9ce35
2026-10-30T18:01:12.257Z [billing-consumer@286de68024c8] Evento recibido: OrderCreated orderId=ORD-05CEDD1E eventId=evt-0d8f0b4
5-f1a1-4866-869f-5dd41944c3bd correlationId=corr-6ed1e22e-14a6-4ad9-8c94-f2013f32a276
Timeout esperando LAG=0 en inventory-service-group

# Estado final de todos los grupos
inventory-service-group LAG=1
billing-service-group LAG=0
notification-service-group LAG=0
audit-service-group LAG=0
```

CP-12 (C): análisis del LAG residual y reentrega

evidencias/kafka/CP-12-analisis-lag.txt

```
==================== CP-12 (C) Análisis del LAG residual en inventory-service-group ====================
# Ejecutado: 2026-10-30 13:07:16 (hora local) / 18:07:16 UTC

# Estado del grupo: la partición 0 tiene offset confirmado 14 y fin de log 15

GROUP                   TOPIC           PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG             CONSUMER-ID
HOST            CLIENT-ID
inventory-service-group orders.events   0          14              15              1               inventory-consumer-321d7c6f-
07e7-4a8e-8119-e61928d1cf6a /172.20.0.9     inventory-consumer
inventory-service-group orders.events   2          9               9               0               inventory-consumer-321d7c6f-
07e7-4a8e-8119-e61928d1cf6a /172.20.0.9     inventory-consumer
inventory-service-group orders.events   1          11              11              0               inventory-consumer-6103019b-
0c4b-42f3-923e-00096f3390bf /172.20.0.3     inventory-consumer

# La instancia a2c481ced28e sí procesó el offset 14 justo antes de que el broker se detuviera (18:00:33.497Z)
$ docker compose logs --no-log-prefix inventory-consumer | grep -E "partition=0 offset=14|ORD-05CEDD1E|ECONNREFUSED" | head -n
5
2026-10-30T18:00:32.332Z [inventory-consumer@a2c481ced28e] Evento recibido: OrderCreated orderId=ORD-05CEDD1E eventId=evt-0d8f0
b45-f1a1-4866-869f-5dd41944c3bd correlationId=corr-6ed1e22e-14a6-4ad9-8c9
2026-10-30T18:00:32.333Z [inventory-consumer@a2c481ced28e] topic=orders.events partition=0 offset=14 key=ORD-05CEDD1E
2026-10-30T18:00:33.332Z [inventory-consumer@a2c481ced28e] Inventario reservado: 1 x "Producto B2" (orderId=ORD-05CEDD1E)
{"level":"ERROR","timestamp":"2026-10-30T18:00:33.334Z","logger":"kafkajs","message":"[Connection] Connection error: connect EC
ONNREFUSED 172.20.0.2:9092","broker":"kafka:9092","clientId":"inventory-c

# Se reinicia la instancia: retoma desde el último offset confirmado (14) y vuelve a entregar ese evento
$ docker restart kafka-pedidos-inventory-consumer-2
kafka-pedidos-inventory-consumer-2
$ docker compose logs --no-log-prefix --since 2026-10-30T18:07:19Z inventory-consumer | grep -E 'Unido|Evento recibido|offset='
2026-10-30T18:07:24.441Z [inventory-consumer@d5d9dfbc2cc6] Unido al consumer group "inventory-service-group". Particiones asign
adas de orders.events: [0,2]
2026-10-30T18:07:24.443Z [inventory-consumer@d5d9dfbc2cc6] Evento recibido: OrderCreated orderId=ORD-05CEDD1E eventId=evt-0d8f0
b45-f1a1-4866-869f-5dd41944c3bd correlationId=corr-6ed1e22e-14a6-4ad9-8c94-f2013f32a276
2026-10-30T18:07:24.443Z [inventory-consumer@d5d9dfbc2cc6] topic=orders.events partition=0 offset=14 key=ORD-05CEDD1E
2026-10-30T18:07:24.442Z [inventory-consumer@a2c481ced28e] Unido al consumer group "inventory-service-group". Particiones asign
adas de orders.events: [1]

# Estado final del grupo

GROUP                   TOPIC           PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG             CONSUMER-ID
HOST            CLIENT-ID
inventory-service-group orders.events   0          15              15              0               inventory-consumer-6103019b-
0c4b-42f3-923e-00096f3390bf /172.20.0.3     inventory-consumer
inventory-service-group orders.events   2          9               9               0               inventory-consumer-6103019b-
0c4b-42f3-923e-00096f3390bf /172.20.0.3     inventory-consumer
```

```
inventory-service-group orders.events   1          11              11              0               inventory-consumer-fa2a13d9-
fb67-421f-9671-7de579ced1ae /172.20.0.9     inventory-consumer
```

6. Conclusiones

¿Qué arquitectura es más robusta desde QA?

Para este caso de estudio, **Kafka** resultó más robusta desde la perspectiva de QA, con un matiz importante. Lo que la hace más robusta:

- **Los datos no se consumen al leerlos.** Los 19 eventos seguían disponibles después de que los procesaran tres grupos, y un consumidor creado después los leyó todos (CP-11 B). En RabbitMQ, un mensaje confirmado desaparece: si un consumidor tiene un error de lógica, no hay forma de volver a procesar lo ya confirmado.
- **Aislamiento de fallos por grupo.** La caída de billing no afectó a inventory ni a notification (CP-09). Cada responsabilidad avanza a su ritmo con su propio offset.
- **Más observable para pruebas.** La pareja (partición, offset) identifica cada evento de forma única en la API, el broker y los consumidores (CP-06), y el LAG por grupo es una métrica directa de "cuánto falta por procesar" (CP-07, CP-09).

El matiz es que **RabbitMQ fue más predecible en la recuperación**: el mensaje no confirmado volvió a la cola al instante y se reentregó marcado como `redelivered=true` (CP-12). En Kafka, el consumidor que murió sin despedirse tardó ~26 s en recuperar sus particiones (expiración de la sesión), y el reinicio del broker dejó un commit perdido que produjo un evento procesado dos veces. Ambas arquitecturas garantizaron *cero pérdida* en todas las pruebas, pero ninguna garantiza *exactamente una vez*: en las dos el consumidor debe ser idempotente.

¿Qué dificultades se presentaron en pruebas?

- **Las métricas de la consola web de RabbitMQ llegan con retraso.** En la primera corrida, CP-02 mostró Ready=0 justo después de publicar, aunque el mensaje sí estaba en la cola. Se redujo el intervalo de estadísticas a 1 s y, para la evidencia textual, se usó `rabbitmqctl list_queues`, que consulta el estado real. Lección: en QA no se debe asumir que un panel muestra el estado instantáneo.
- **Las aserciones sobre sistemas asíncronos necesitan esperas explícitas.** No basta un `sleep` fijo: los scripts esperan condiciones (consumers=N, Total=0, LAG=0) con tiempo límite. Un error en la función que calculaba el LAG (trataba como 0 un grupo sin offsets confirmados) hizo que la primera corrida del consumidor posterior terminara antes de tiempo; se detectó porque el conteo de eventos no cuadraba con el tamaño del log, se corrigió y se repitió la prueba.
- **Las propias herramientas de inspección alteran lo observado.** Leer un mensaje desde la consola de RabbitMQ lo devuelve a la cola marcado como `redelivered` (CP-03), y `kafka-console-consumer` crea un consumer group temporal que aparece en Kafka UI (CP-05).
- **Los tiempos de rebalanceo de Kafka** (unión al grupo, expiración de sesión de un miembro caído) hacen que las pruebas de Kafka sean más lentas y menos deterministas que las de RabbitMQ.
- **El orden de arranque.** `depends_on` no espera a que el broker esté listo: fueron necesarios healthchecks, el contenedor `kafka-init` para crear el tópico antes que los consumidores, y reintentos de conexión en el código.
- **Entorno:** el puerto 3000 estaba ocupado en el equipo; se hizo configurable (sección 1). En Windows, Git Bash reescribía las rutas de los contenedores (`/var/tmp`), lo que se resolvió con `MSYS_NO_PATHCONV=1` .

¿Qué diferencias reales se evidenciaron?

Aspecto	RabbitMQ (evidencia)	Kafka (evidencia)
Modelo de entrega	Cola de trabajo: cada mensaje lo procesa un solo worker y se elimina con el ACK (CP-03, CP-10).	Log de eventos: cada grupo recibe todos los eventos; el mismo OrderCreated lo procesaron inventory, billing y notification (CP-05).
Reparto de carga	Por mensaje, en round-robin: 3/3/3 con 3 workers (CP-10).	Por partición: 5/4 con 2 instancias y 3 particiones; el paralelismo máximo es el número de particiones (CP-10).
Consumidor apagado	Los mensajes se acumulan como Ready (CP-09).	Los eventos ya están en el log; lo que crece es el LAG del grupo (CP-09).
Consumidor nuevo	Solo recibe lo que entre a la cola desde ese momento.	Puede leer toda la historia desde el offset 0 (CP-11 B).
Recuperación de un consumidor caído	Inmediata: el mensaje Unacked vuelve a Ready al cerrarse la conexión (CP-12 A).	Retardada ~26 s por la expiración de la sesión del miembro anterior (CP-12 A).
Duplicados observados	Mensaje reentregado tras matar el worker (redelivered=true).	Evento reprocesado tras perderse un commit durante el reinicio del broker (CP-12 C).
Observabilidad	Contadores Ready/Unacked/Total; la traza de mensajes requiere el plugin de tracing.	Offsets, particiones y LAG por grupo de forma nativa; contenido de los eventos en Kafka UI.
Tiempo de respuesta de la API	4–6 ms por pedido (CP-07).	5–8 ms por pedido (CP-07).

En resumen: RabbitMQ es adecuado para repartir *trabajo* (comandos como "procesar pedido") entre workers; Kafka es adecuado para difundir *hechos* (OrderCreated) a varios servicios que reaccionan de forma independiente y que pueden necesitar volver a leer la historia. En ambos casos la API respondió en milisegundos sin depender de los consumidores, que es la propiedad reactiva central evaluada.

7. Anexo: reproducción y archivos de evidencia

7.1 Cómo reproducir

```
# RabbitMQ
cd rabbitmq
docker compose up -d --build
./pruebas.sh todo            # o un caso: ./pruebas.sh CP-07
# consola: http://localhost:15672 (admin / admin)
docker compose down -v

# Kafka
cd kafka
docker compose up -d --build
./pruebas.sh todo            # incluye CP-11-posterior
# Kafka UI: http://localhost:8080
docker compose --profile late down -v

# Pedido manual (Postman o curl); usar 3000 si no existe el archivo .env
curl -X POST http://localhost:3010/orders -H "Content-Type: application/json" \
     -d '{"customerName":"Pedro","product":"Libro de arquitectura","quantity":1,"unitPrice":85000}'
```

Los scripts necesitan bash (Git Bash en Windows), curl, node y Docker. También se incluye la colección de Postman `postman/pedidos.postman_collection.json`.

7.2 Archivos de evidencia

Ruta	Contenido
`evidencias/rabbitmq/*.txt`	Salida de cada caso (entorno, CP-01…CP-12) y logs completos de orders-api, notification-worker y rabbitmq.
`evidencias/rabbitmq/img/`	Capturas de la consola de administración de RabbitMQ.
`evidencias/kafka/*.txt`	Salida de cada caso (entorno, CP-01…CP-12, CP-11-posterior, CP-12-analisis-lag) y logs completos de todos los servicios y del broker.
`evidencias/kafka/img/`	Capturas de Kafka UI.

Cada archivo de caso empieza con la fecha y hora de ejecución. Los `orderId`, `eventId` y `correlationId` citados en este documento pueden buscarse en los logs completos para verificar la trazabilidad.